

今さら聞けない量子誤り訂正

理化学研究所量子コンピュータ研究センター（RQC） 研究員
東京大学大学院情報理工学系研究科システム情報学専攻 特任助教
Nanofiber Quantum Technologies, Visiting Scientist
上野 洋典

上野 洋典 (うえの ようすけ)

● 経歴：

- 1994年 福岡県北九州市生まれ
 - 2013.4~2017.3: 東大工学部計数工学科システム情報工学コース
 - 2017.4~2022.3: 東大情報理工 システム情報学専攻
 - 指導教員：近藤正章先生（現慶應）、中村宏先生
 - 博論：超伝導デジタル回路を用いたオンライン量子誤り訂正
 - 2022.5~2023.2: ミュンヘン工科大学 訪問研究員
 - 2023.4~2026.3: 理研RQC 基礎科学特別研究員
 - 2024.7~現在: (兼務) NanoQT Visiting Scientist
 - 2025.5~現在: (兼務) 東大 情報理工 システム情報学専攻 特任助教（非常勤）
 - 2026.4~現在: (本務) 理研量子コンピュータ研究センター 研究員（鈴木チーム）
- 研究対象：量子計算機アーキテクチャ、超伝導デジタル回路
 - 物理の方々に囲まれつつ計算機アーキテクチャやっています！



ドイツでの最初と最後の
ビール

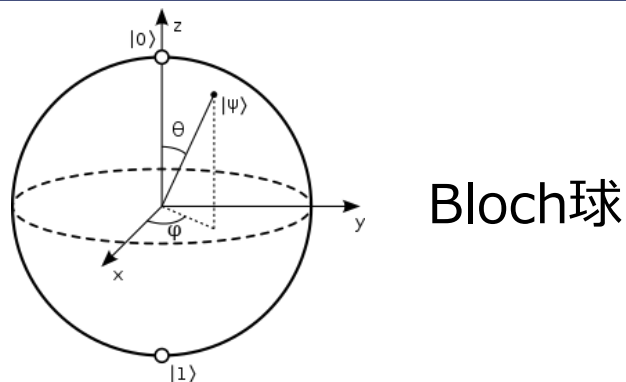
- 「量子計算機アーキテクチャ分野の研究動向」と題して明日の11時から喋ります！
 - (近しい内容の資料は上野のresearchmap or speakerdeckに上げていますので、「上野 量子計算機 アーキテクチャ」とかで検索してください)



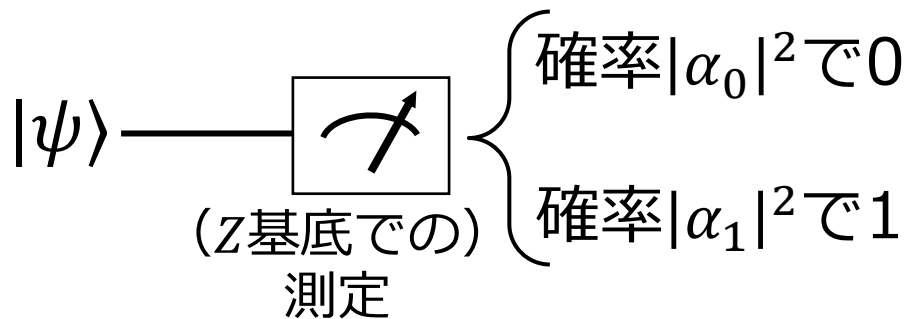
今日の内容

- 量子誤り訂正の基礎
 - 量子誤り訂正に求められる条件
 - 古典符号と量子符号
 - スタビライザ形式
 - Shorの符号
- 表面符号
 - 表面符号の誤り推定
 - Stim + Crumbleを用いた表面符号シミュレーション
 - 極低温環境での表面符号の誤り推定処理
- 表面符号 + 格子手術に基づく用いた誤り耐性量子計算
- まとめ

量子計算の基礎



- 量子ビット : $|\psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle$ (α_0, α_1 は複素数)



様々なゲート操作

Pauli gates

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = iXZ$$

Clifford gates

Hadamard (H) gate

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{\sqrt{2}} (X + Z)$$

Phase gate

$$S = \sqrt{Z} = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

Control-not (CNOT)

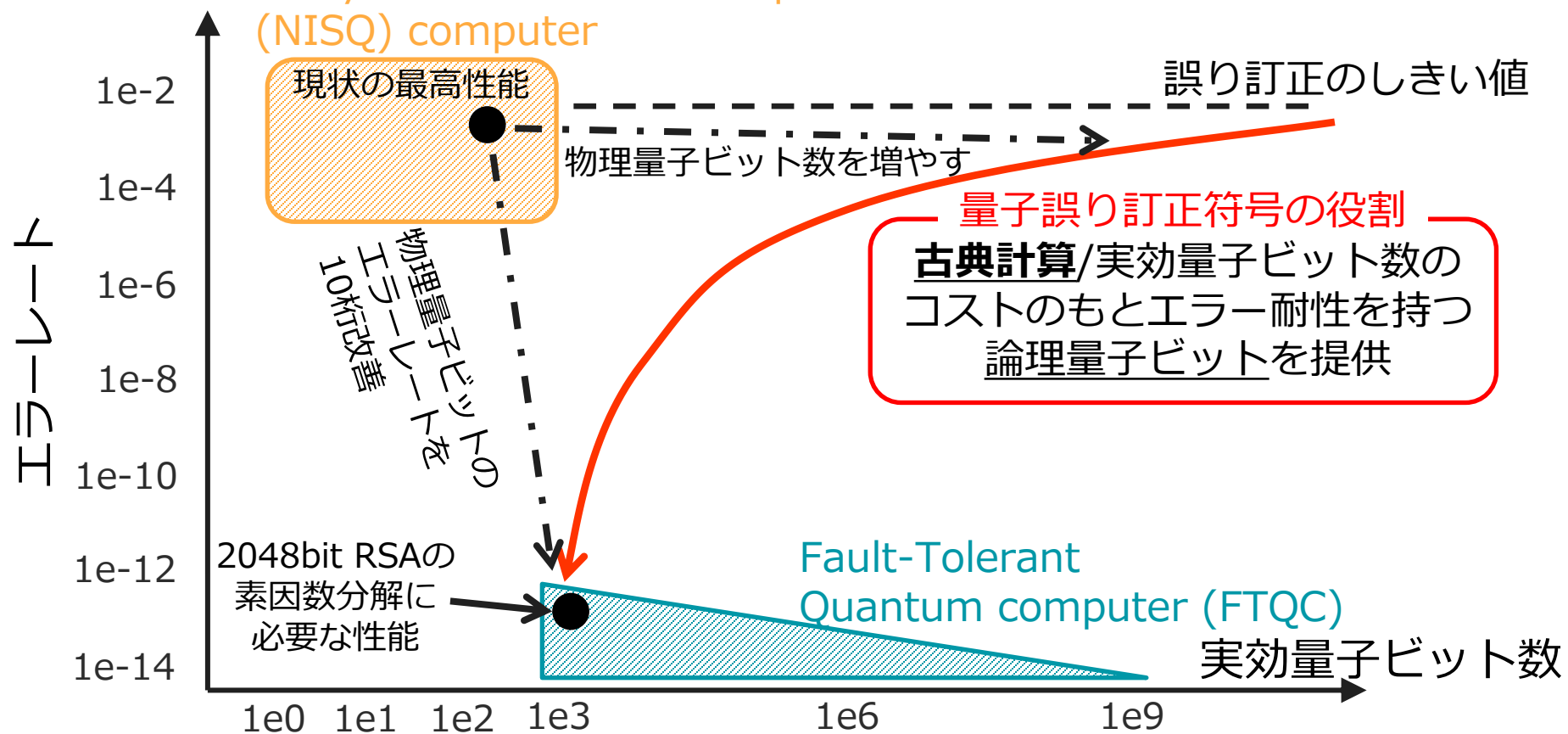
$$\text{CNOT} = \begin{pmatrix} I & 0 \\ 0 & X \end{pmatrix}$$

Non-Clifford gates

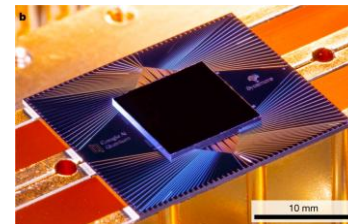
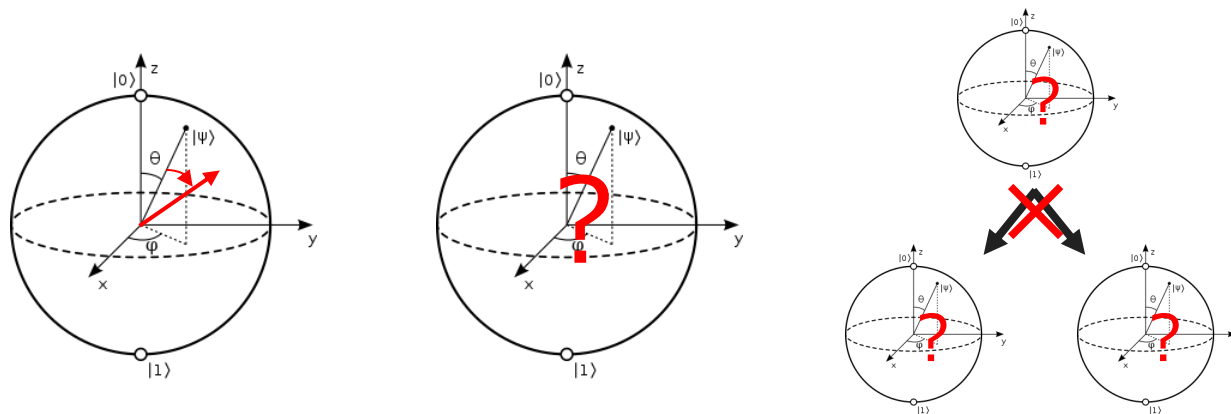
$$T = \sqrt{S} = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} \quad R_Y\left(\frac{\pi}{4}\right) = \begin{pmatrix} \cos \frac{\pi}{8} & -\sin \frac{\pi}{8} \\ \sin \frac{\pi}{8} & \cos \frac{\pi}{8} \end{pmatrix}$$

量子計算機の発展と量子誤り訂正の役割

Noisy-intermediate scale quantum (NISQ) computer



量子誤り訂正に求められる条件



[3] より

- 量子ビット数: 100~1000
- エラーレート: 0.1~1%

- ① 連続的な情報に生じたエラーを訂正できる
- ② 直接状態を観測することなく訂正できる
- ③ 未知の量子状態を複製することなく訂正できる
- (④ 実験的に達成できる誤り率を超えるしきい値を持つ)

[3] Frank Arute, Kunal Arya, Ryan Babbush, et al. Quantum supremacy using a programmable superconducting processor. Nature 574, 505–510 (2019).

連続的な情報に生じるエラー

$$|\psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle = \alpha_0 \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \alpha_1 \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$



量子ビットエラー E

(α_i, α'_i は複素数)

$$|\psi'\rangle = E|\psi\rangle := \alpha'_0|0\rangle + \alpha'_1|1\rangle = \alpha'_0 \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \alpha'_1 \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

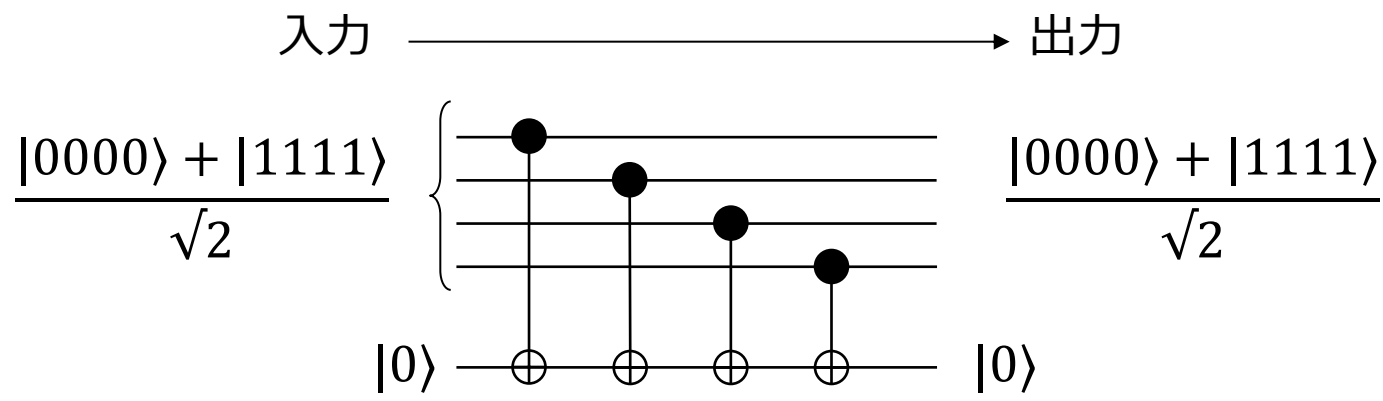
E を書き下すと

$$E = \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \frac{a+d}{2}I + \frac{a-d}{2}\underline{Z} + \frac{c+b}{2}\underline{X} + \frac{c-b}{2}\underline{XZ}$$

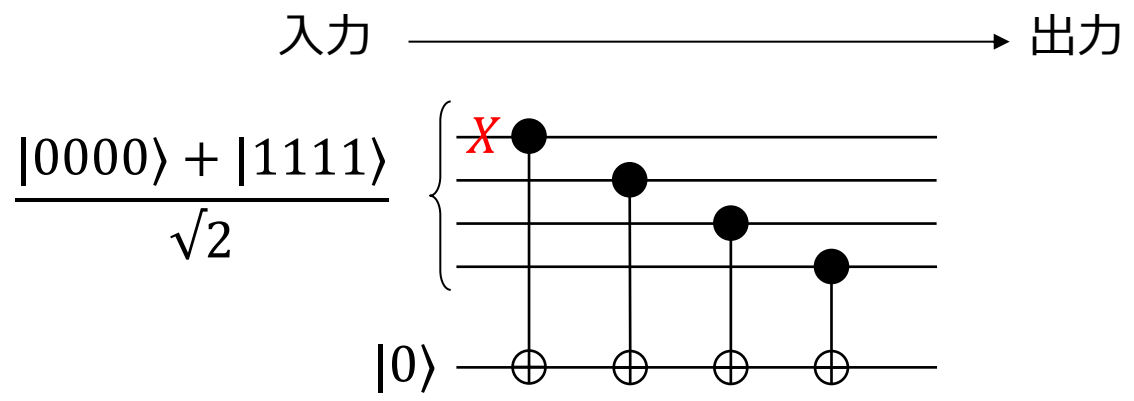
X はビット反転、 Z は位相反転を表す

➡ X エラーと Z エラーを訂正できれば任意の量子エラーが訂正できる！

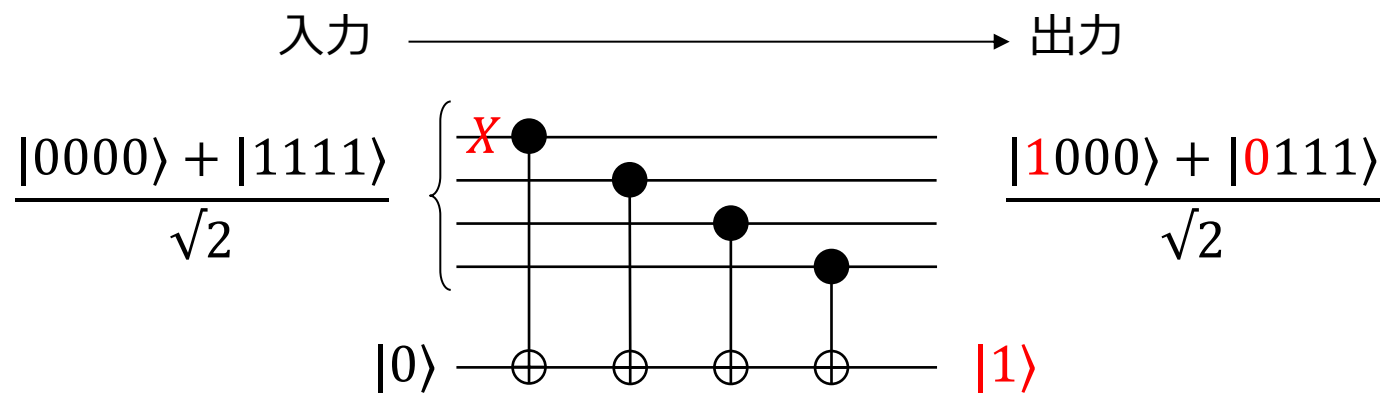
適切に初期化した補助量子ビット（アンシラ）を利用して
エラーだけを測定する



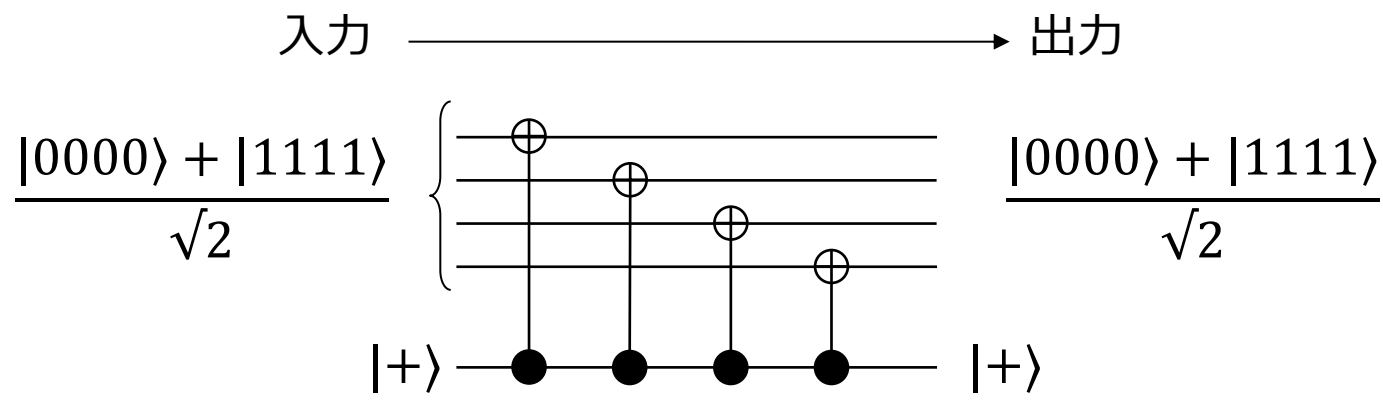
適切に初期化した補助量子ビット（アンシラ）を利用して
エラーだけを測定する



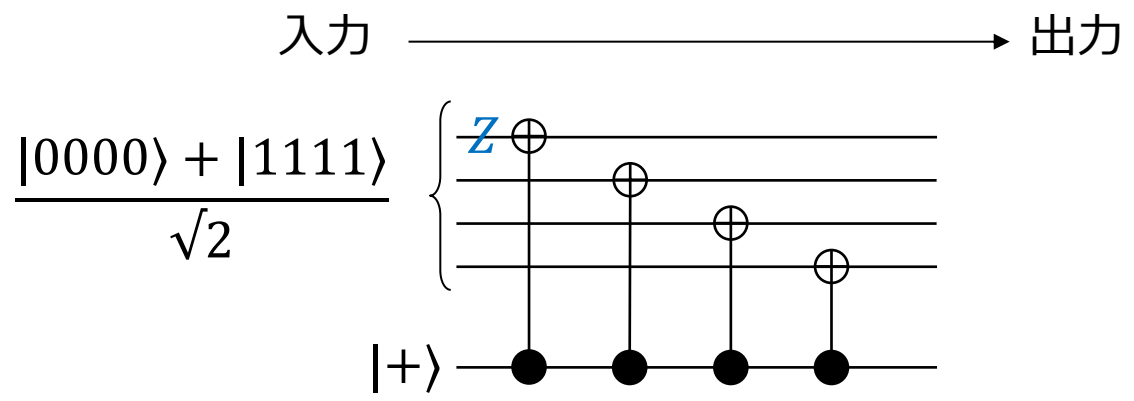
適切に初期化した補助量子ビット（アンシラ）を利用して
エラーだけを測定する



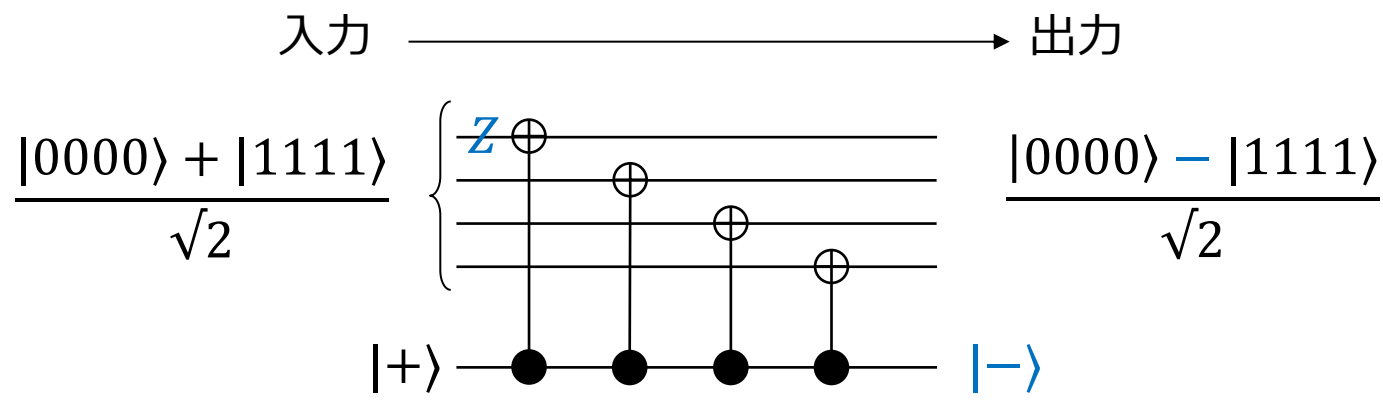
適切に初期化した補助量子ビット（アンシラ）を利用して
エラーだけを測定する



適切に初期化した補助量子ビット（アンシラ）を利用して
エラーだけを測定する



適切に初期化した補助量子ビット（アンシラ）を利用して
エラーだけを測定する



量子誤り訂正に求められる条件まとめ

① 連続的な情報に生じたエラーを訂正できる

➡ 任意の量子エラーはXエラーとZエラーに離散化できるので、それぞれを訂正できればOK！

{ ② 直接状態を観測することなく訂正できる

{ ③ 未知の量子状態を複製することなく訂正できる

➡ 適切に初期化した補助量子ビット（アンシラ）を利用してエラーだけを測定する

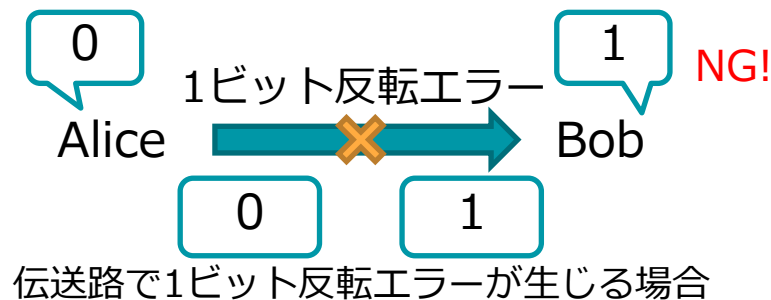
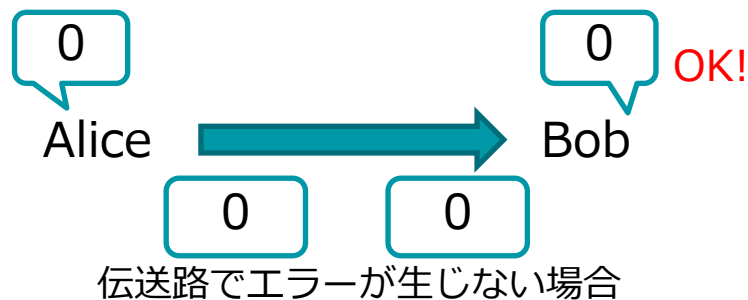
今日の内容

- 量子誤り訂正の基礎
 - 量子誤り訂正に求められる条件
 - 古典符号と量子符号
 - スタビライザ形式
 - Shorの符号
- 表面符号
 - 表面符号の誤り推定
 - Stim + Crumbleを用いた表面符号シミュレーション
 - 極低温環境での表面符号の誤り推定処理
- 表面符号 + 格子手術に基づく用いた誤り耐性量子計算
- まとめ

誤り訂正符号の役割

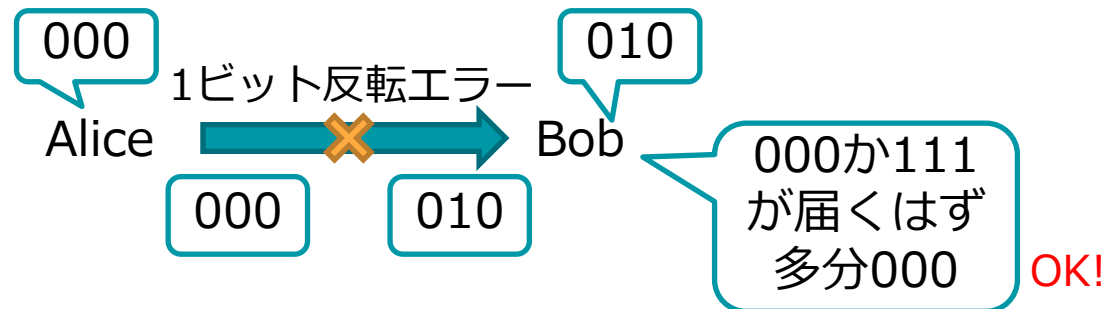
例：ノイズのある伝送路での1ビット情報の送信

パターン1：1ビットの情報をそのまま送信



パターン2：1ビットの情報を3ビットに冗長化して送信

事前の取り決め
送信するデータは
000
または
111



誤り訂正符号の役割

元データ ($k = 1$)

符号語 ($n = 3$)

0	符号化 →	000
1		111

- **誤り訂正符号**: 元データを冗長に表現して誤りの検出・訂正を可能にする符号
- **符号語**: 元のデータを符号化した際に得られる符号として有効なビット列
- **符号パラメータ** $[n, k, d]$
 - n : 符号長 (元データをどの程度冗長に表現するか = 符号語のビット数)
 - k : 情報数 (元データのビット数)
 - d : 符号距離 (ある符号語を別の符号語に変えるのに必要な最小ビット数)
 - $\lfloor (d - 1)/2 \rfloor$ ビットまでは訂正可能、 $d - 1$ 個までは検出可能

符号の作り方

$\{0,1\}^k$

$\{0,1\}^n$

00	0000	0001	0010	0011
01	0100	0101	0110	0111
10	1000	1001	1010	1011
11	1100	1101	1110	1111

適当に符号語に
割り当てると対応表が
膨大に

$k = 2, n = 4$ の例

- k 次元空間の各点を n 次元空間の一部（**符号空間**）に割当
- 割当の際は何かしらのルールが必要

誤り訂正符号の構成 古典の場合

ルールの例： k ビットの元データを繰り返して n ビットにする

$n = 3, k = 1$ の例

元データ		符号語
0	符号化 →	$0_L: 0\ 0\ 0$
1		$1_L: 1\ 1\ 1$

符号語に1ビットエラーが起きると

$0\ 0\ 0 \xrightarrow{\text{エラー}} 0\ \underline{1}\ 0 \xrightarrow{\text{訂正}} 0\ 0\ 0$

$n = 4, k = 2$ の例

元データ		符号語
00	符号化 →	$00_L: 0\ 0\ 0\ 0$
01		$01_L: 0\ 1\ 0\ 1$
10		$10_L: 1\ 0\ 1\ 0$
11		$11_L: 1\ 1\ 1\ 1$

誤り検出方法

= 符号を構成する各ビットをチェックしてそのようになっているか

誤り検出のために符号語を構成するビットを直接観測する
必要があるため、量子では使えないアプローチ

誤り訂正符号の構成 古典の場合

ルールの例：隣接ビットのパリティ（偶奇）が特定のパターン

$n = 3, k = 1$ の例

元データ	符号化	符号語
0		$0_L: 0\ 0\ 0$ 偶 偶
1		$1_L: 1\ 1\ 1$ 偶 偶
符号語に1ビットエラーが起きると		
$0\ 0\ 0$ 偶 偶	$\rightarrow$	$0\ \underline{1}\ 0$ エラー 奇 奇
		$\rightarrow$
		$0\ 0\ 0$ 訂正 偶 偶

$n = 4, k = 2$ の例

元データ	符号化	符号語
00		$00_L: 0\ 0\ 0\ 0$ 偶 偶 偶
01		$01_L: 0\ 1\ 0\ 1$ 奇 奇 奇
10		$10_L: 1\ 0\ 1\ 0$ 奇 奇 奇
11		$11_L: 1\ 1\ 1\ 1$ 偶 偶 偶

誤り検出方法
＝隣接ビットの**パリティチェック**をおこないパターンをチェック

誤り検出のために符号語を構成するビットを直接観測する必要がない
➡量子でも使えるアプローチ！

誤り訂正符号の構成 古典の場合

ルールの例：隣接ビットのパリティ（偶奇）が特定のパターン

$n = 3, k = 1$ の例

元データ		符号語
0	符号化 →	$0_L: ???$ 偶 偶
1		$1_L: ???$ 偶 偶

符号語に1ビットエラーが起きると

$??? \rightarrow ??? \rightarrow ???$
偶 偶 エラー 奇 奇 訂正 偶 偶

$n = 4, k = 2$ の例

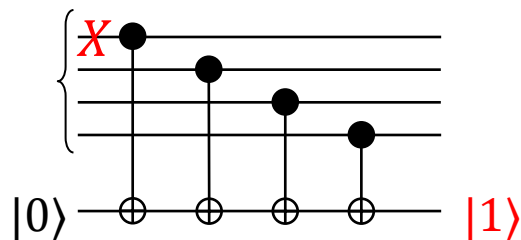
元データ		符号語
00	符号化 →	$00_L: ????$ 偶 偶 偶
01		$01_L: ????$ 奇 奇 奇
10		$10_L: ????$ 奇 奇 奇
11		$11_L: ????$ 偶 偶 偶

誤り検出方法
= 隣接ビットの**パリティチェック**をおこないパターンをチェック

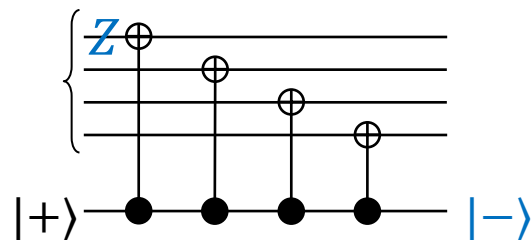
誤り検出のために符号語を構成するビットを直接観測する必要がない
➡ 量子でも使えるアプローチ！

誤り訂正符号の構成 古典の場合

適切に初期化した補助量子ビット（アンシラ）を利用して
量子ビットのエラーパリティチェックをおこなう



Xエラーパリティチェック



Zエラーパリティチェック

誤り検出のために符号語を構成するビットを直接観測する必要がない
➡ 量子でも使えるアプローチ！

今日の内容

- 量子誤り訂正の基礎
 - 量子誤り訂正に求められる条件
 - 古典符号と量子符号
 - スタビライザ形式
 - Shorの符号
- 表面符号
 - 表面符号の誤り推定
 - Stim + Crumbleを用いた表面符号シミュレーション
 - 極低温環境での表面符号の誤り推定処理
- 表面符号 + 格子手術に基づく用いた誤り耐性量子計算
- まとめ

スタビライザ形式

Bell状態

$$|\psi\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

$$ZZ|\psi\rangle = +|\psi\rangle$$

$$XX|\psi\rangle = +|\psi\rangle$$

- 状態ベクトルそのものを並べる代わりに、
「その状態を変えない演算子」の組で量子状態を表す方法
 - どの演算子の固有値が+1か？
- 状態 $|\psi\rangle$ のスタビライザ： $S_i|\psi\rangle = +|\psi\rangle$ となる S_i
 - S_i は互いに可換なPauli演算子の積

スタビライザ形式：1量子ビットの場合

Pauli gate : エルミートかつユニタリ。有限群を成す (Pauli群)

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = iXZ$$

$$I^2 = I$$

$$X^2 = I$$

$$Z^2 = I$$

$$Y^2 = I$$

$$\left. \begin{array}{l} X|0\rangle = |1\rangle \\ X|1\rangle = |0\rangle \end{array} \right\} \begin{array}{l} \text{ビット} \\ \text{反転} \end{array}$$

$$\begin{array}{l} X|+\rangle = |+\rangle \\ X|-\rangle = -|-\rangle \end{array}$$

$$\begin{array}{l} Z|0\rangle = |0\rangle \\ Z|1\rangle = -|1\rangle \end{array}$$

$$\left. \begin{array}{l} Z|+\rangle = |-\rangle \\ Z|-\rangle = |+\rangle \end{array} \right\} \begin{array}{l} \text{位相} \\ \text{反転} \end{array}$$

$$\begin{array}{l} Y|+i\rangle = |+i\rangle \\ Y|-i\rangle = -|-i\rangle \end{array}$$

$$(| \pm i \rangle = \frac{|0\rangle \pm i|1\rangle}{\sqrt{2}})$$

Pauli gateはanti-commute:

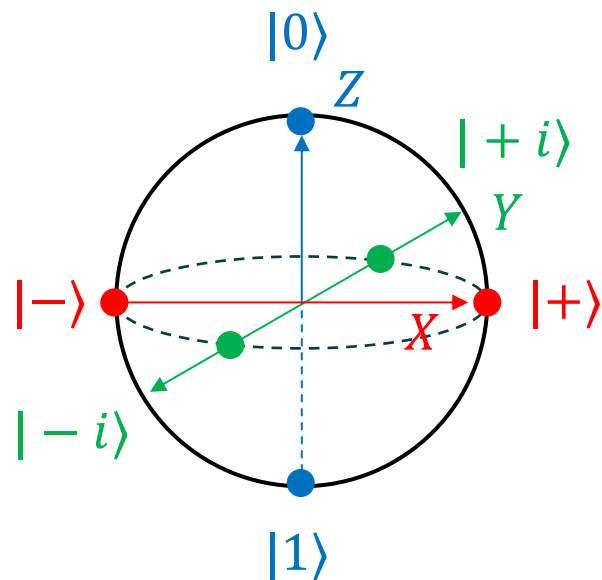
$$XZ = -ZX$$

$$XY = -YX$$

$$ZY = -YZ$$

スタビライザ状態

状態	スタビライザ
$ 0\rangle$	Z
$ 1\rangle$	$-Z$
$ +\rangle$	X
$ -\rangle$	$-X$
$ +i\rangle$	Y
$ -i\rangle$	$-Y$



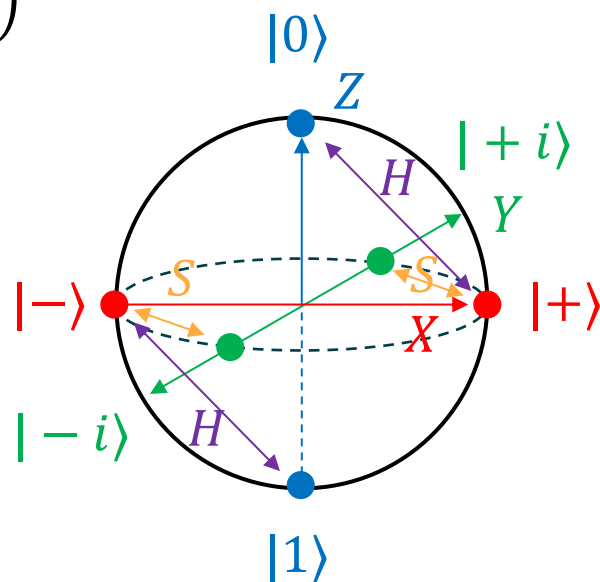
スタビライザ形式：1量子ビットの場合

Clifford gate : PauliをPauliに変換するユニタリ。
Pauli群とともに有限群を成す (Clifford群)

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{\sqrt{2}} (X + Z) \quad S = \sqrt{Z} = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

$$\left. \begin{array}{l} H^2 = I \\ H|0\rangle = |+\rangle \\ H|1\rangle = |-\rangle \\ HX = ZH \\ HZ = XH \\ HY = -YH \end{array} \right\} \begin{array}{l} x \text{ と } z \text{ の} \\ \text{入れ替え} \end{array}$$

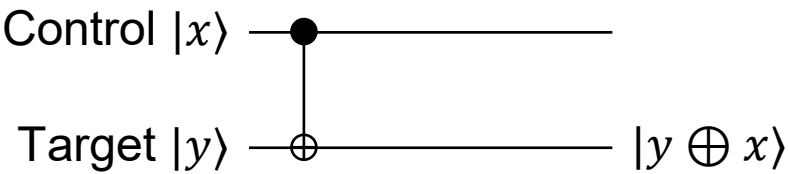
$$\left. \begin{array}{l} S^2 = Z \\ S|+\rangle = |+i\rangle \\ S|-\rangle = |-i\rangle \\ SX = YS \\ SY = XS \\ SZ = ZS \end{array} \right\} \begin{array}{l} x \text{ と } y \text{ の} \\ \text{入れ替え} \end{array}$$



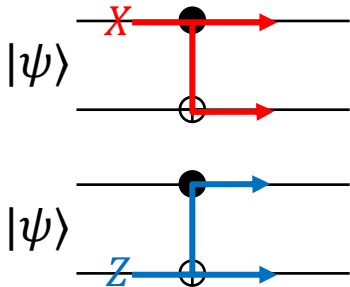
スタビライザ形式：2量子ビットの場合

Clifford gate : CNOTゲートも含めて有限群を成す (Clifford群)

$$\text{CNOT} = \begin{pmatrix} I & 0 \\ 0 & X \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$



$\text{CNOT}(X \otimes I) = (X \otimes X)\text{CNOT}$
 $\text{CNOT}(I \otimes X) = (I \otimes X)\text{CNOT}$
 $\text{CNOT}(Z \otimes I) = (Z \otimes I)\text{CNOT}$
 $\text{CNOT}(I \otimes Z) = (Z \otimes Z)\text{CNOT}$



スタビライザ状態で閉じる

$\text{CNOT}(X \otimes I)|\psi\rangle = (X \otimes X)\text{CNOT}|\psi\rangle$

$\text{CNOT}(I \otimes Z)|\psi\rangle = (Z \otimes Z)\text{CNOT}|\psi\rangle$

CNOTはXをcontrolからtargetへ、Zをtargetからcontrolへ伝搬させる

ユニタリゲートによるPauli演算子の変換

1量子ビットゲート

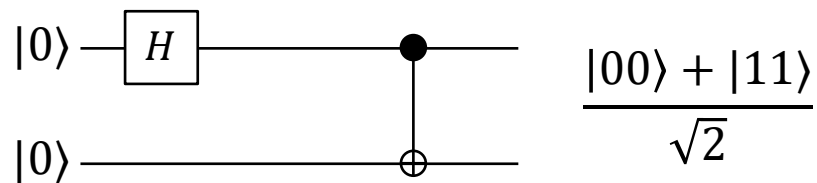
2量子ビットゲート

ゲート	入力	出力
H	X	Z
	Z	X
S	X	Y
	Z	Z
X	X	X
	Z	$-Z$
Y	X	$-X$
	Z	$-Z$
Z	X	$-X$
	Z	Z

ゲート	入力	出力
CNOT	X_1	$X_1 \otimes X_2$
	X_2	X_2
	Z_1	Z_1
CZ	Z_2	$Z_1 \otimes Z_2$
	X_1	$X_1 \otimes Z_2$
	X_2	$Z_1 \otimes X_2$
	Z_1	Z_1
	Z_2	Z_2

スタビライザ形式：2量子ビットの場合

例：Bell状態の生成

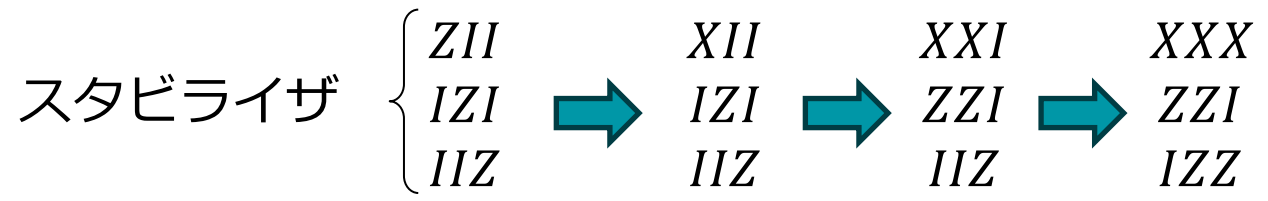
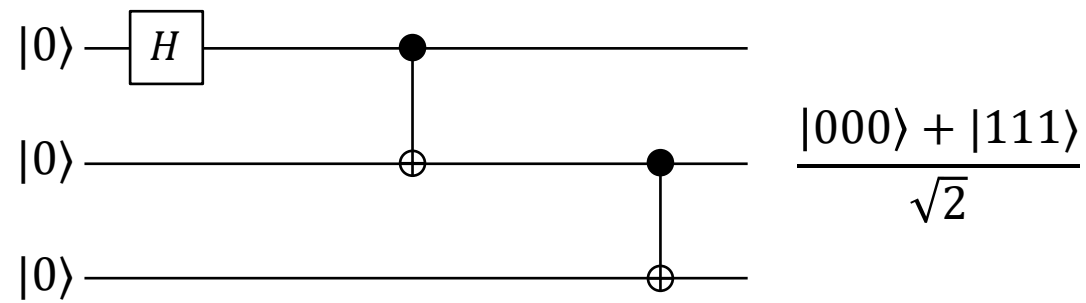


スタビライザ $\begin{cases} ZI \\ IZ \end{cases} \Rightarrow \begin{cases} XI \\ IZ \end{cases} \Rightarrow \begin{cases} XX \\ ZZ \end{cases}$

入力が**スタビライザ状態**で操作が**Cliffordゲートのみ**であれば、
2量子ビットの出力状態を $2^2 = 4$ 個の基底を用いた状態ベクトルの代わりに、
2個のスタビライザで簡潔に表せる

スタビライザ形式：3量子ビットの場合

例：GHZ状態の生成



入力が**スタビライザ状態**で操作が**Cliffordゲートのみ**であれば、
3量子ビットの出力状態を $2^3 = 8$ 個の基底を用いた状態ベクトルの代わりに、
3個のスタビライザで簡潔に表せる

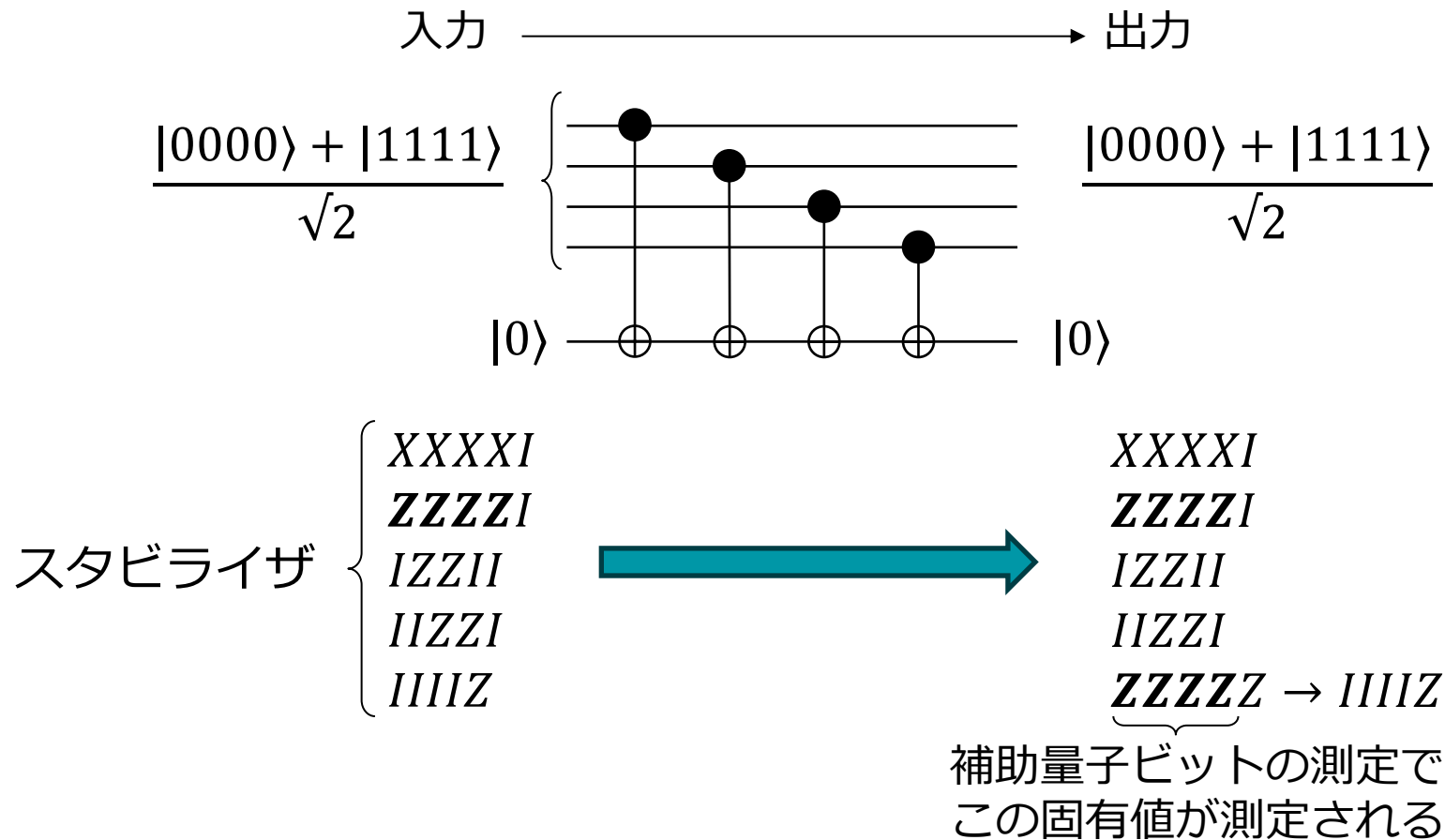
スタビライザ形式： N 量子ビットの場合

入力が**スタビライザ状態**で操作が**Cliffordゲートのみ**であれば、 N 量子ビットの出力状態を 2^N 個の基底を用いた状態ベクトルの代わりに、 **N 個のスタビライザ**で簡潔に表せる



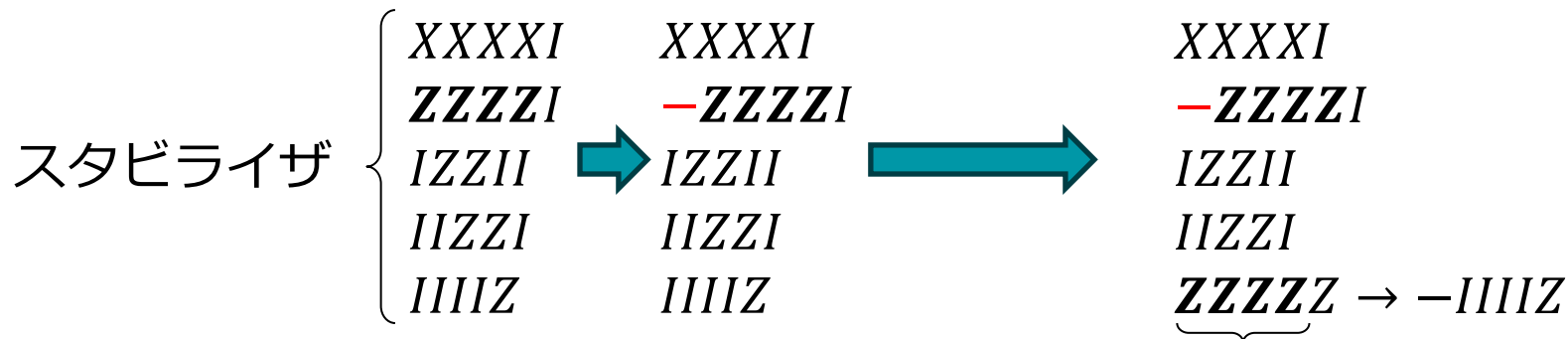
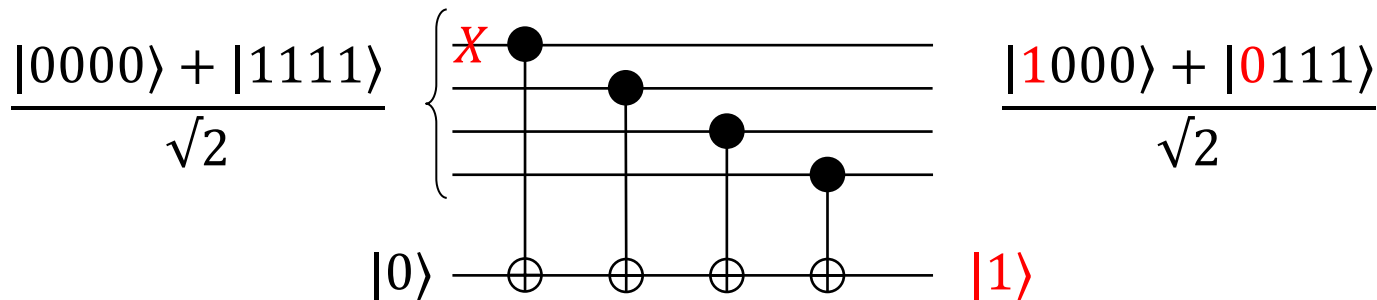
入力が**スタビライザ状態**で操作が**Cliffordゲートのみ**であれば、古典計算機で効率的にシミュレートできる（Gottesman-Knillの定理）

補助量子ビットを用いたエラー検出



補助量子ビットを用いたエラー検出

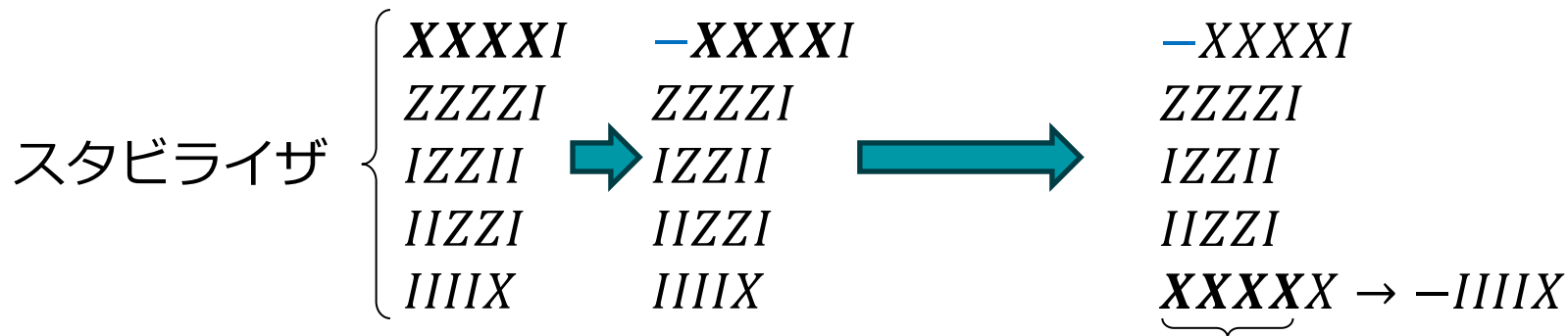
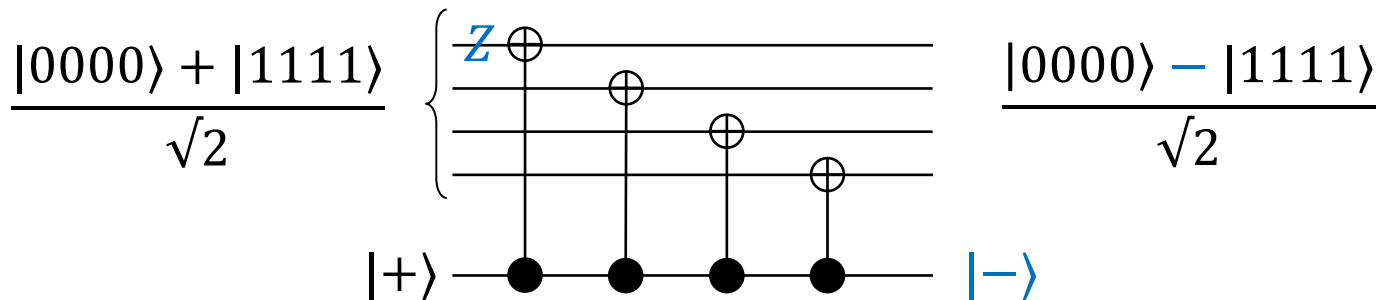
入力 $\longrightarrow$ 出力



補助量子ビットの測定で
この固有値が測定される

補助量子ビットを用いたエラー検出

入力 $\longrightarrow$ 出力



補助量子ビットの測定で
この固有値が測定される

スタビライザ符号

- スタビライザ符号 = 複数のスタビライザ状態の +1 固有状態として符号語が定義される

N 個の
物理量子ビット



可換なPauliチェック $S_1, \dots, S_r$

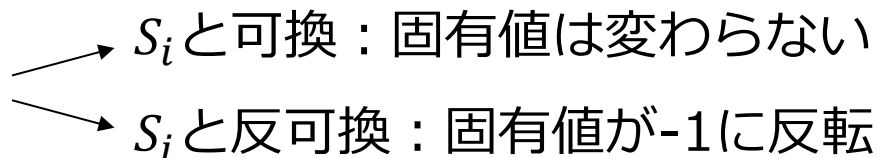


全てのチェックで固有値 +1

符号空間 C (k 個の論理量子ビット)

$$C = \{|\psi\rangle : S_i|\psi\rangle = |\psi\rangle, i = 1, \dots, r\}, r = n - k$$

誤り E が発生

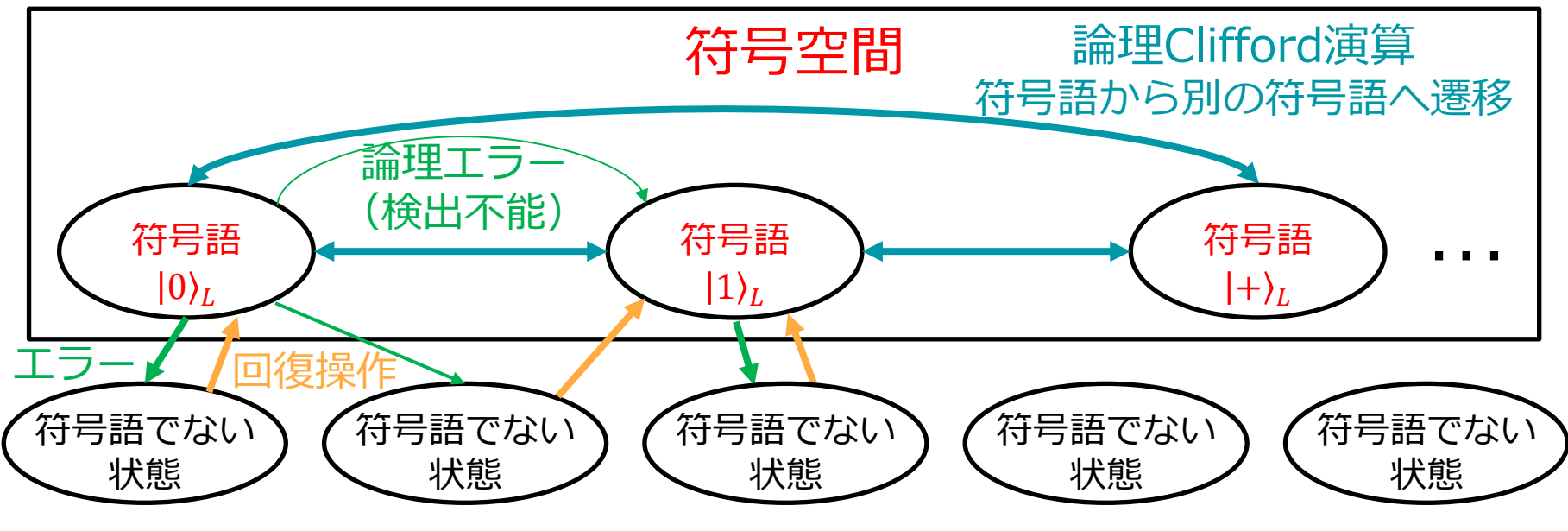


シンδροーム

$$\text{純粋状態 } |\psi\rangle = \sum_i^{2^n} \alpha_i |i\rangle$$

スタビライザ状態

$0(n)$ 個程度のスタビライザ演算子で特徴付けられる



今日の内容

- 量子誤り訂正の基礎
 - 量子誤り訂正に求められる条件
 - 古典符号と量子符号
 - スタビライザ形式
 - Shorの符号
- 表面符号
 - 表面符号の誤り推定
 - Stim + Crumbleを用いた表面符号シミュレーション
 - 極低温環境での表面符号の誤り推定処理
- 表面符号 + 格子手術に基づく用いた誤り耐性量子計算
- まとめ

量子反復符号

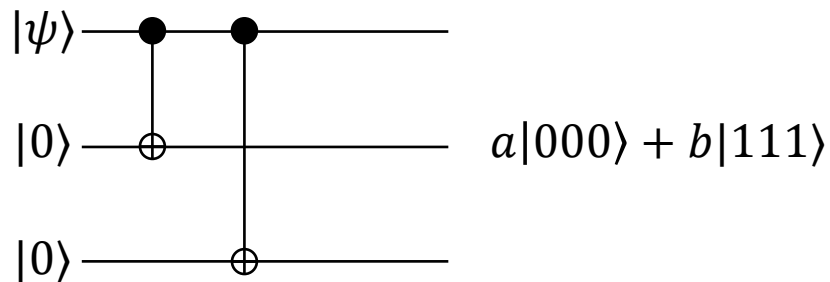
- 古典の場合と同様に3ビット反復符号を考える

符号語

$$|0\rangle_L = |000\rangle$$

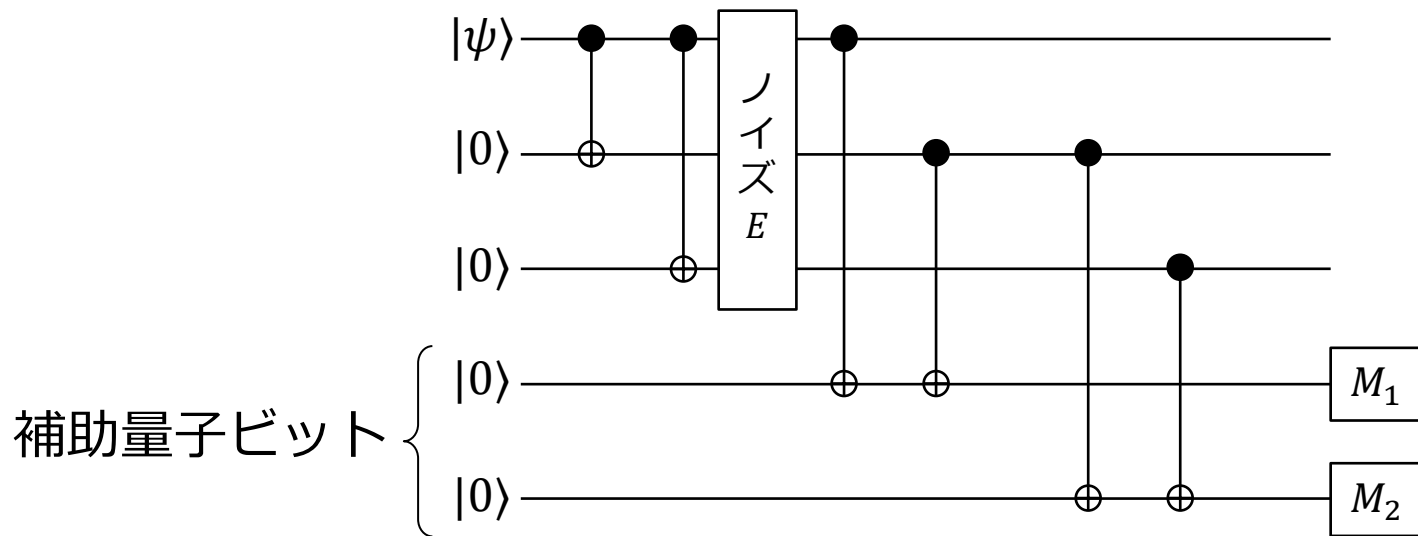
$$|1\rangle_L = |111\rangle$$

元のデータ $|\psi\rangle = a|0\rangle + b|1\rangle$ は以下の回路で符号化できる。



量子反復符号

- 補助量子ビットを追加して隣接ビットのパリティチェック

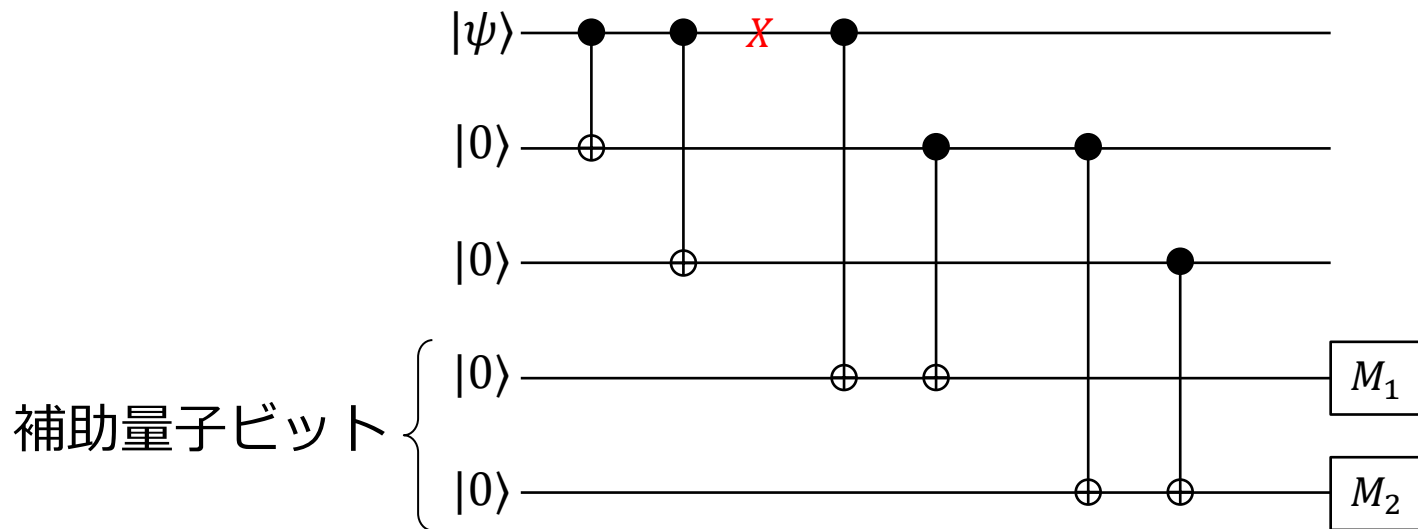


$M_1 = Z_1 Z_2$ の測定値 (パリティ)

$M_2 = Z_2 Z_3$ の測定値 (パリティ)

量子反復符号

- 補助量子ビットを追加して隣接ビットのパリティチェック

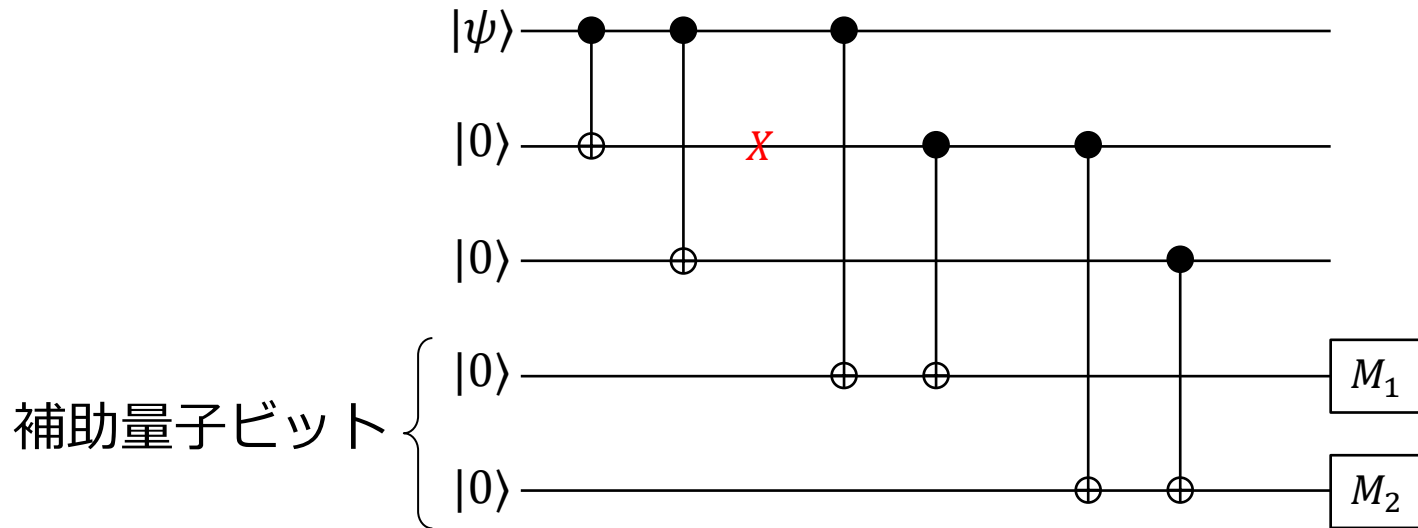


$M_1 = Z_1 Z_2$ の測定値 (パリティ)
奇

$M_2 = Z_2 Z_3$ の測定値 (パリティ)
偶

量子反復符号

- 補助量子ビットを追加して隣接ビットのパリティチェック

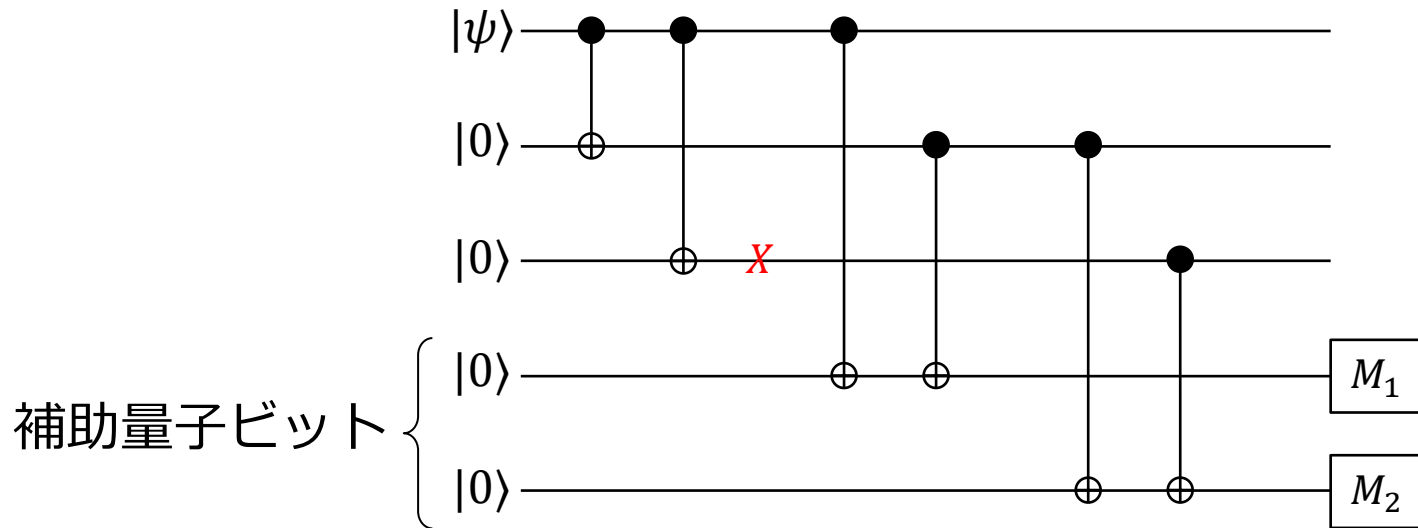


$M_1 = Z_1 Z_2$ の測定値 (パリティ)
奇

$M_2 = Z_2 Z_3$ の測定値 (パリティ)
奇

量子反復符号

- 補助量子ビットを追加して隣接ビットのパリティチェック

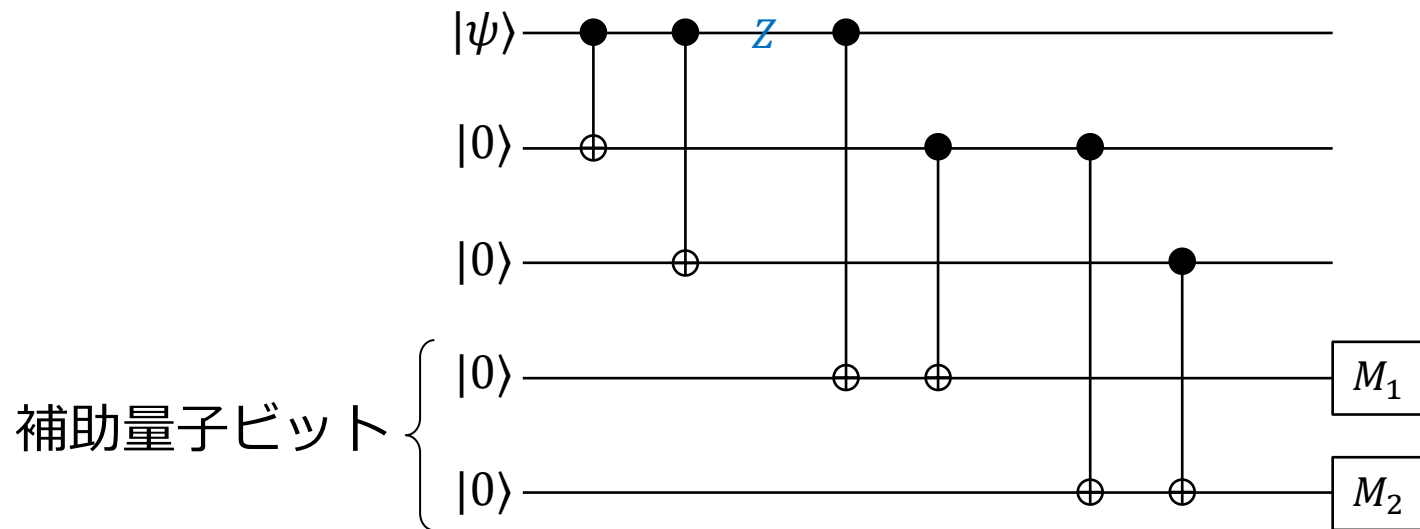


$M_1 = Z_1 Z_2$ の測定値 (パリティ)
偶

$M_2 = Z_2 Z_3$ の測定値 (パリティ)
奇

量子反復符号

- 1ビット **Xエラー** は検知できるが **Zエラー** は検知できない

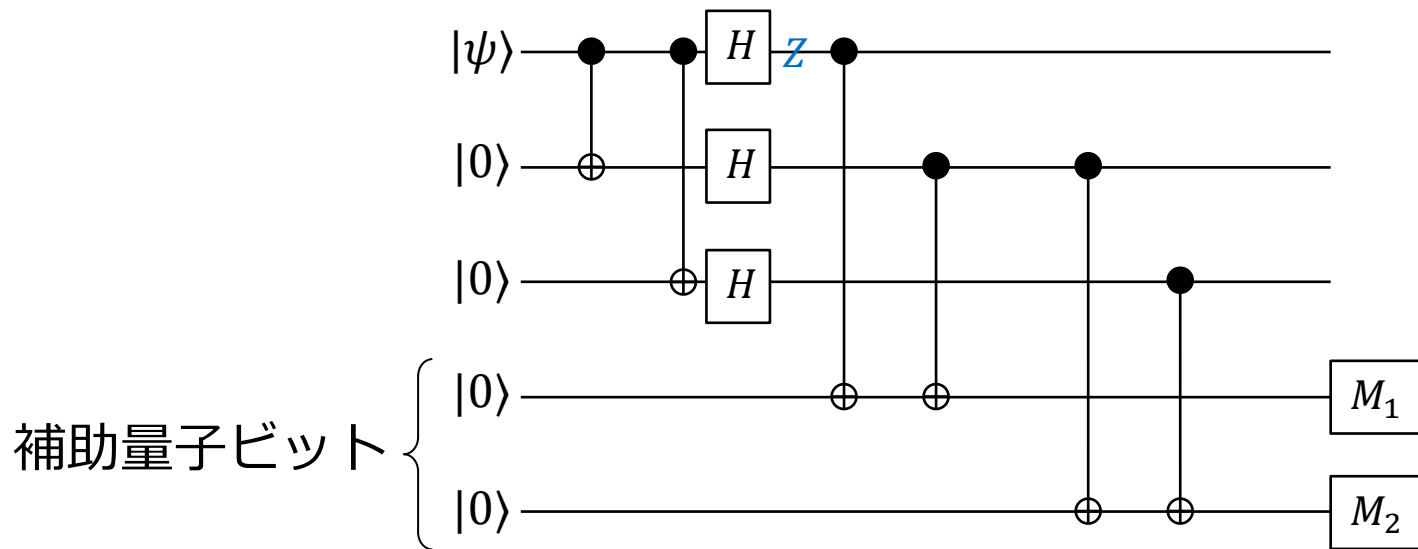


$M_1 = Z_1 Z_2$ の測定値 (パリティ)
偶

$M_2 = Z_2 Z_3$ の測定値 (パリティ)
偶

量子反復符号

- H を入れると $Z \rightarrow X$ を変換するので検知できるようになる



$M_1 = Z_1 Z_2$ の測定値 (パリティ)

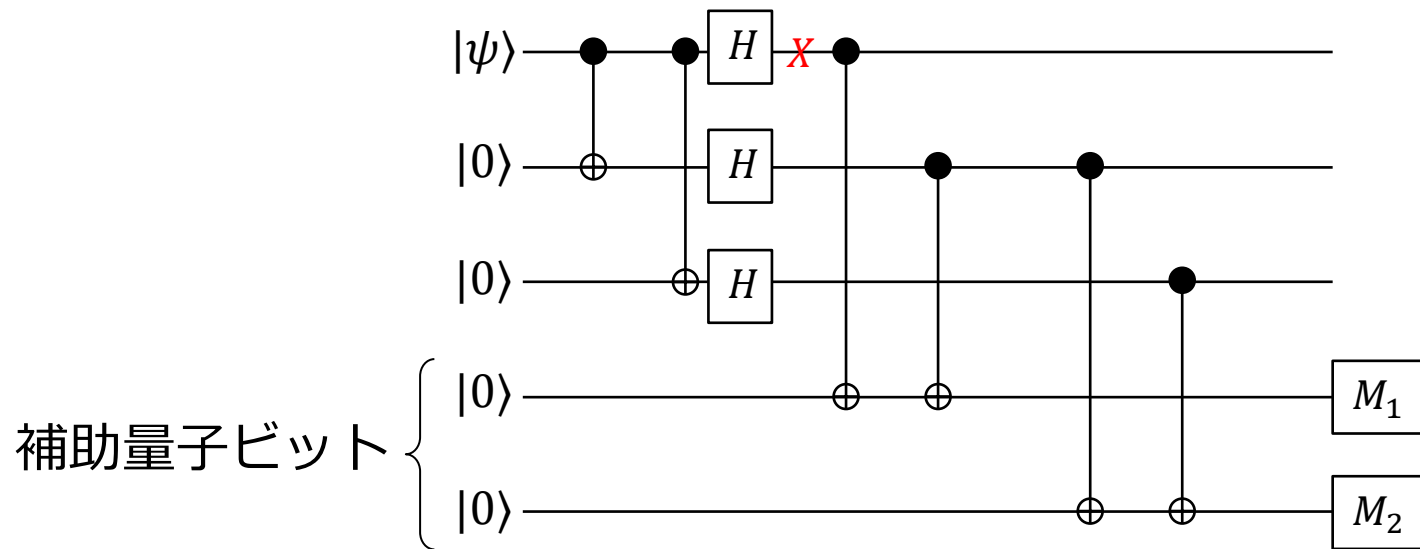
奇

$M_2 = Z_2 Z_3$ の測定値 (パリティ)

偶

量子反復符号

- しかし、今度はXエラーが検知できなくなる



$M_1 = Z_1 Z_2$ の測定値 (パリティ)
偶

$M_2 = Z_2 Z_3$ の測定値 (パリティ)
偶

Shorの符号

- X エラーと Z エラーを同時に訂正するためのアイデア
= X と Z のエラー訂正用の反復符号を入れ子にする (**連接**)

X エラーは3ビット反復符号で訂正:

$$|0\rangle_{L_1} := |000\rangle, |1\rangle_{L_1} := |111\rangle$$

Z エラーは $|\pm\rangle_{L_1}$ の3ビット反復符号で訂正:

$$|0\rangle_{L_2} := |+++ \rangle_{L_1}, |1\rangle_{L_2} := |-- - \rangle_{L_1}$$

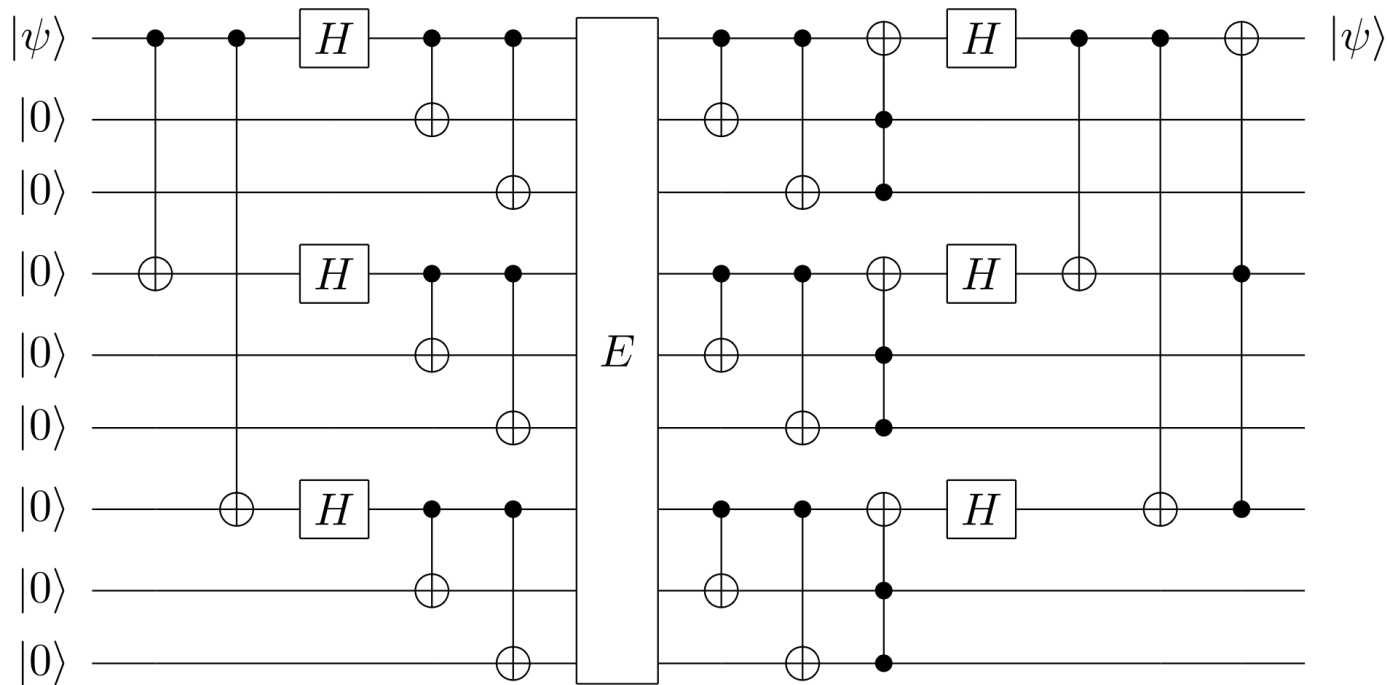


[[9, 1, 3]] Shor符号

$$|0\rangle_L := \frac{|000\rangle+|111\rangle}{\sqrt{2}} \frac{|000\rangle+|111\rangle}{\sqrt{2}} \frac{|000\rangle+|111\rangle}{\sqrt{2}}$$

$$|1\rangle_L := \frac{|000\rangle-|111\rangle}{\sqrt{2}} \frac{|000\rangle-|111\rangle}{\sqrt{2}} \frac{|000\rangle-|111\rangle}{\sqrt{2}}$$

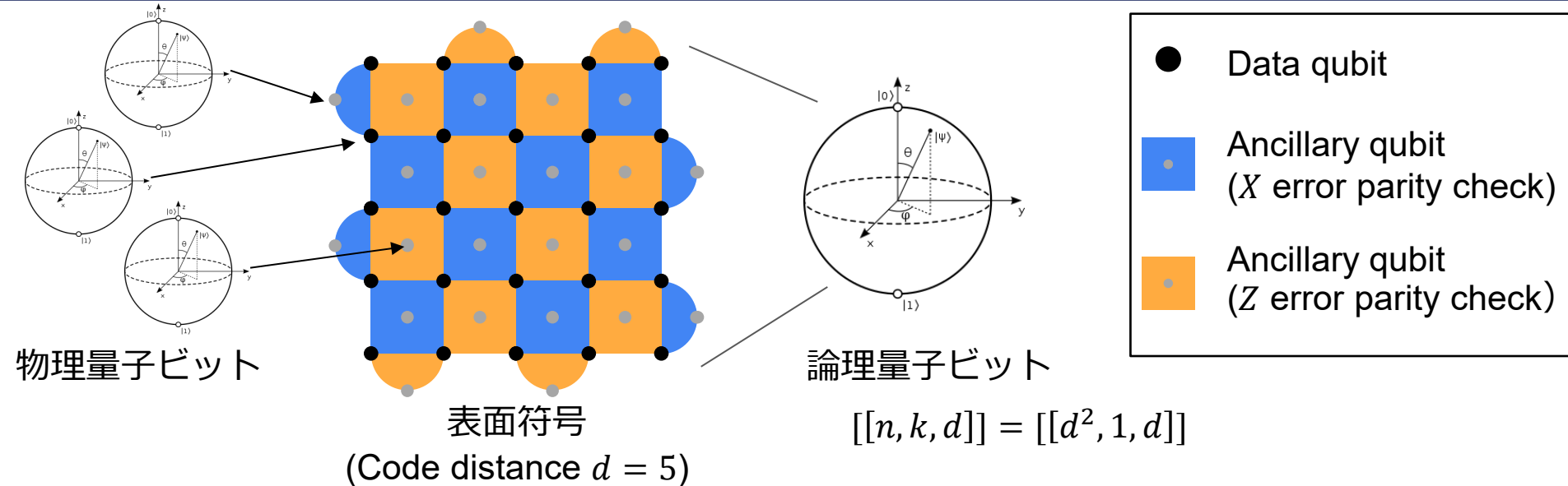
Shorの符号



今日の内容

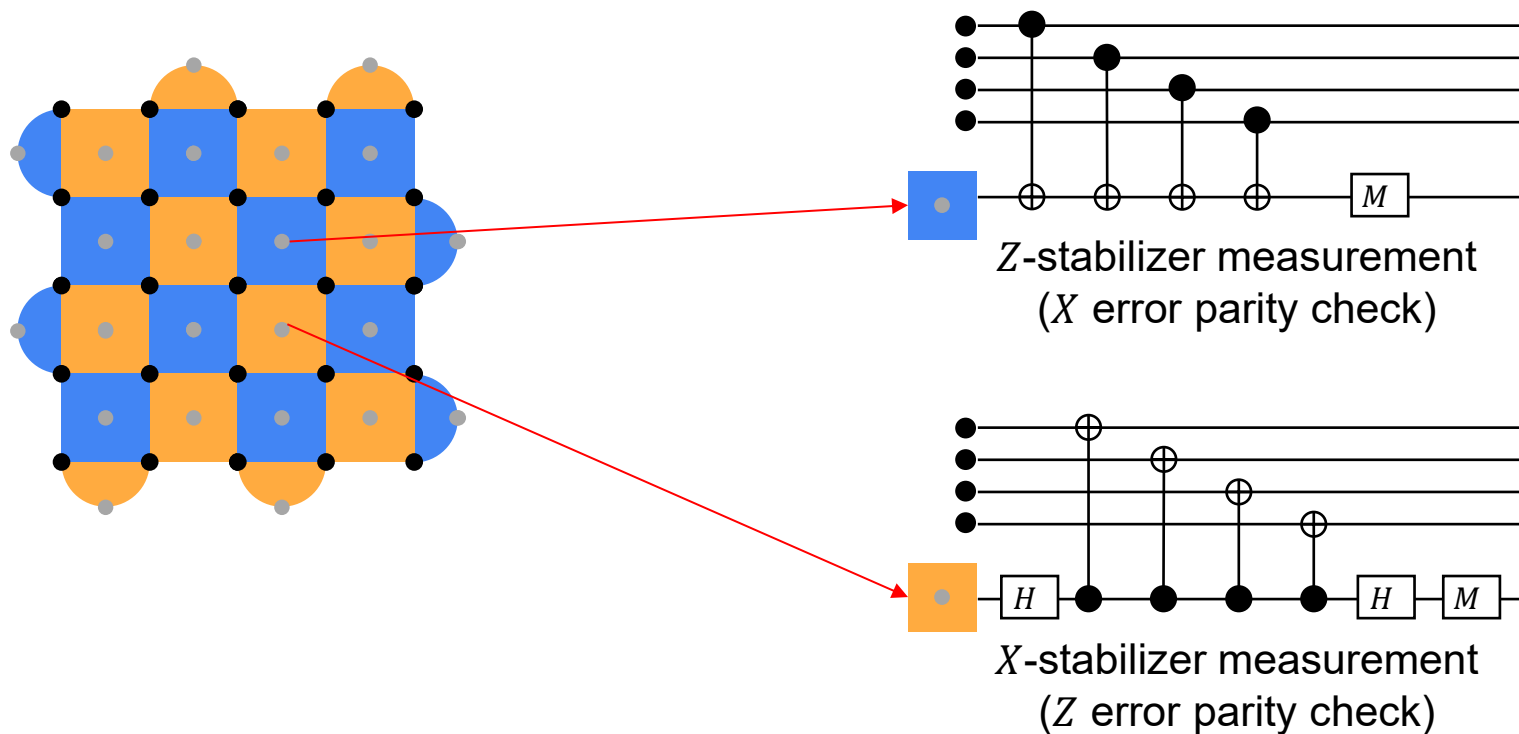
- 量子誤り訂正の基礎
 - 量子誤り訂正に求められる条件
 - 古典符号と量子符号
 - スタビライザ形式
 - Shorの符号
- 表面符号
 - 表面符号の誤り推定
 - Stim + Crumbleを用いた表面符号シミュレーション
 - 極低温環境での表面符号の誤り推定処理
- 表面符号 + 格子手術に基づく用いた誤り耐性量子計算
- まとめ

表面符号

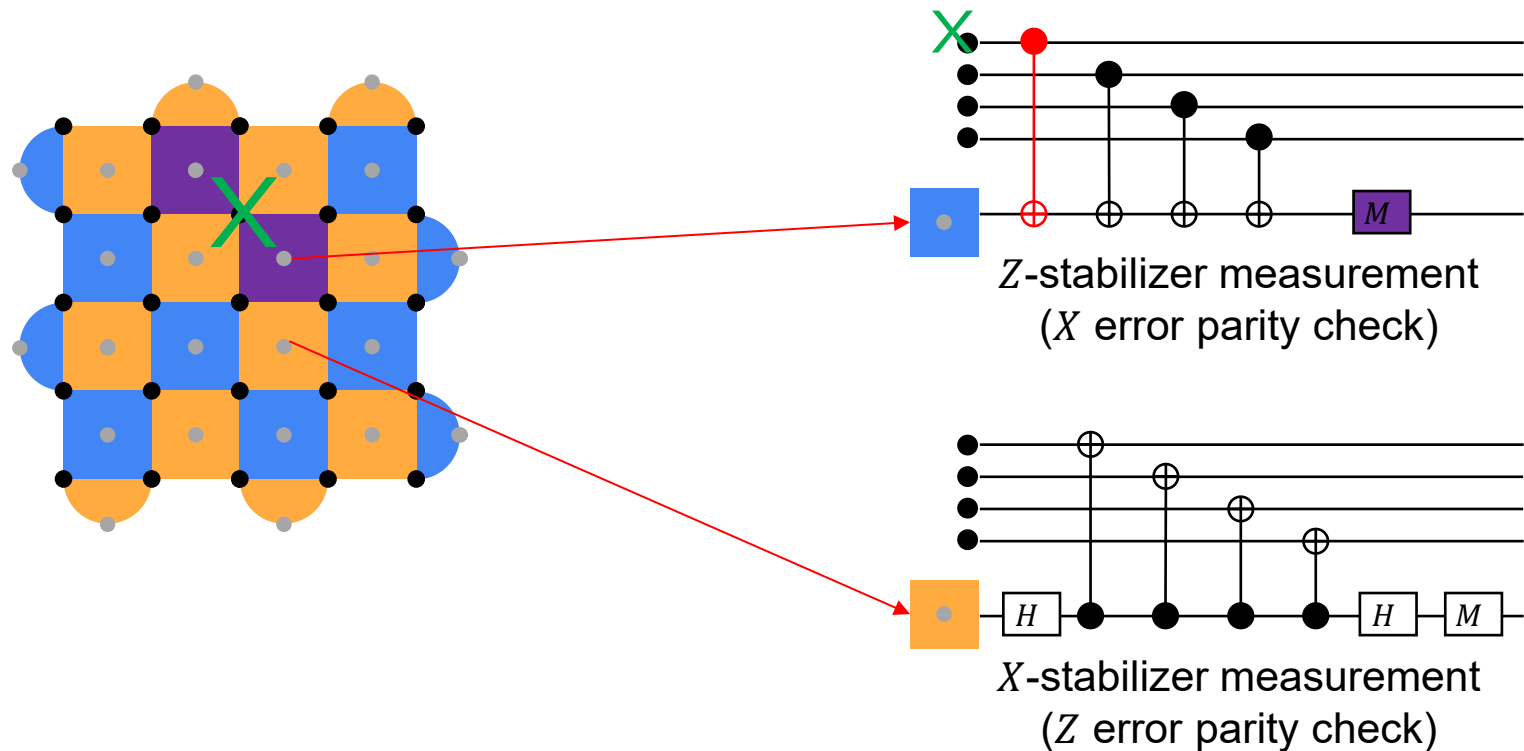


- 規則的に並んだ複数の量子ビットで論理量子ビットを表現
 - 論理ビットの状態を表すデータ量子ビットと観測用の補助量子ビット
- 補助量子ビットの観測値: XとZの2種類のエラーのうち、対応するもののパリティチェック

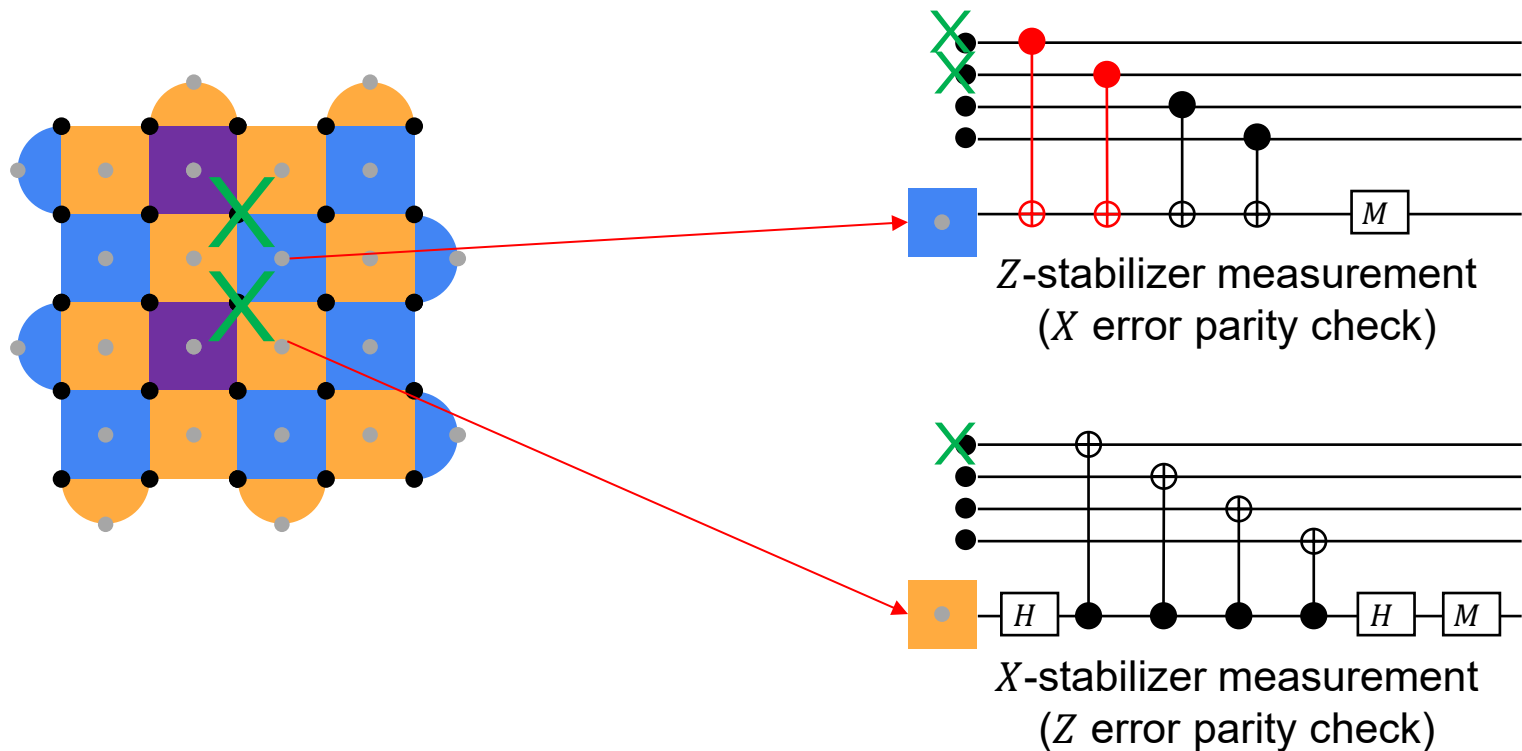
表面符号におけるエラーパリティチェック



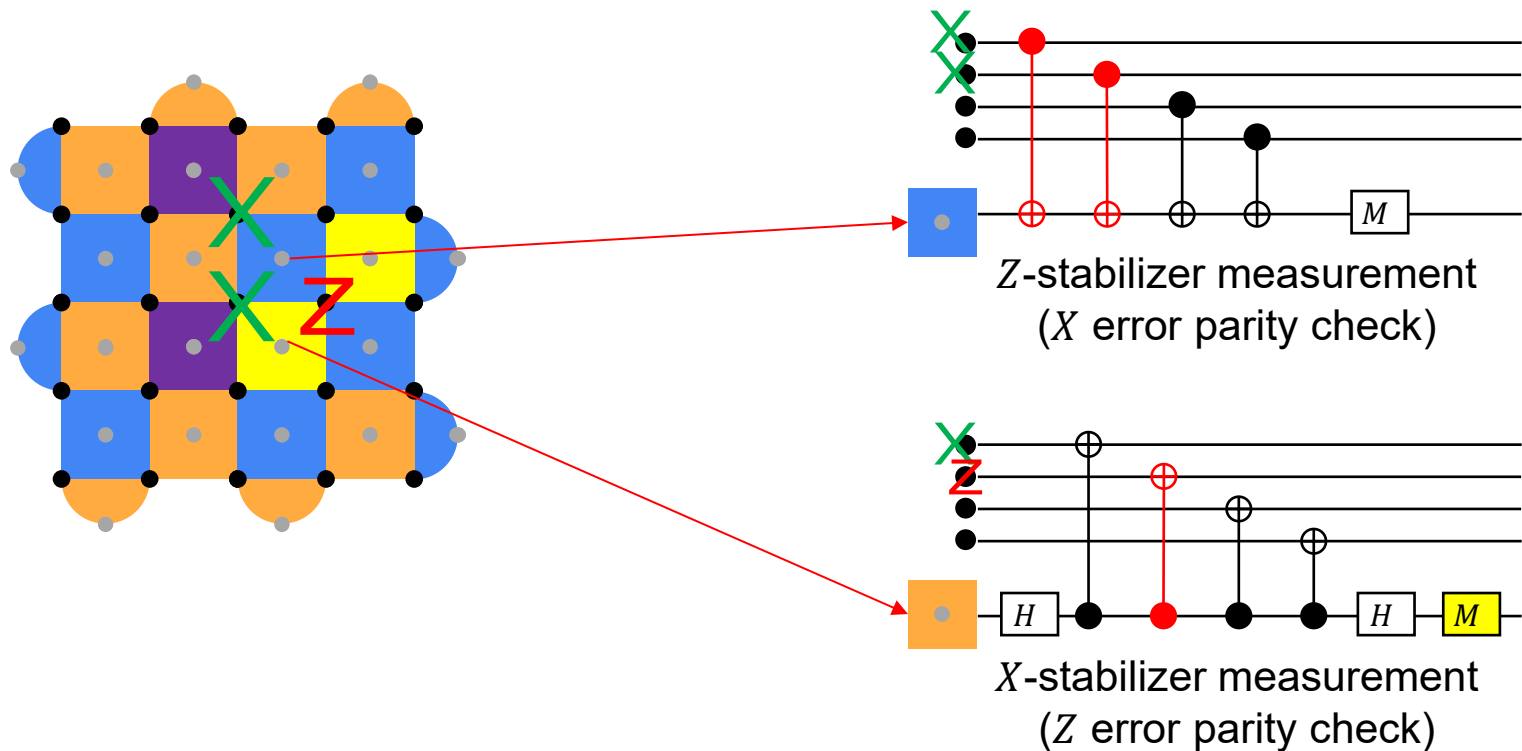
表面符号におけるエラーパリティチェック



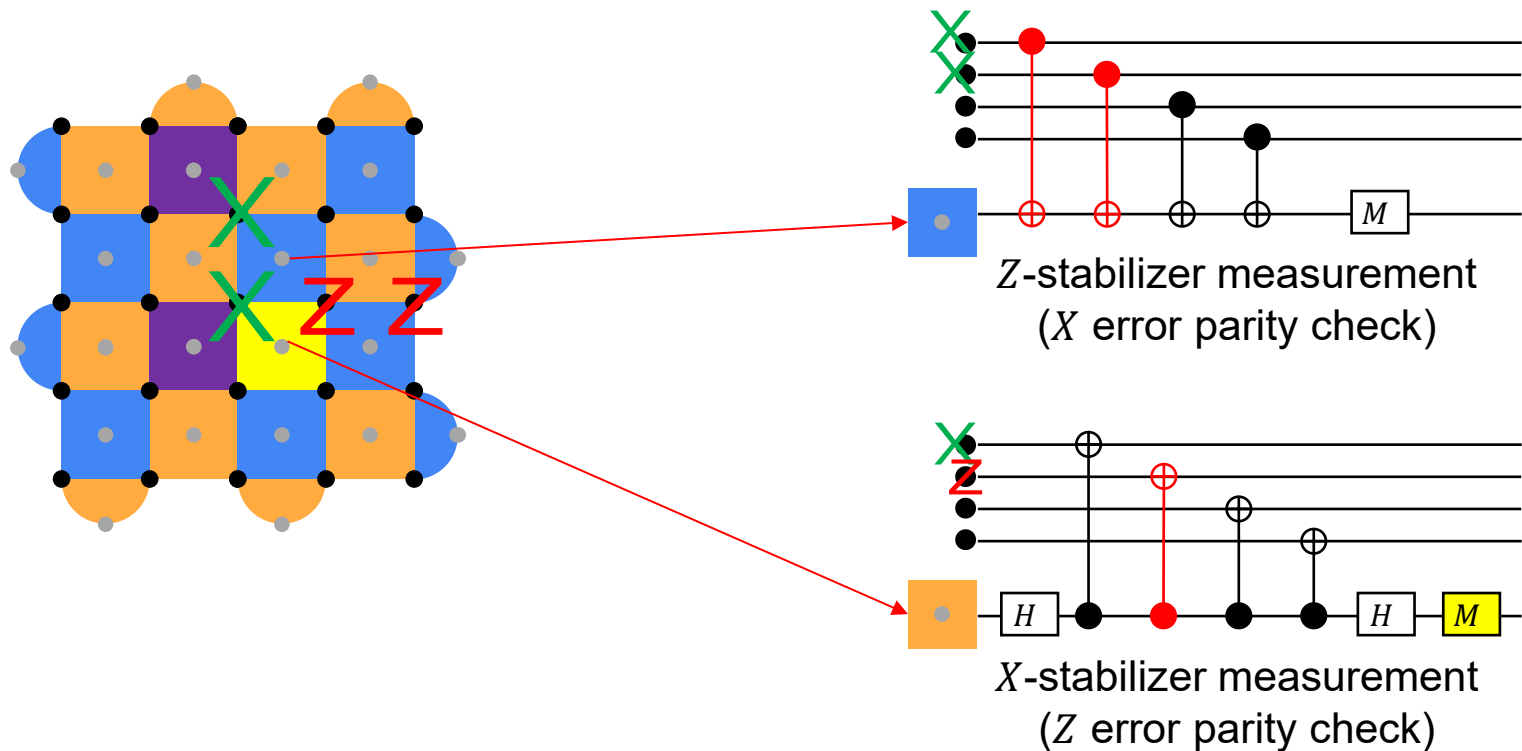
表面符号におけるエラーパリティチェック



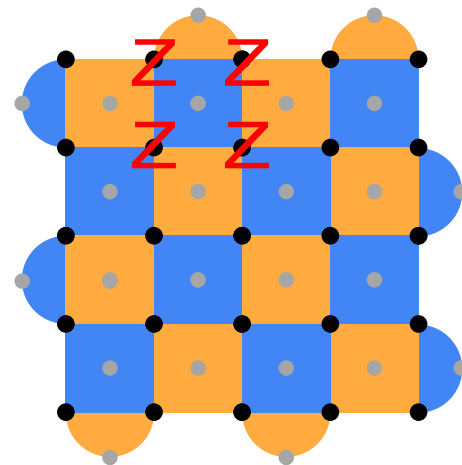
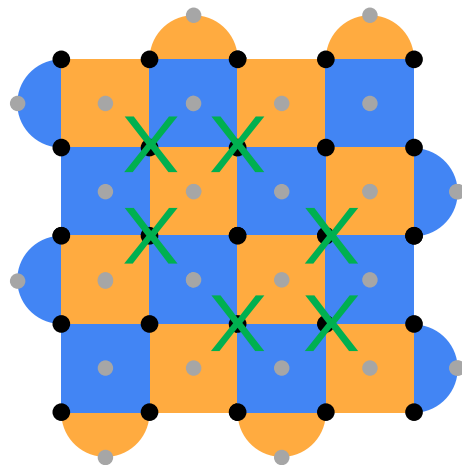
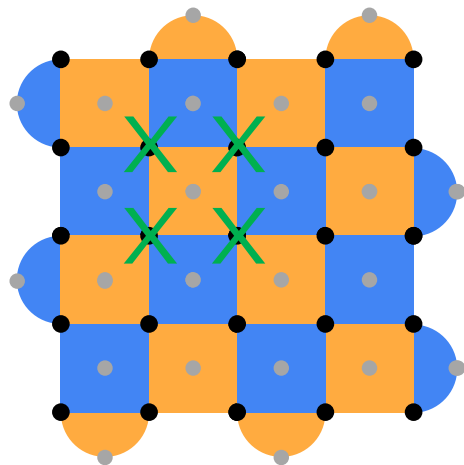
表面符号におけるエラーパリティチェック



表面符号におけるエラーパリティチェック



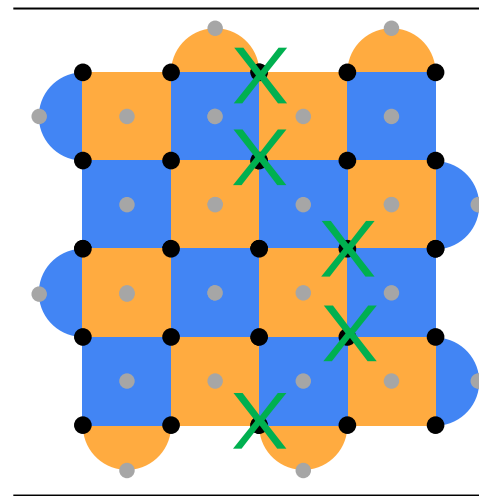
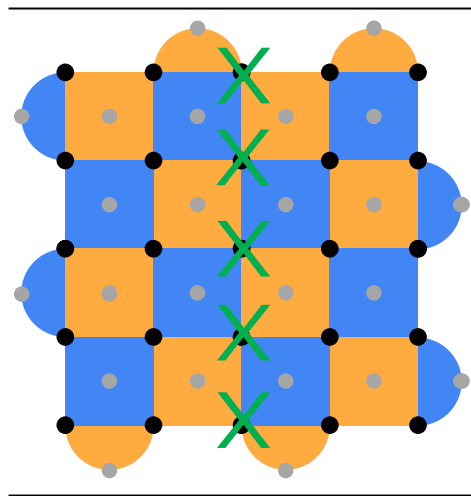
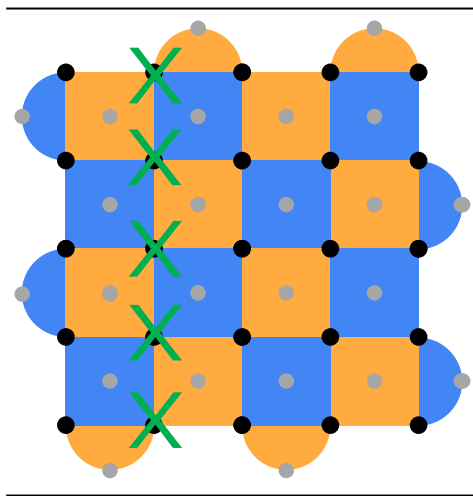
表面符号における論理I演算



いずれも論理I演算

- 表面符号の領域内で閉じたループを描くPauli操作は論理I演算になる

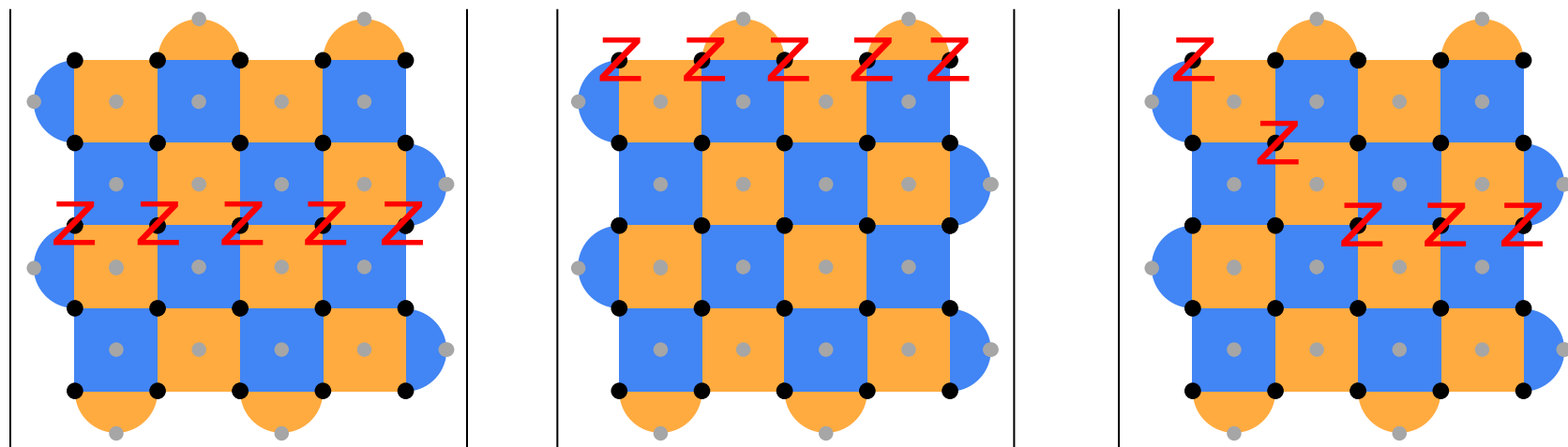
表面符号における論理 X 演算



いずれも論理 X 演算

- 2体の X -stabilizerが露出した境界同士を繋ぐ一連の X 操作が論理 X 演算になる

表面符号における論理Z演算

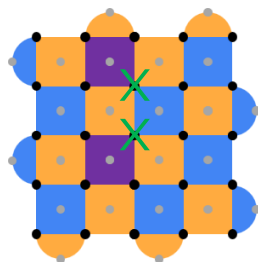
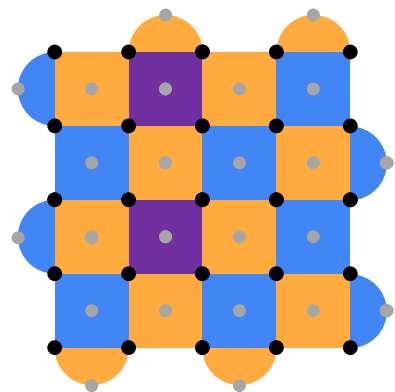


いずれも論理Z演算

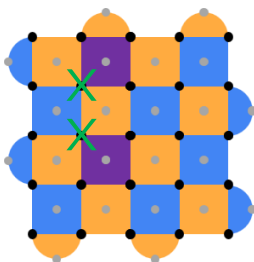
- 2体のZ-stabilizerが露出した境界同士を繋ぐ一連のZ操作が論理Z演算になる

表面符号におけるエラー推定（復号）

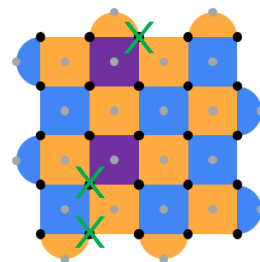
- シンドローム = エラーパリティチェックの結果（どこのパリティが反転しているか）からエラーの種類と位置を推定
- 一般にシンドロームを再現するエラーは一意には定まらない



エラー
パターン1



エラー
パターン2

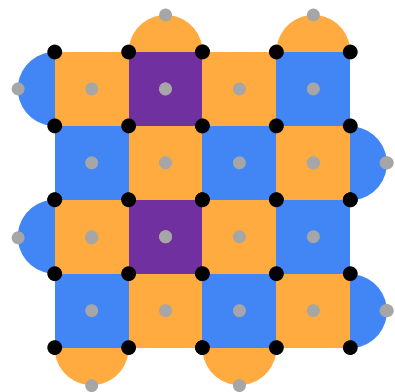


エラー
パターン3

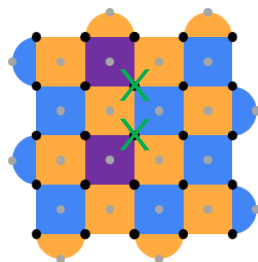
...

表面符号におけるエラー推定（復号）

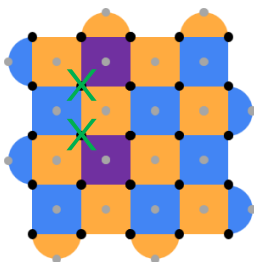
- シンドローム = エラーパリティチェックの結果（どのパリティが反転しているか）からエラーの種類と位置を推定
- 一般にシンドロームを再現するエラーは一意には定まらない



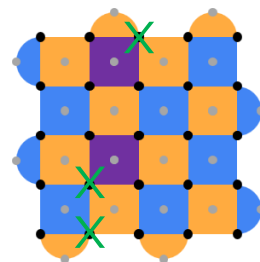
シンドロームを再現するのに
必要なエラーの数



エラー
パターン1
2



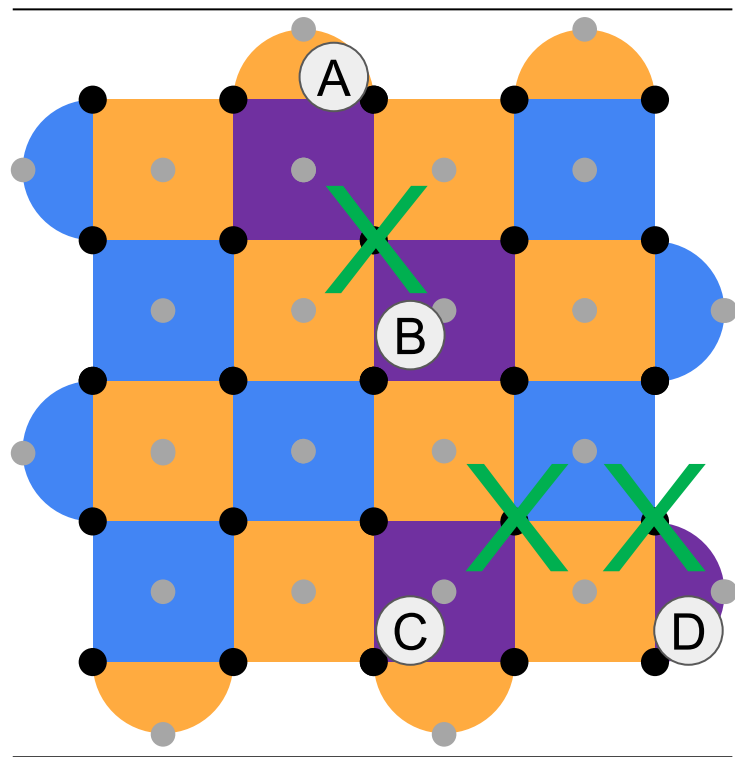
エラー
パターン2
2



エラー
パターン3
3

...

グラフマッチング問題への帰着

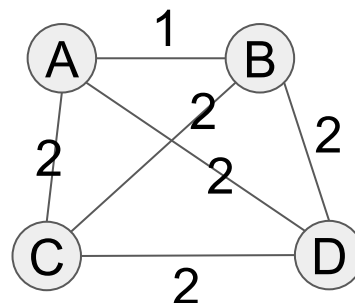


仮定

- XとZのエラーは独立に推定できる
- なるべく短いエラー鎖が生じる



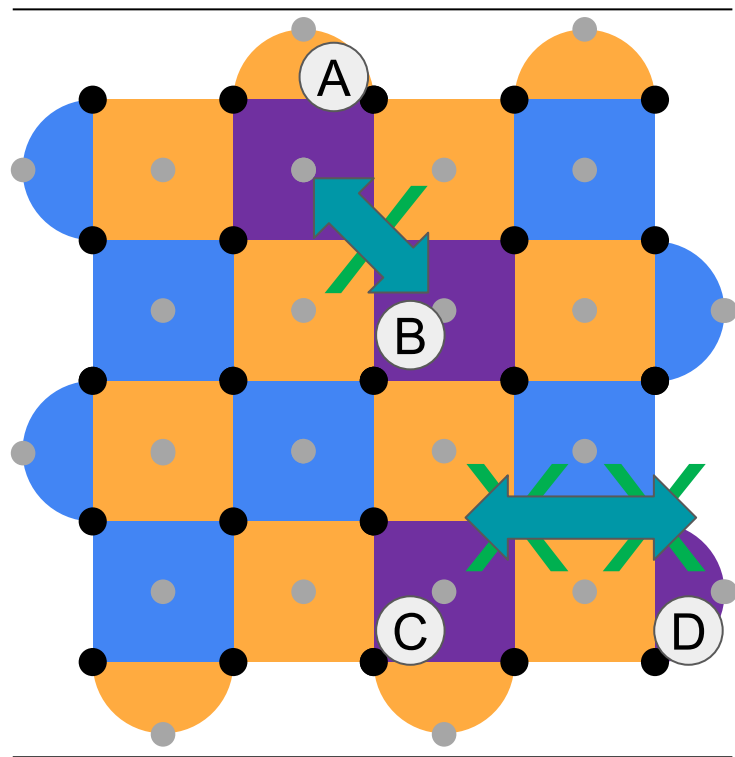
Minimum Weight Perfect Matching (MWPM)



V : Hot syndromes
 W_e : Manhattan distance

Exact solution: **Blossom algorithm** ($O(n^3)$)

グラフマッチング問題への帰着

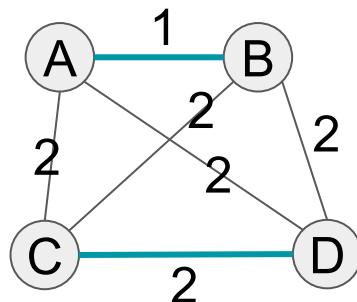


仮定

- XとZのエラーは独立に推定できる
- なるべく短いエラー鎖が生じる



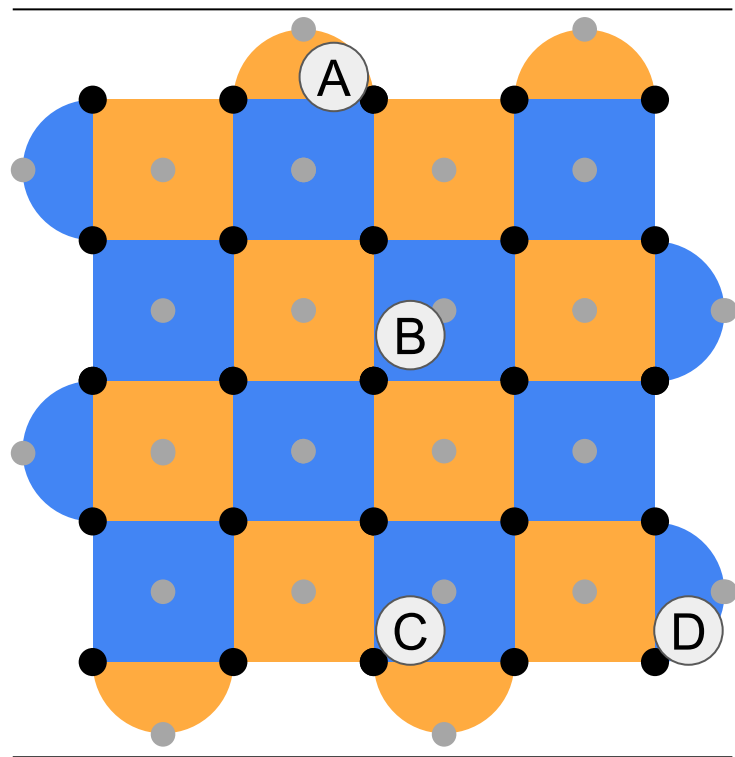
Minimum Weight Perfect Matching (MWPM)



V : Hot syndromes
 W_e : Manhattan distance

Exact solution: **Blossom algorithm** ($O(n^3)$)

グラフマッチング問題への帰着

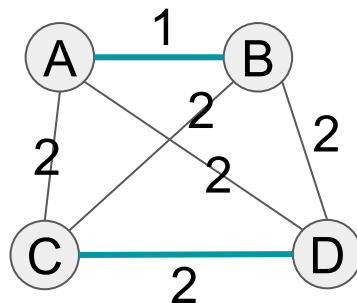


仮定

- XとZのエラーは独立に推定できる
- なるべく短いエラー鎖が生じる



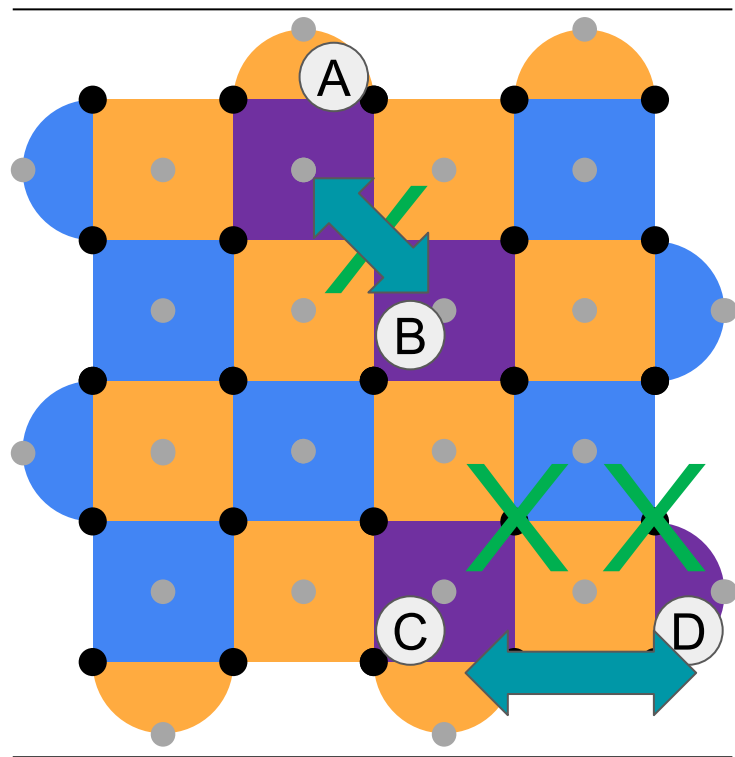
Minimum Weight Perfect Matching (MWPM)



V : Hot syndromes
 W_e : Manhattan distance

Exact solution: **Blossom algorithm** ($O(n^3)$)

異なる繋ぎ方をした場合

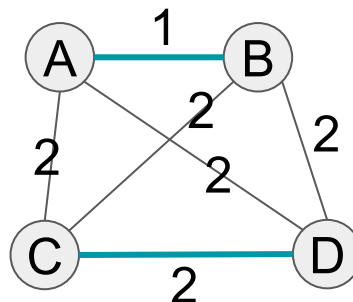


仮定

- XとZのエラーは独立に推定できる
- なるべく短いエラー鎖が生じる



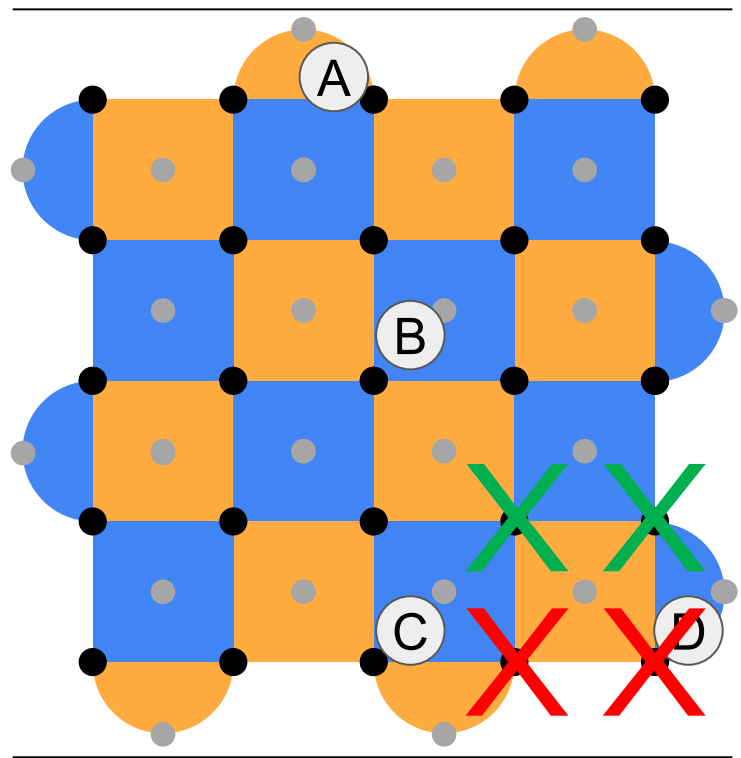
Minimum Weight Perfect Matching (MWPM)



V : Hot syndromes
 W_e : Manhattan distance

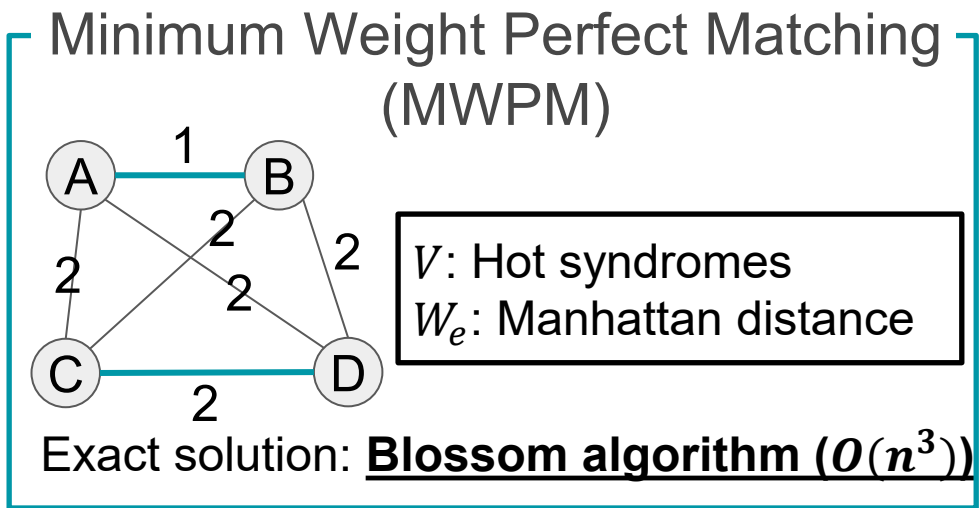
Exact solution: **Blossom algorithm** ($O(n^3)$)

異なる繋ぎ方をした場合

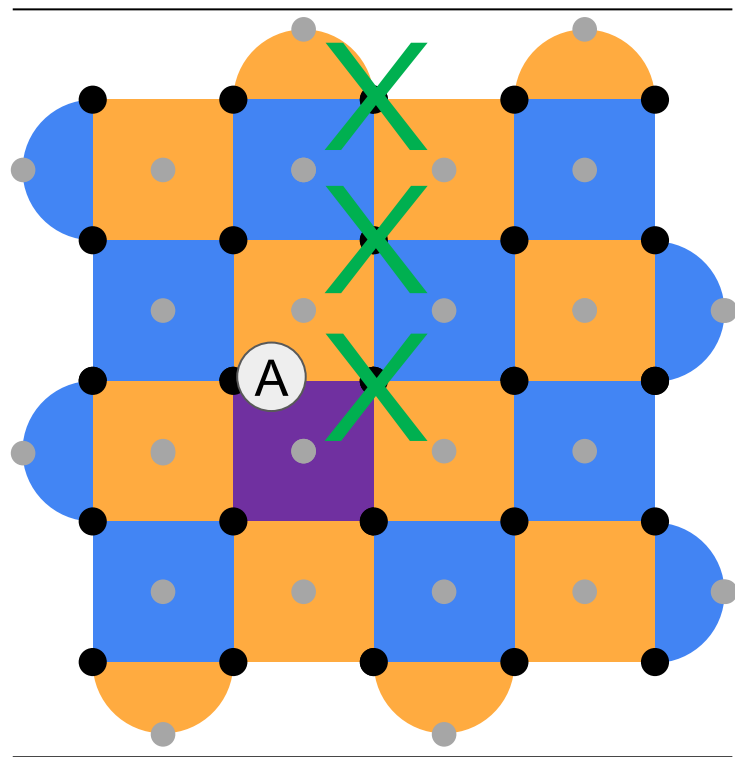


実際のエラー+訂正操作で論理演算になるため問題なし
->繋ぎ方に依らないためマッチングに帰着できる

- 仮定
- XとZのエラーは独立に推定できる
 - なるべく短いエラー鎖が生じる



エラー推定に失敗する場合

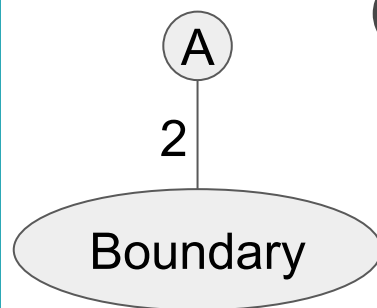


仮定

- XとZのエラーは独立に推定できる
- なるべく短いエラー鎖が生じる



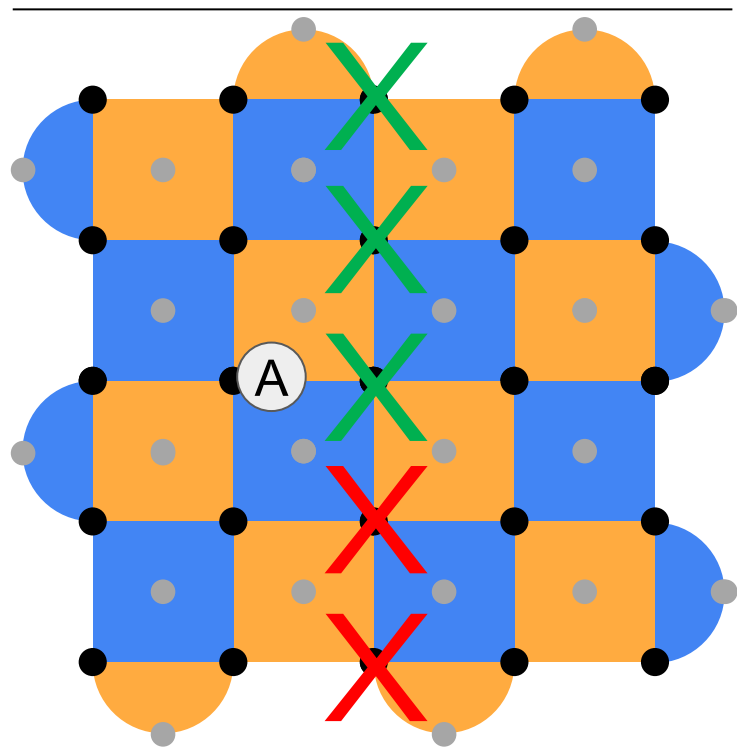
Minimum Weight Perfect Matching (MWPM)



V : Hot syndromes
 W_e : Manhattan distance

Exact solution: **Blossom algorithm** ($O(n^3)$)

エラー推定に失敗する場合



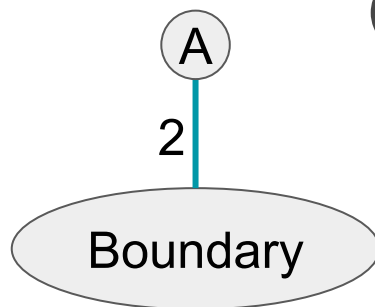
実際のエラー + 訂正操作で望まない論理X演算になる
一般に $\lfloor (d-1)/2 \rfloor$ 個以上のエラーは正しく推定できない

仮定

- XとZのエラーは独立に推定できる
- なるべく短いエラー鎖が生じる



Minimum Weight Perfect Matching (MWPM)



V : Hot syndromes
 W_e : Manhattan distance

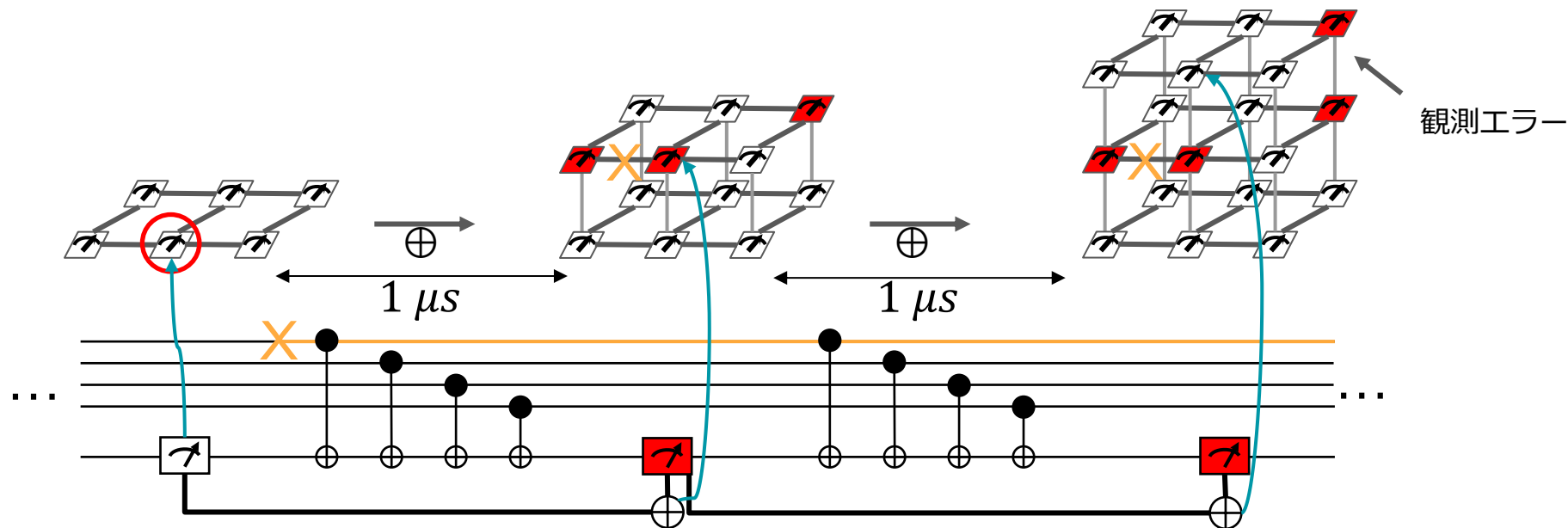
Exact solution: **Blossom algorithm ($O(n^3)$)**

補助量子ビットの観測エラーへの対応

1回目の観測

2回目の観測

3回目の観測



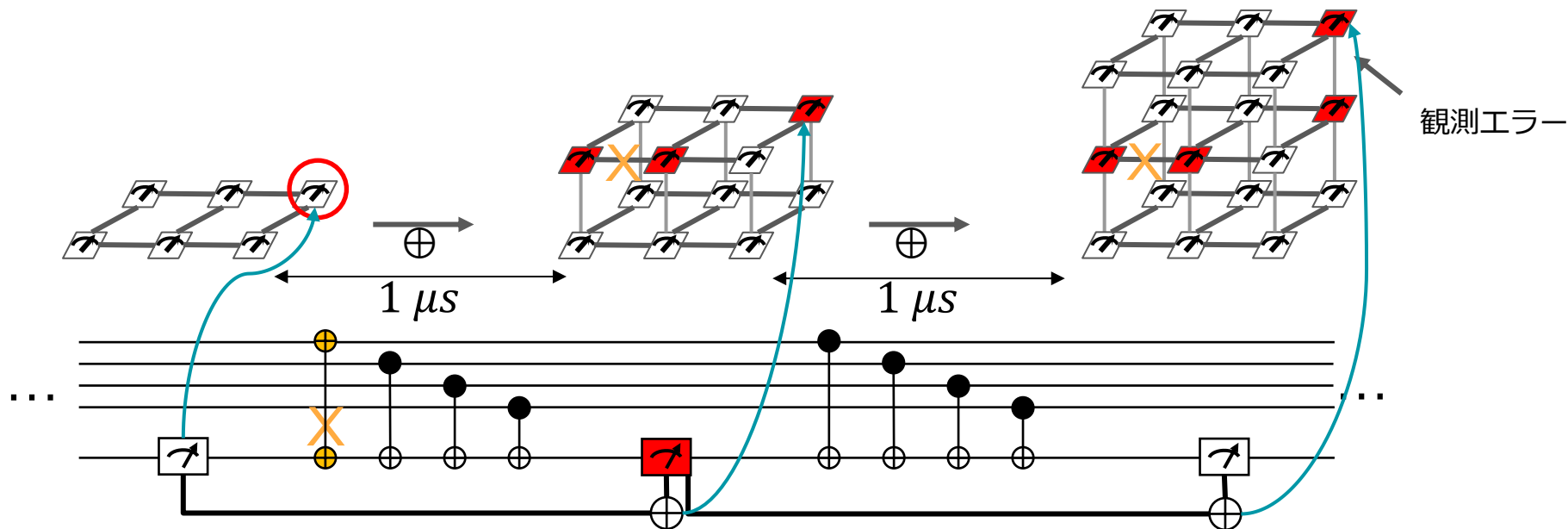
- データ量子ビットのエラーは訂正するまで残る、観測エラーは一時的
- 複数回観測をおこない観測値を垂直にスタックする
 - 前時刻の観測値とXORをとることで、観測値の変化した時点を表す
- エラー推定 = 3次元格子上でグラフマッチング問題を解く

補助量子ビットの観測エラーへの対応

1回目の観測

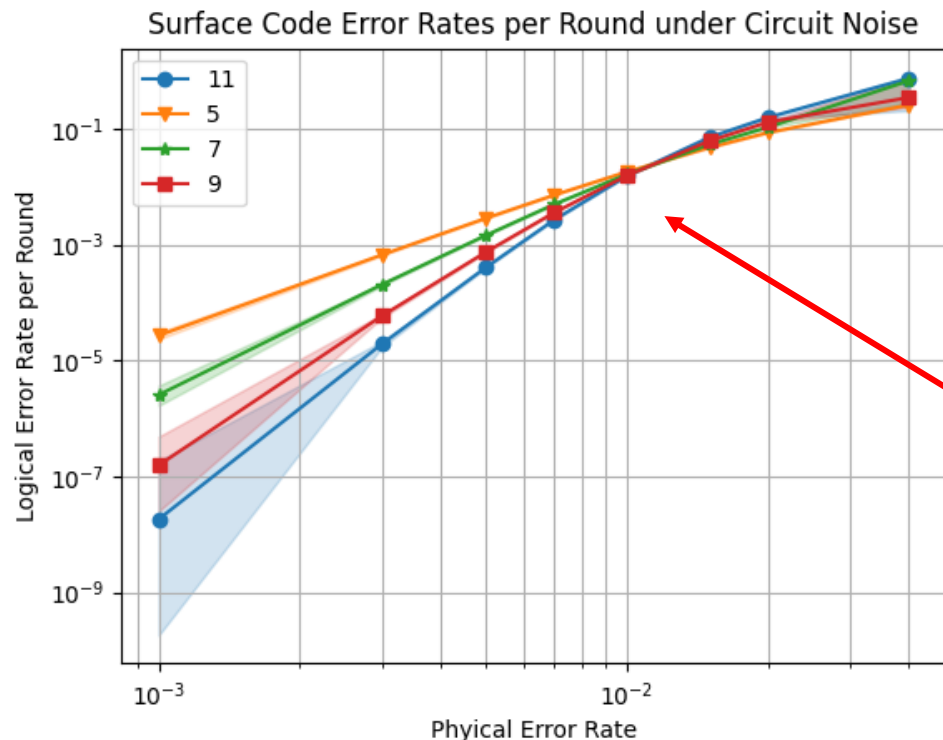
2回目の観測

3回目の観測



- データ量子ビットのエラーは訂正するまで残る、観測エラーは一時的
- 複数回観測をおこない観測値を垂直にスタックする
 - 前時刻の観測値とXORをとることで、観測値の変化した時点を表す
- エラー推定 = 3次元格子上でグラフマッチング問題を解く

表面符号の性能



しきい値
1%程度

- しきい値: 物理エラー率がこの値以下であれば、符号距離 d を増やせば指数的に論理エラー率を下げられる

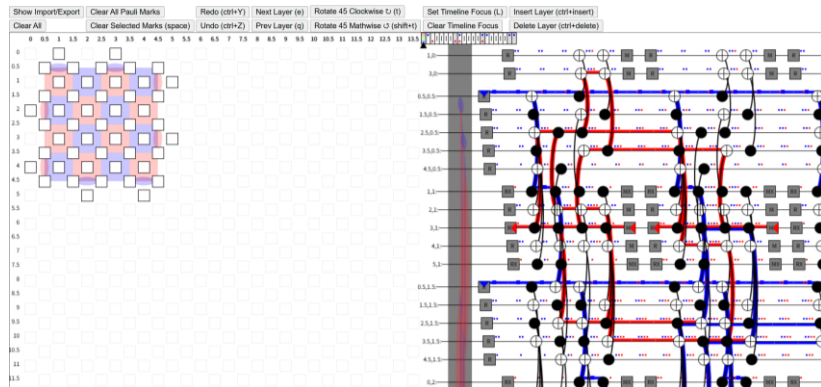
今日の内容

- 量子誤り訂正の基礎
 - 量子誤り訂正に求められる条件
 - 古典符号と量子符号
 - スタビライザ形式
 - Shorの符号
- 表面符号
 - 表面符号の誤り推定
 - Stim + Crumbleを用いた表面符号シミュレーション
 - 極低温環境での表面符号の誤り推定処理
- 表面符号 + 格子手術に基づく用いた誤り耐性量子計算
- まとめ

Stim+Crumbleを用いたシミュレーション



- Stim: Python経由で使えるClifford回路シミュレータ
 - 回路レベルノイズのシミュレーションを高速に行える
 - PyMatchingによる誤り推定、Sinterによる論理エラー率算出が容易
 - QEC符号の誤り訂正性能の評価ツールのデファクトスタンダード
- Crumble: スタビライザ回路とエラーイベントの可視化
 - Web上でインタラクティブにstim回路およびエラーイベントを可視化
 - Stim回路 ↔ Crumble双方のImport/Exportが可能



Stim+Crumbleを用いたシミュレーション

Stimコード例 ($d = 5$ のRotated surface code)

量子ビットの宣言

```
QUBIT_COORDS(0, 2) 0
QUBIT_COORDS(0, 4) 1
QUBIT_COORDS(0.5, 0.5) 2
QUBIT_COORDS(0.5, 1.5) 3
QUBIT_COORDS(0.5, 2.5) 4
QUBIT_COORDS(0.5, 3.5) 5
QUBIT_COORDS(0.5, 4.5) 6
QUBIT_COORDS(1, 0) 7
QUBIT_COORDS(1, 1) 8
QUBIT_COORDS(1, 2) 9
QUBIT_COORDS(1, 3) 10
...
QUBIT_COORDS(1, 4) 11
QUBIT_COORDS(5, 1) 47
QUBIT_COORDS(5, 3) 48
```

スタビライザ測定

```
TICK
R 2 3 4 5 6 12 13 14 15 16 22 23 24 ...
TICK
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```

Detectorの定義

```
DETECTOR(1, 2, 0) rec[-24]
DETECTOR(1, 4, 0) rec[-23]
DETECTOR(2, 1, 0) rec[-22]
DETECTOR(2, 3, 0) rec[-21]
DETECTOR(2, 5, 0) rec[-20]
DETECTOR(3, 2, 0) rec[-19]
DETECTOR(3, 4, 0) rec[-18]
DETECTOR(4, 1, 0) rec[-17]
DETECTOR(4, 3, 0) rec[-16]
DETECTOR(4, 5, 0) rec[-15]
DETECTOR(3, 0, 0) rec[-14]
DETECTOR(1, 0, 0) rec[-13]
TICK
R 9 11 17 19 21 29 31 37 39 41 7 27
RX 8 10 18 20 28 30 38 40 47 48 1 0
...
```

Stim+Crumbleを用いたシミュレーション

Stimコード例 ($d = 5$ のRotated surface code)

...  スタビライザ測定+Detectorの定義

```
TICK
R 2 3 4 5 6 12 13 14 15 16 22 23 24 ...
TICK
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```

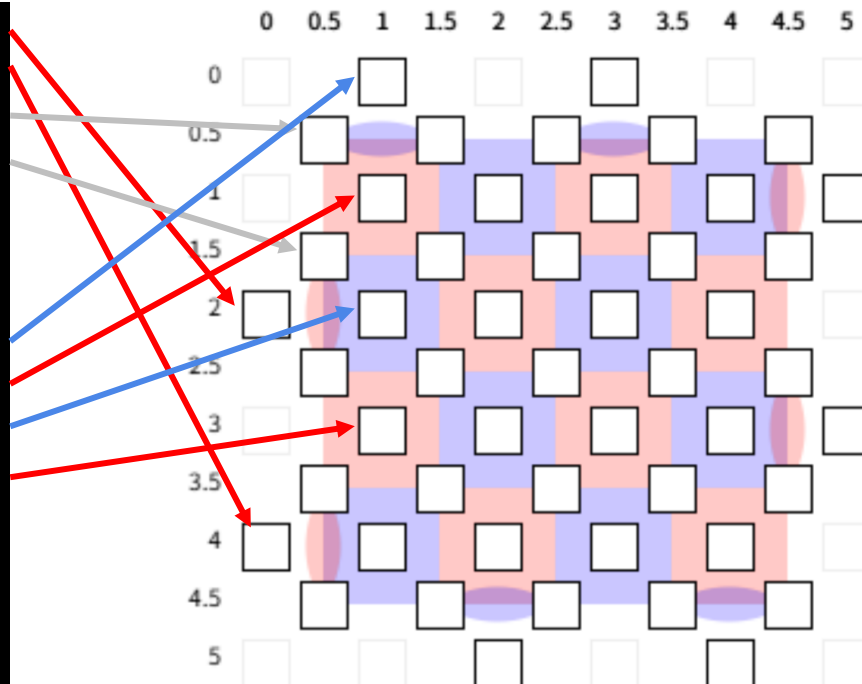
$\times O(d)$ 回

 論理測定

```
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
DETECTOR(1, 2, 2) rec[-24] rec[-48]
DETECTOR(1, 4, 2) rec[-23] rec[-47]
...
DETECTOR(3, 0, 2) rec[-14] rec[-38]
DETECTOR(1, 0, 2) rec[-13] rec[-37]
DETECTOR(1, 1, 2) rec[-12] rec[-36]
DETECTOR(1, 3, 2) rec[-11] rec[-35]
...
DETECTOR(0, 4, 2) rec[-1] rec[-25]
TICK
M 2 3 4 5 6 12 13 14 15 16 22 23 ..
```

量子ビットの宣言

```
QUBIT_COORDS(0, 2) 0
QUBIT_COORDS(0, 4) 1
QUBIT_COORDS(0.5, 0.5) 2
QUBIT_COORDS(0.5, 1.5) 3
QUBIT_COORDS(0.5, 2.5) 4
QUBIT_COORDS(0.5, 3.5) 5
QUBIT_COORDS(0.5, 4.5) 6
QUBIT_COORDS(1, 0) 7
QUBIT_COORDS(1, 1) 8
QUBIT_COORDS(1, 2) 9
QUBIT_COORDS(1, 3) 10
...
QUBIT_COORDS(1, 4) 11
QUBIT_COORDS(5, 1) 47
QUBIT_COORDS(5, 3) 48
```



データ量子ビット

[2, 3, 4, 5, 6,
12, 13, 14, 15, 16,
22, 23, 24, 25, 26,
32, 33, 34, 35, 36,
42, 43, 44, 45, 46]

補助量子ビット (Zエラー)

[0, 1, 8, 10, 18, 20,
28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー)

[7, 9, 11, 17, 19, 21,
27, 29, 31, 37, 39, 41]

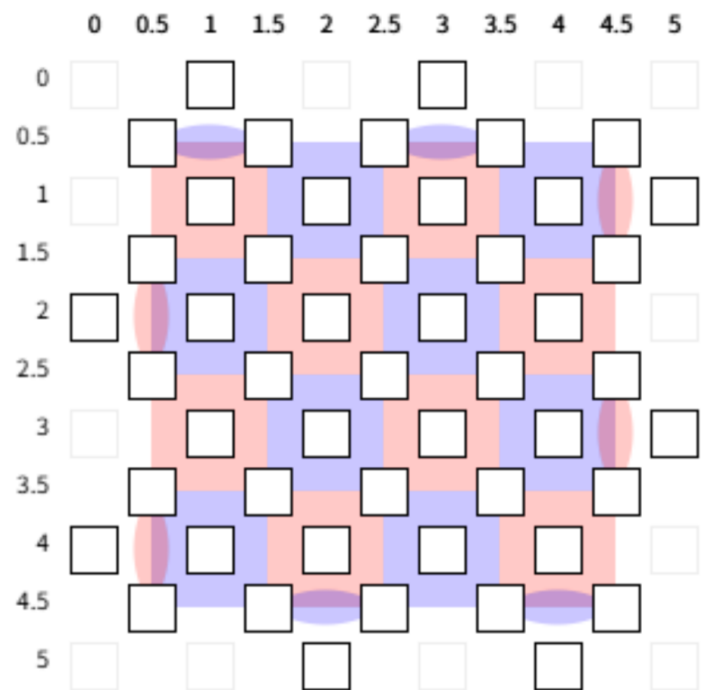
スタビライザ測定

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
TICK
R 2 3 4 5 6 12 13 14 15 16 22 23 24 ...
TICK
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



スタビライザ測定

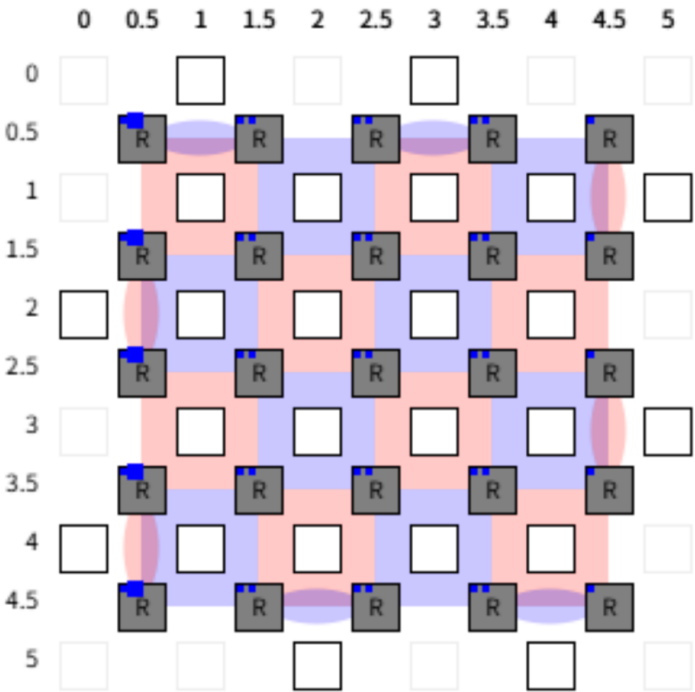
データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

データ量子ビットを
 $|0\rangle$ にリセット
-> 論理 $|0\rangle$ 状態の
メモリ操作

```
TICK
R 2 3 4 5 6 12 13 14 15 16 22 23 24 ...
TICK
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



スタビライザ測定

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

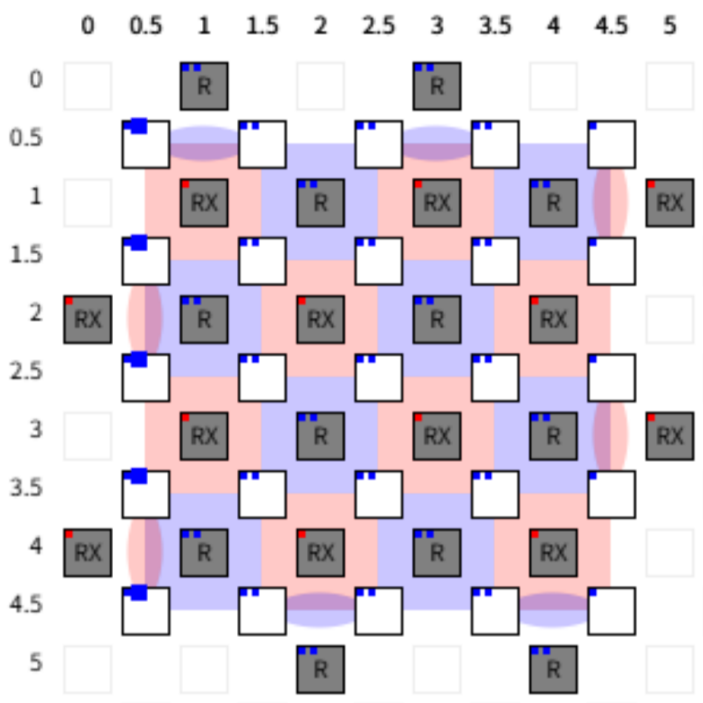
Xエラー検出用補助
量子ビットを

|0>にリセット

Zエラー検出用補助
量子ビットを

|+>にリセット

TICK	R	2	3	4	5	6	12	13	14	15	16	22	23	24	...
TICK	R	9	11	17	19	21	29	31	37	39	41	27	7		
	RX	8	10	18	20	28	30	38	40	47	48	0	1		
TICK	CX	8	2	3	9	10	4	5	11	12	17	18	13	14	...
TICK	CX	13	9	15	11	22	17	24	19	26	21	33	...		
TICK	CX	4	9	6	11	13	17	15	19	24	29	26	31	...	
TICK	CX	8	13	14	9	10	15	16	11	23	17	18	...		
TICK	M	9	11	17	19	21	29	31	37	39	41	27	7		
	MX	8	10	18	20	28	30	38	40	47	48	0	1		



スタビライザ測定

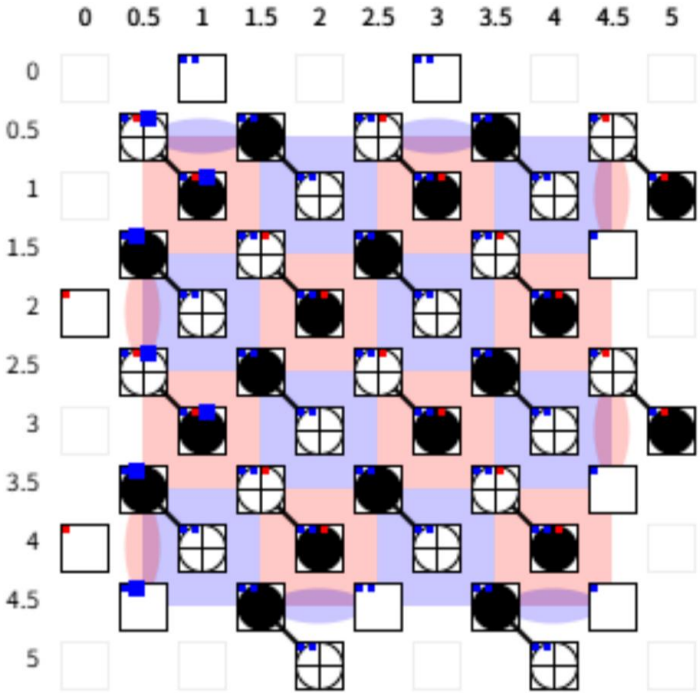
データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

データ量子ビットと
補助量子ビットの間
でCNOT (第1層)

```
TICK
R 2 3 4 5 6 12 13 14 15 16 22 23 24 ...
TICK
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



スタビライザ測定

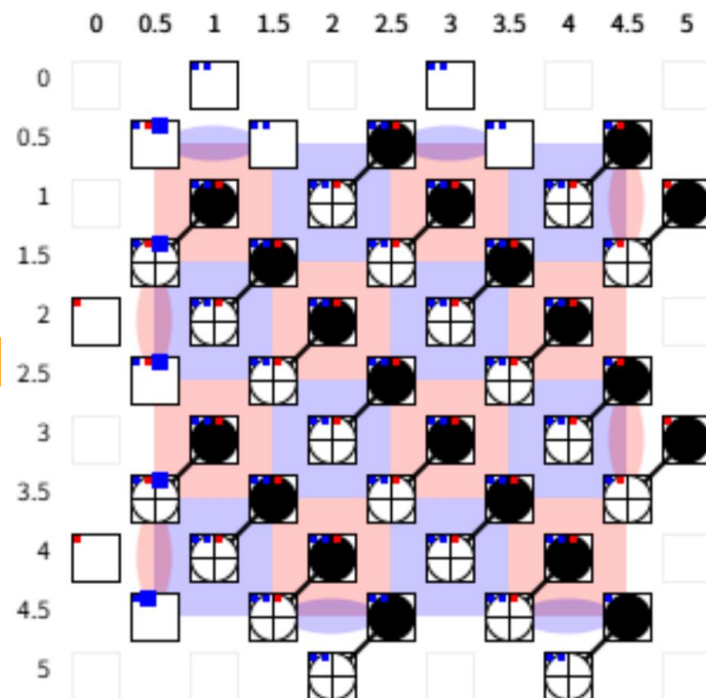
データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

データ量子ビットと
補助量子ビットの間
でCNOT (第2層)

```
TICK
R 2 3 4 5 6 12 13 14 15 16 22 23 24 ...
TICK
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



スタビライザ測定

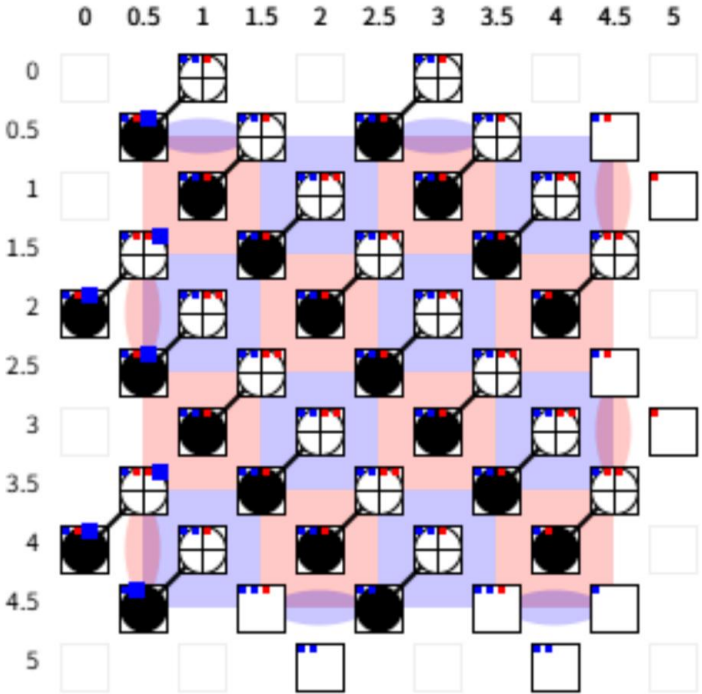
データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

データ量子ビットと
補助量子ビットの間
でCNOT (第3層)

```
TICK
R 2 3 4 5 6 12 13 14 15 16 22 23 24 ...
TICK
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



スタビライザ測定

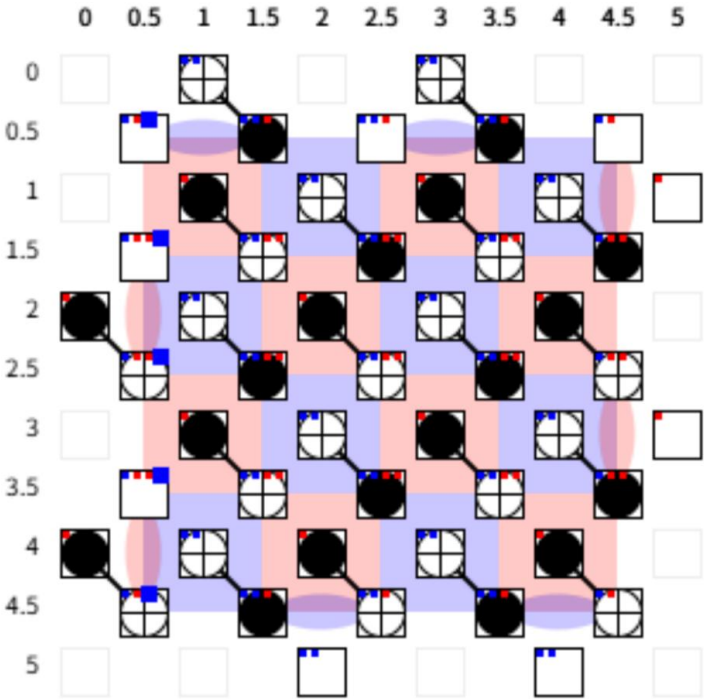
データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

データ量子ビットと
補助量子ビットの間
でCNOT (第4層)

```
TICK
R 2 3 4 5 6 12 13 14 15 16 22 23 24 ...
TICK
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



スタビライザ測定

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

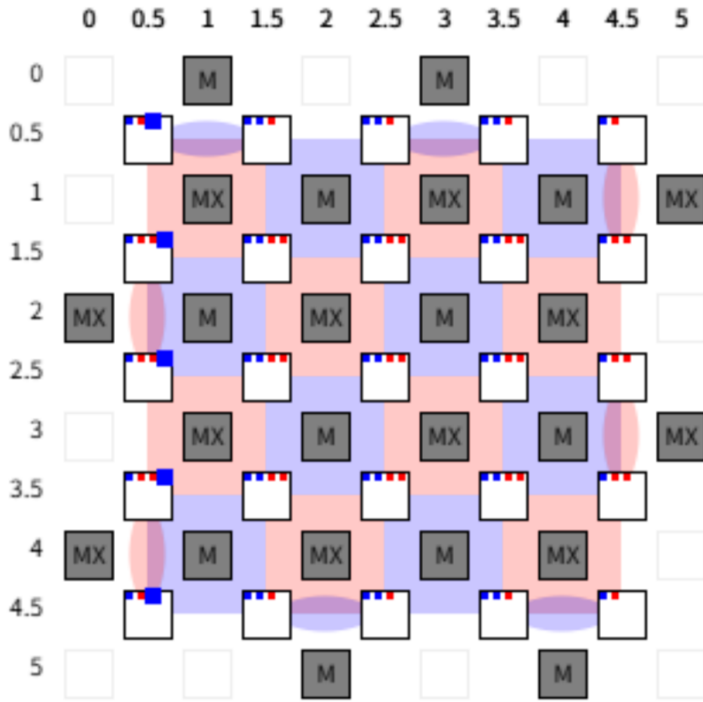
補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

Xエラー検出用補助
量子ビットを
Z基底で測定

Zエラー検出用補助
量子ビットを
X基底で測定

```
TICK
R 2 3 4 5 6 12 13 14 15 16 22 23 24 ...
TICK
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



Detectorの定義 (初回)

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```

```
DETECTOR(1, 2, 0) rec[-24]
DETECTOR(1, 4, 0) rec[-23]
DETECTOR(2, 1, 0) rec[-22]
DETECTOR(2, 3, 0) rec[-21]
DETECTOR(2, 5, 0) rec[-20]
DETECTOR(3, 2, 0) rec[-19]
DETECTOR(3, 4, 0) rec[-18]
DETECTOR(4, 1, 0) rec[-17]
DETECTOR(4, 3, 0) rec[-16]
DETECTOR(4, 5, 0) rec[-15]
DETECTOR(3, 0, 0) rec[-14]
DETECTOR(1, 0, 0) rec[-13]
```

TICK

```
R 9 11 17 19 21 29 31 37 39 41 7 27
RX 8 10 18 20 28 30 38 40 47 48 1 0
```

recの中身:

```
[
  (9の初回の測定値), <- rec[-24]
  (11の初回の測定値), <- rec[-23]
  ...,
  (7の初回の測定値), <- rec[-13]
  (8の初回の測定値), <- rec[-12]
  ...,
  (1の初回の測定値), <- rec[-1]
]
```

Detectorの定義 (初回)

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1

DETECTOR(1, 2, 0) rec[-24]
DETECTOR(1, 4, 0) rec[-23]
DETECTOR(2, 1, 0) rec[-22]
DETECTOR(2, 3, 0) rec[-21]
DETECTOR(2, 5, 0) rec[-20]
DETECTOR(3, 2, 0) rec[-19]
DETECTOR(3, 4, 0) rec[-18]
DETECTOR(4, 1, 0) rec[-17]
DETECTOR(4, 3, 0) rec[-16]
DETECTOR(4, 5, 0) rec[-15]
DETECTOR(3, 0, 0) rec[-14]
DETECTOR(1, 0, 0) rec[-13]

TICK

R 9 11 17 19 21 29 31 37 39 41 7 27
RX 8 10 18 20 28 30 38 40 47 48 1 0
```

マッチング問題を解く
グラフのノードを定義

Xエラー検出用の
補助量子ビット :
観測値が-1であれば
ホットシンδροーム

Zエラー検出用の
補助量子ビット :
初回の測定値は完全にランダムな値
になるが論理Zエラーは論理|0>に影
響を与えないので初回は無視でOK

recの中身:

```
[
(9の初回の測定値), <- rec[-24]
(11の初回の測定値), <- rec[-23]
...,
(7の初回の測定値), <- rec[-13]
(8の初回の測定値), <- rec[-12]
...,
(1の初回の測定値), <- rec[-1]
]
```

Detectorの定義 (初回)

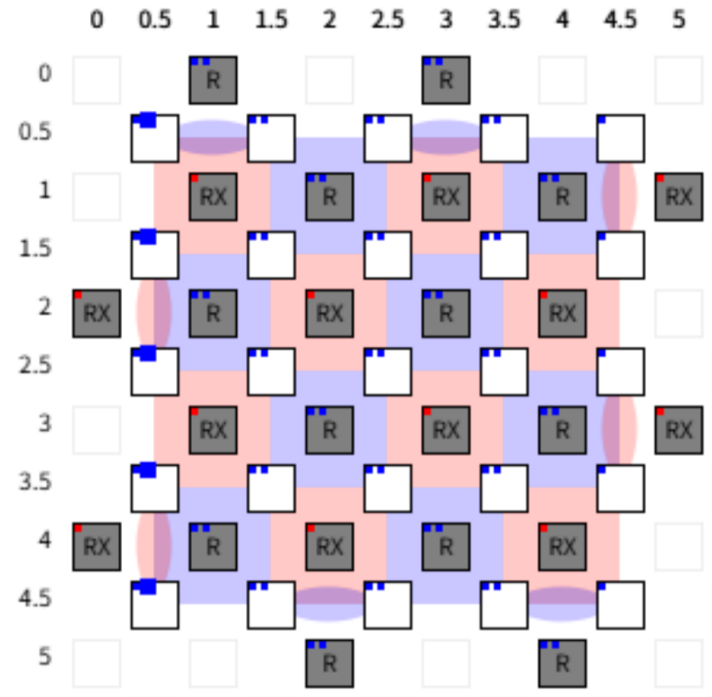
データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
DETECTOR(1, 2, 0) rec[-24]
DETECTOR(1, 4, 0) rec[-23]
DETECTOR(2, 1, 0) rec[-22]
DETECTOR(2, 3, 0) rec[-21]
DETECTOR(2, 5, 0) rec[-20]
DETECTOR(3, 2, 0) rec[-19]
DETECTOR(3, 4, 0) rec[-18]
DETECTOR(4, 1, 0) rec[-17]
DETECTOR(4, 3, 0) rec[-16]
DETECTOR(4, 5, 0) rec[-15]
DETECTOR(3, 0, 0) rec[-14]
DETECTOR(1, 0, 0) rec[-13]
TICK
R 9 11 17 19 21 29 31 37 39 41 7 27
RX 8 10 18 20 28 30 38 40 47 48 1 0
```

補助量子ビットをそれぞれ
 $|0\rangle, |+\rangle$ にリセット



スタビライザ測定 (2回目)

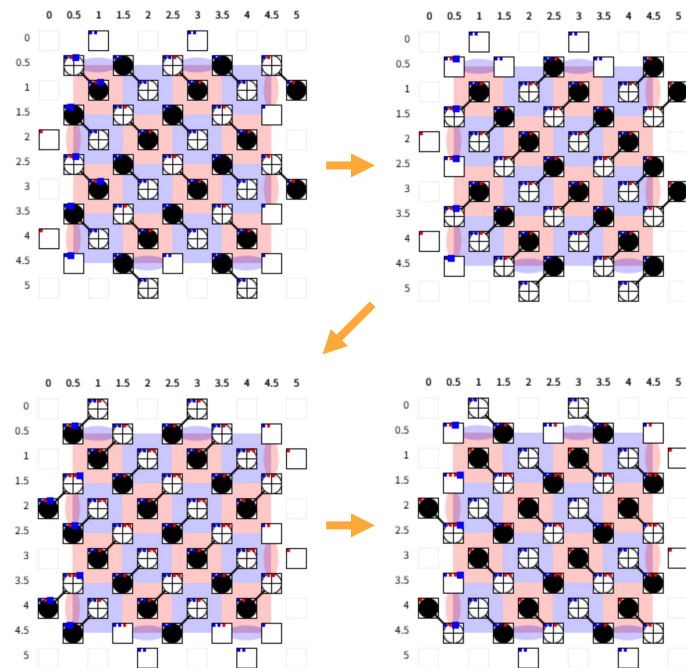
データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

データ量子ビットと
補助量子ビットの間
でCNOT×4

```
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



スタビライザ測定 (2回目)

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

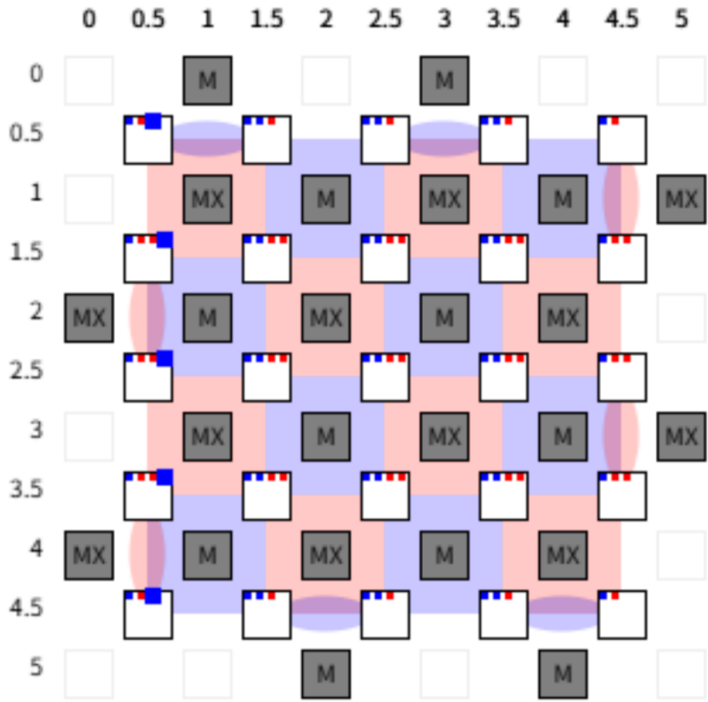
補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

Xエラー検出用補助
量子ビットを
Z基底で測定

Zエラー検出用補助
量子ビットを
X基底で測定

```
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



Detectorの定義 (2回目)

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```

```
DETECTOR(1, 2, 1) rec[-24] rec[-48]
DETECTOR(1, 4, 1) rec[-23] rec[-47]
...
DETECTOR(3, 0, 1) rec[-14] rec[-38]
DETECTOR(1, 0, 1) rec[-13] rec[-37]
DETECTOR(1, 1, 1) rec[-12] rec[-36]
DETECTOR(1, 3, 1) rec[-11] rec[-35]
...
DETECTOR(0, 4, 1) rec[-1] rec[-25]
TICK
R 9 11 17 19 21 29 31 37 39 41 7 27
RX 8 10 18 20 28 30 38 40 47 48 1 0
```

recの中身:

```
[
  (9の初回の測定値), <- rec[-48]
  ...,
  (7の初回の測定値), <- rec[-37]
  (8の初回の測定値), <- rec[-36]
  ...,
  (1の初回の測定値), <- rec[-25]
  (9の2回目の測定値), <- rec[-24]
  ...
  (1の2回目の測定値), <- rec[-1]
]
```

Detectorの定義 (2回目)

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1

DETECTOR(1, 2, 1) rec[-24] rec[-48]
DETECTOR(1, 4, 1) rec[-23] rec[-47]
...
DETECTOR(3, 0, 1) rec[-14] rec[-38]
DETECTOR(1, 0, 1) rec[-13] rec[-37]
DETECTOR(1, 1, 1) rec[-12] rec[-36]
DETECTOR(1, 3, 1) rec[-11] rec[-35]
...
DETECTOR(0, 4, 1) rec[-1] rec[-25]

TICK
R 9 11 17 19 21 29 31 37 39 41 7 27
RX 8 10 18 20 28 30 38 40 47 48 1 0
```

マッチング問題を解く
グラフのノードを定義

X/Zエラー検出用の
補助量子ビット :
前回の観測値との差分を
取り、異なっていれば
ホットシンδροーム
-> 前回の測定から新た
に生じたエラーを検出

recの中身:

```
[
  (9の初回の測定値),      <- rec[-48]
  ...,
  (7の初回の測定値),      <- rec[-37]
  (8の初回の測定値),      <- rec[-36]
  ...,
  (1の初回の測定値),      <- rec[-25]
  (9の2回目の測定値),     <- rec[-24]
  ...,
  (1の2回目の測定値),     <- rec[-1]
]
```

Detectorの定義 (2回目)

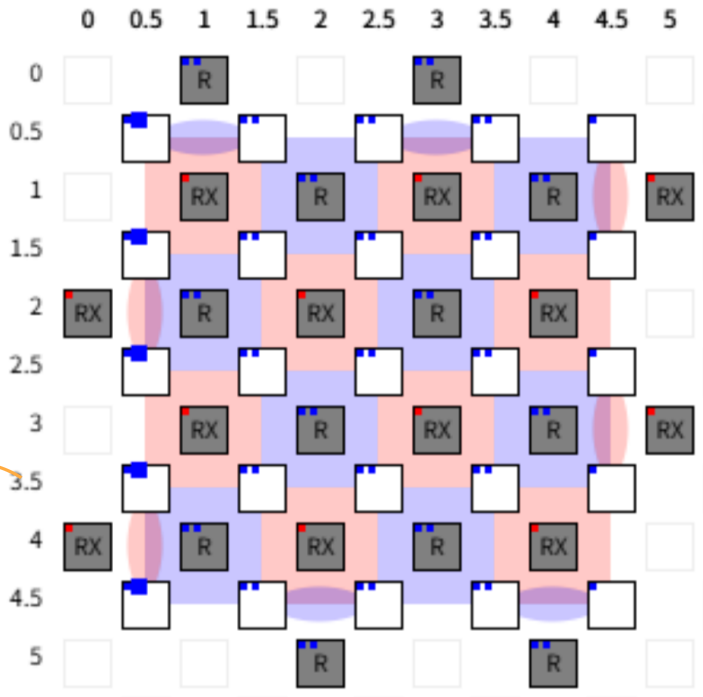
データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
DETECTOR(1, 2, 1) rec[-24] rec[-48]
DETECTOR(1, 4, 1) rec[-23] rec[-47]
...
DETECTOR(3, 0, 1) rec[-14] rec[-38]
DETECTOR(1, 0, 1) rec[-13] rec[-37]
DETECTOR(1, 1, 1) rec[-12] rec[-36]
DETECTOR(1, 3, 1) rec[-11] rec[-35]
...
DETECTOR(0, 4, 1) rec[-1] rec[-25]
TICK
R 9 11 17 19 21 29 31 37 39 41 7 27
RX 8 10 18 20 28 30 38 40 47 48 1 0
```

補助量子ビットを
それぞれ $|0\rangle, |+\rangle$ にリセット



スタビライザ測定 (3回目)

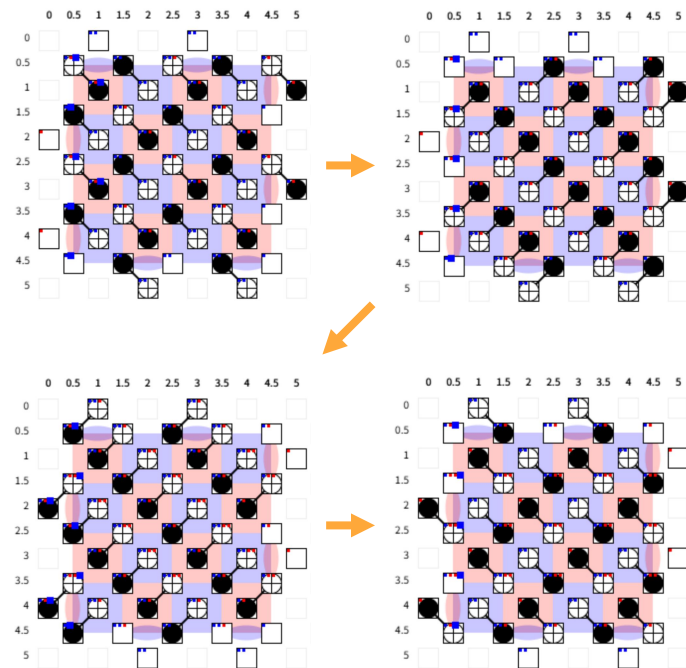
データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

データ量子ビットと
補助量子ビットの間
でCNOT×4

```
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



スタビライザ測定 (3回目)

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

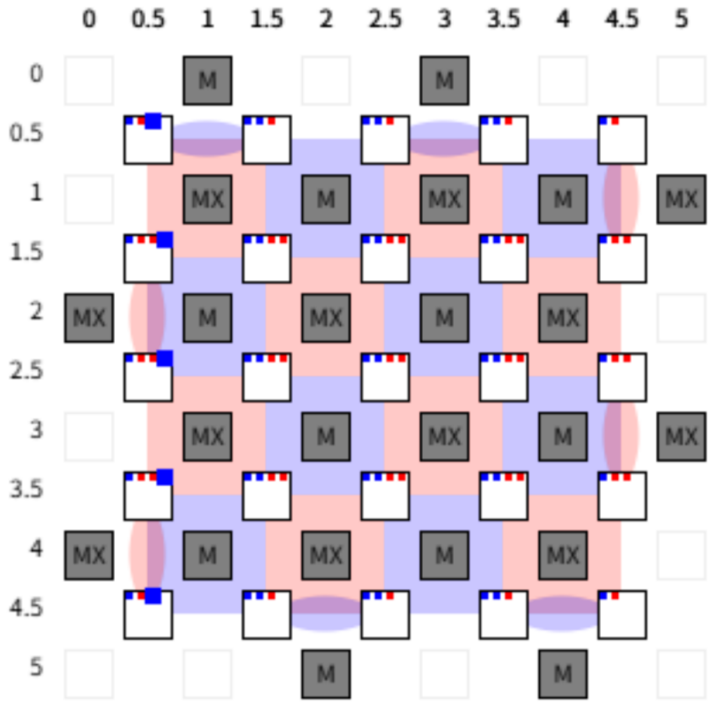
補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

Xエラー検出用補助
量子ビットを
Z基底で測定

Zエラー検出用補助
量子ビットを
X基底で測定

```
R 9 11 17 19 21 29 31 37 39 41 27 7
RX 8 10 18 20 28 30 38 40 47 48 0 1
TICK
CX 8 2 3 9 10 4 5 11 12 17 18 13 14 ...
TICK
CX 13 9 15 11 22 17 24 19 26 21 33 ...
TICK
CX 4 9 6 11 13 17 15 19 24 29 26 31 ...
TICK
CX 8 13 14 9 10 15 16 11 23 17 18 ...
TICK
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```



Detectorの定義 (3回目)

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
```

```
DETECTOR(1, 2, 2) rec[-24] rec[-48]
DETECTOR(1, 4, 2) rec[-23] rec[-47]
...
DETECTOR(3, 0, 2) rec[-14] rec[-38]
DETECTOR(1, 0, 2) rec[-13] rec[-37]
DETECTOR(1, 1, 2) rec[-12] rec[-36]
DETECTOR(1, 3, 2) rec[-11] rec[-35]
...
DETECTOR(0, 4, 2) rec[-1] rec[-25]
TICK
M 2 3 4 5 6 12 13 14 15 16 22 23 ...
```

recの中身:

```
[
  (9の初回の測定値), <- rec[-72]
  ...,
  (1の初回の測定値), <- rec[-49]
  (9の2回目の測定値), <- rec[-48]
  ...
  (1の2回目の測定値), <- rec[-25]
  (9の3回目の測定値), <- rec[-24]
  ...
  (1の3回目の測定値), <- rec[-1]
]
```

Detectorの定義 (3回目)

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
DETECTOR(1, 2, 2) rec[-24] rec[-48]
DETECTOR(1, 4, 2) rec[-23] rec[-47]
...
DETECTOR(3, 0, 2) rec[-14] rec[-38]
DETECTOR(1, 0, 2) rec[-13] rec[-37]
DETECTOR(1, 1, 2) rec[-12] rec[-36]
DETECTOR(1, 3, 2) rec[-11] rec[-35]
...
DETECTOR(0, 4, 2) rec[-1] rec[-25]
TICK
M 2 3 4 5 6 12 13 14 15 16 22 23 ...
```

マッチング問題を解く
グラフのノードを定義

X/Zエラー検出用の
補助量子ビット :
前回の観測値との差分を
取り、異なっていれば
ホットシンドローム
-> 前回の測定から新た
に生じたエラーを検出

recの中身:
[
 (9の初回の測定値), <- rec[-72]
 ...,
 (1の初回の測定値), <- rec[-49]
 (9の2回目の測定値), <- rec[-48]
 ...,
 (1の2回目の測定値), <- rec[-25]
 (9の3回目の測定値), <- rec[-24]
 ...,
 (1の3回目の測定値), <- rec[-1]
]

論理測定

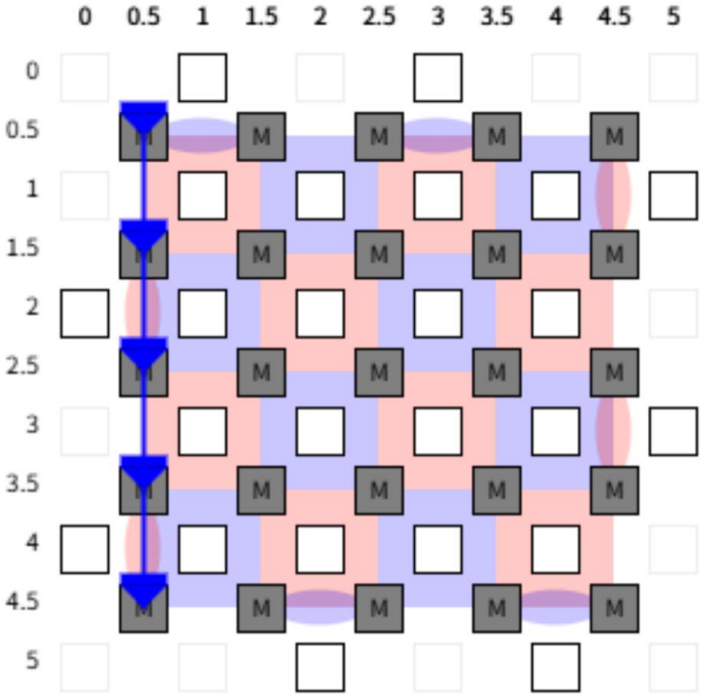
データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
M 9 11 17 19 21 29 31 37 39 41 27 7
MX 8 10 18 20 28 30 38 40 47 48 0 1
DETECTOR(1, 2, 2) rec[-24] rec[-48]
DETECTOR(1, 4, 2) rec[-23] rec[-47]
...
DETECTOR(3, 0, 2) rec[-14] rec[-38]
DETECTOR(1, 0, 2) rec[-13] rec[-37]
DETECTOR(1, 1, 2) rec[-12] rec[-36]
DETECTOR(1, 3, 2) rec[-11] rec[-35]
...
DETECTOR(0, 4, 2) rec[-1] rec[-25]
TICK
M 2 3 4 5 6 12 13 14 15 16 22 23 ..
```

データ量子ビットを
Z基底で測定



Detectorの定義 (最後)

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

M 2 3 4 5 6 12 13 14 15 16 22 23 24 25 26 32 33 34 35 36 42 43 44 45 46

```
DETECTOR(1, 0, 3) rec[-20] rec[-25] rec[-39]
DETECTOR(1, 2, 3) rec[-18] rec[-19] rec[-23] rec[-24] rec[-49]
DETECTOR(1, 4, 3) rec[-16] rec[-17] rec[-21] rec[-22] rec[-48]
DETECTOR(2, 1, 3) rec[-14] rec[-15] rec[-19] rec[-20] rec[-47]
DETECTOR(2, 3, 3) rec[-12] rec[-13] rec[-17] rec[-18] rec[-46]
DETECTOR(2, 5, 3) rec[-11] rec[-16] rec[-45]
DETECTOR(3, 0, 3) rec[-10] rec[-15] rec[-38]
DETECTOR(3, 2, 3) rec[-8] rec[-9] rec[-13] rec[-14] rec[-44]
DETECTOR(3, 4, 3) rec[-6] rec[-7] rec[-11] rec[-12] rec[-43]
DETECTOR(4, 1, 3) rec[-4] rec[-5] rec[-9] rec[-10] rec[-42]
DETECTOR(4, 3, 3) rec[-2] rec[-3] rec[-7] rec[-8] rec[-41]
DETECTOR(4, 5, 3) rec[-1] rec[-6] rec[-40]
```

```
OBSERVABLE_INCLUDE(0) rec[-21] rec[-22] rec[-23] rec[-24] rec[-25]
```

Xエラー検出用の補助量子ビットの**前回の観測値**とデータ量子ビットの観測値を比較してDetectorを定義
-> データ量子ビットに生じるエラーと測定エラーを区別なく扱うため

データ量子ビットをZ基底で測定しているので、
論理Zエラーの影響はない

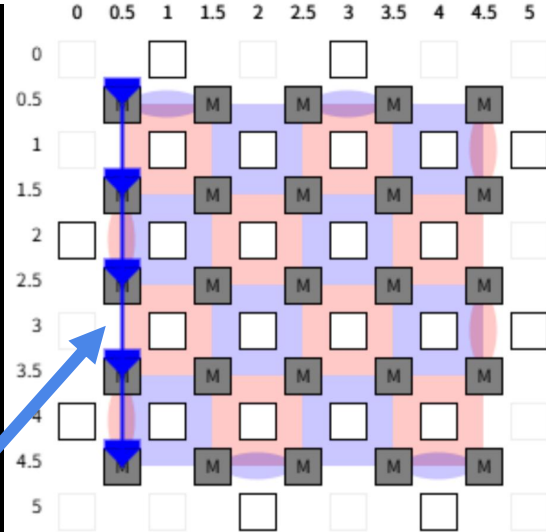
Observableの定義

データ量子ビット : [2, 3, 4, 5, 6, 12, 13, 14, 15, 16, 22, 23, 24, 25, 26, 32, 33, 34, 35, 36, 42, 43, 44, 45, 46]

補助量子ビット (Zエラー検出用) : [0, 1, 8, 10, 18, 20, 28, 30, 38, 40, 47, 48]

補助量子ビット (Xエラー検出用) : [7, 9, 11, 17, 19, 21, 27, 29, 31, 37, 39, 41]

```
M 2 3 4 5 6 12 13 14 15 16 22 23 24 25 26 32 33 34 35 36 42 43 44 45 46
DETECTOR(1, 0, 3) rec[-20] rec[-25] rec[-39]
DETECTOR(1, 2, 3) rec[-18] rec[-19] rec[-23] rec[-24] rec[-49]
DETECTOR(1, 4, 3) rec[-16] rec[-17] rec[-21] rec[-22] rec[-48]
DETECTOR(2, 1, 3) rec[-14] rec[-15] rec[-19] rec[-20] rec[-47]
DETECTOR(2, 3, 3) rec[-12] rec[-13] rec[-17] rec[-18] rec[-46]
DETECTOR(2, 5, 3) rec[-11] rec[-16] rec[-45]
DETECTOR(3, 0, 3) rec[-10] rec[-15] rec[-38]
DETECTOR(3, 2, 3) rec[-8] rec[-9] rec[-13] rec[-14] rec[-44]
DETECTOR(3, 4, 3) rec[-6] rec[-7] rec[-11] rec[-12] rec[-43]
DETECTOR(4, 1, 3) rec[-4] rec[-5] rec[-9] rec[-10] rec[-42]
DETECTOR(4, 3, 3) rec[-2] rec[-3] rec[-7] rec[-8] rec[-41]
DETECTOR(4, 5, 3) rec[-1] rec[-6] rec[-40]
OBSERVABLE_INCLUDE(0) rec[-21] rec[-22] rec[-23] rec[-24] rec[-25]
```



論理Z演算子 (の1つ) を
observableとして指定

今日の内容

- 量子誤り訂正の基礎
 - 量子誤り訂正に求められる条件
 - 古典符号と量子符号
 - スタビライザ形式
 - Shorの符号
- 表面符号
 - 表面符号の誤り推定
 - Stim + Crumbleを用いた表面符号シミュレーション
 - 極低温環境での表面符号の誤り推定処理
- 表面符号 + 格子手術に基づく用いた誤り耐性量子計算
- まとめ

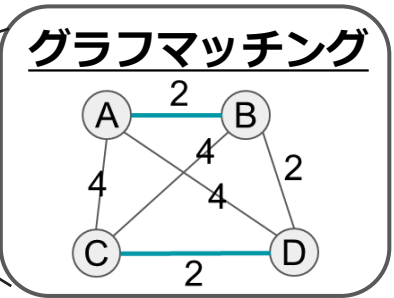
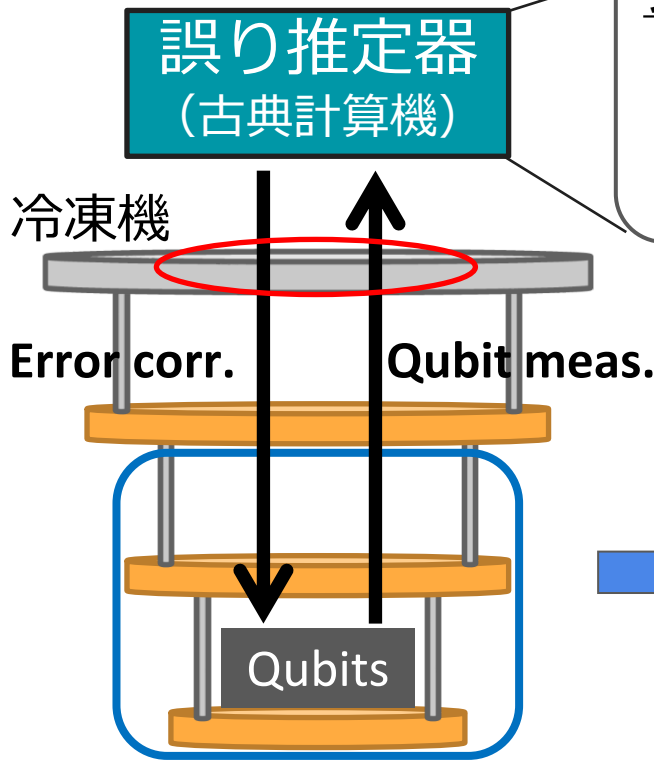
QECool: On-Line Quantum Error Correction with a
Superconducting Decoder for Surface Code [DAC'21]
&
QULATIS: A Quantum Error Correction Methodology
toward Lattice Surgery [HPCA'22]

Yosuke Ueno¹, Masaaki Kondo¹, Masamitsu Tanaka²,
Yasunari Suzuki³, Yutaka Tabuchi⁴

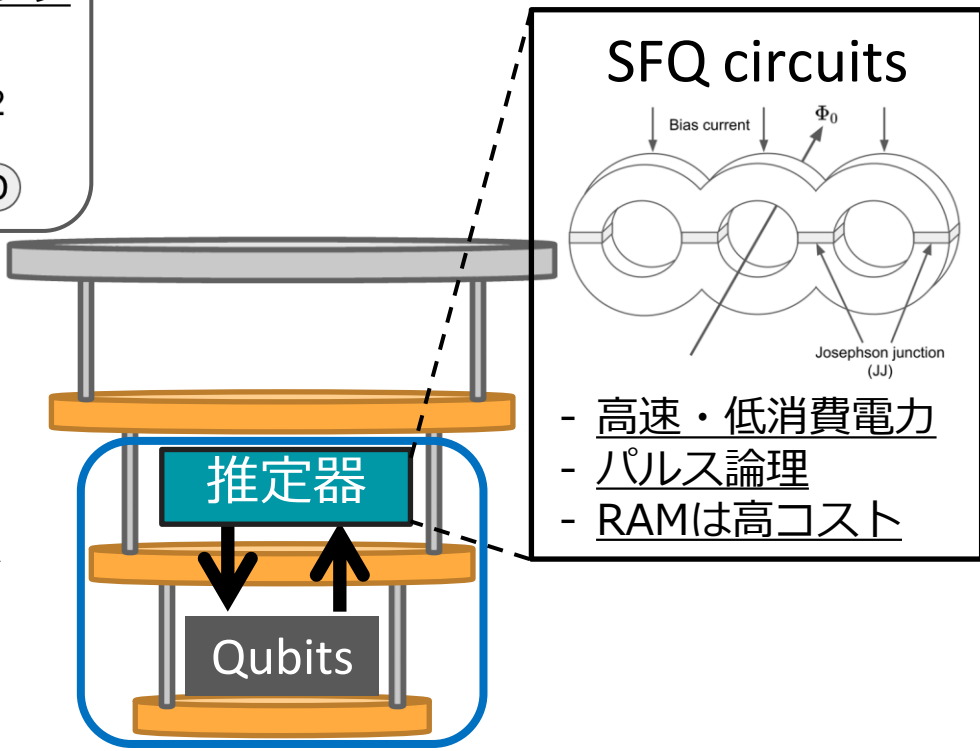
¹The University of Tokyo ²Nagoya University ³NTT ⁴RIKEN

超伝導量子ビット+極低温環境で動作する誤り推定器

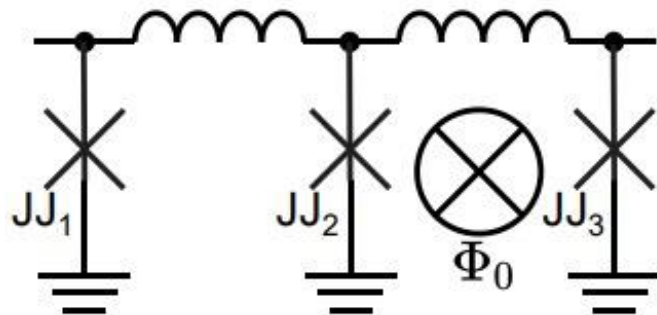
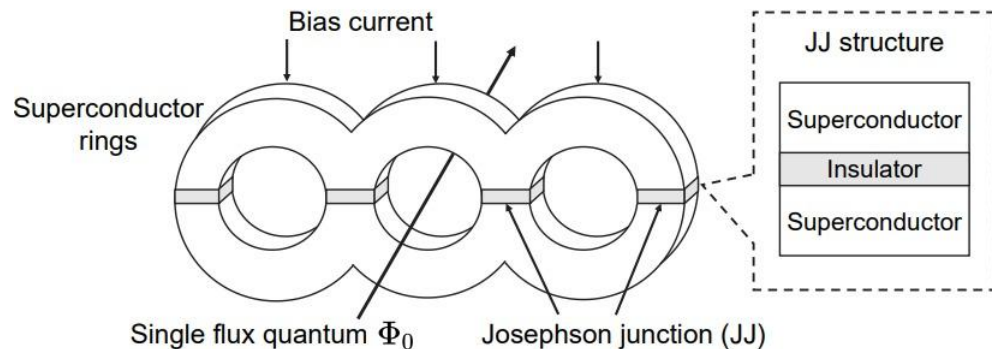
従来アーキテクチャ



提案アーキテクチャ

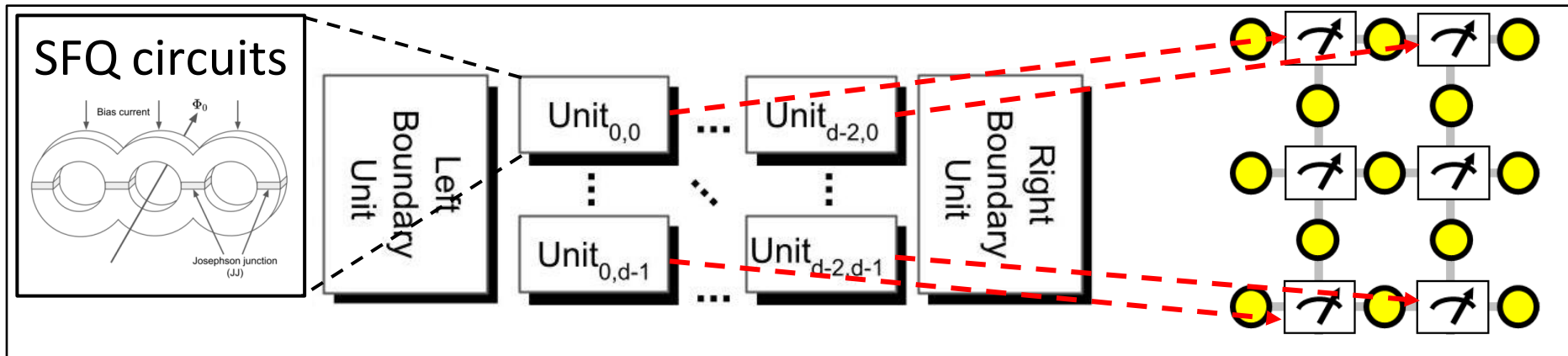


超伝導古典回路によるエラー推定



- 単一磁束量子 (SFQ: Single Flux Quantum) 回路
- 超伝導リング内の磁束量子の有無で0 or 1を表現
- 4K程度の極低温環境でのみ動作
- CMOSに比べて高速・低消費電力
- 大規模なメモリの構築は難しい
 - Blossom AlgorithmをSFQで実行するのは現実的でない

提案手法: QECOOOL

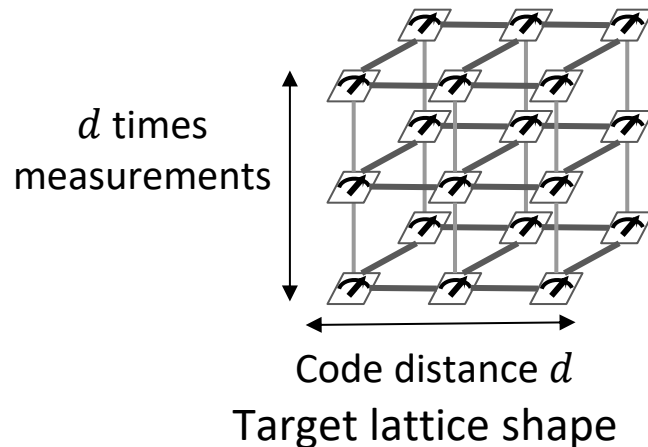
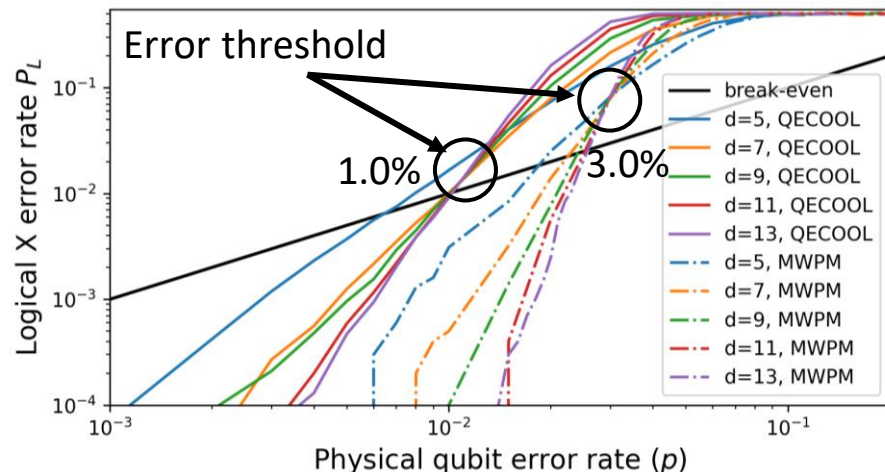


QECOOOLのアーキテクチャ

- Quantum Error COrrrection by On-Line decoding algorithm
- 大規模なRAMを必要としない分散型のアーキテクチャ
 - 補助量子ビットに1対1に対応するUnitを導入
 - Unit同士の3種類の信号伝播によりマッチング問題を解く

Y. Ueno, M. Kondo, M. Tanaka, Y. Suzuki, Y. Tabuchi, "QECOOOL: On-Line Quantum Error Correction with a Superconducting Decoder for Surface Code", 58th IEEE/ACM Design Automation Conference. (DAC 2021)

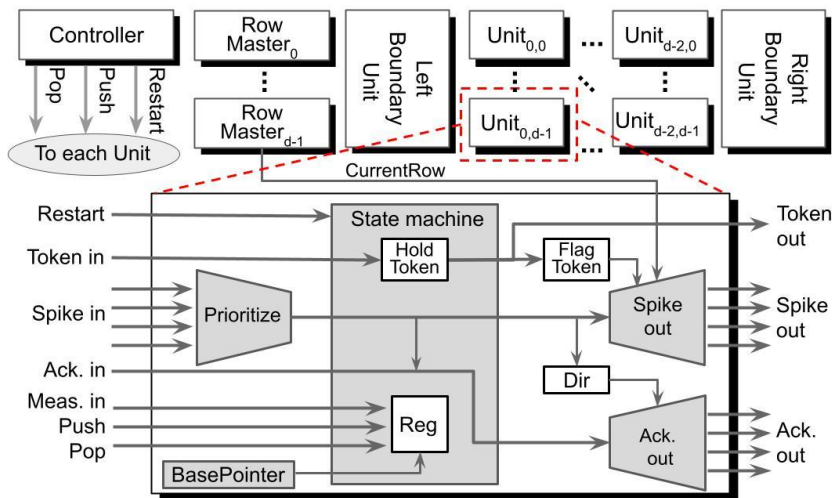
QECoolの誤り推定性能



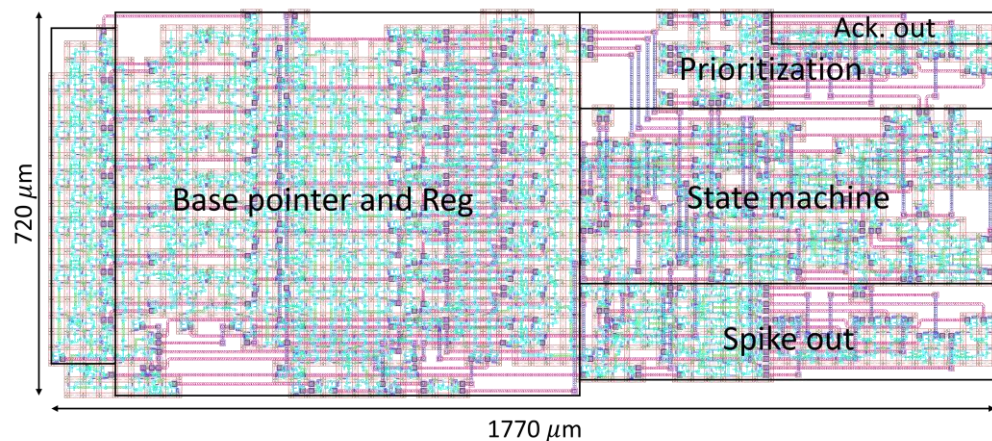
Experimental condition

- Measurement process is performed **once every $1 \mu s$**
- Each QECool Unit has a **7-bit** buffer to store syndrome values
- If buffer entry size is greater than $K = 3$, QECool is performed; otherwise, each Unit waits for measurement process
- MWPM operates with batch-QEC manner
- しきい値: QECool $p = 0.01$, MWPM $p = 0.03$

QECool誤り推定器のSFQ回路による実装



Architecture overview of QECool



SFQ design layout of QECool Unit

JJs: 3177

Area: 1.274 mm²

Latency: 215 ps

Power cons.: **2.78 μW**

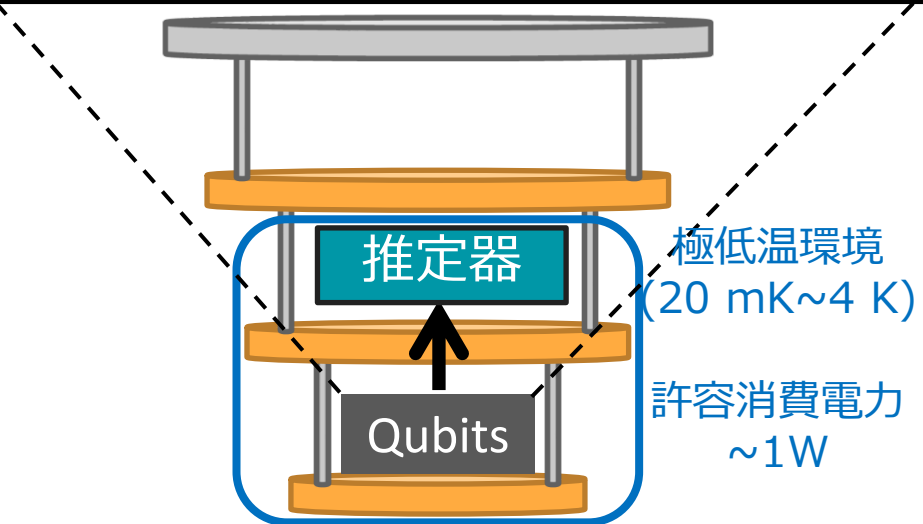
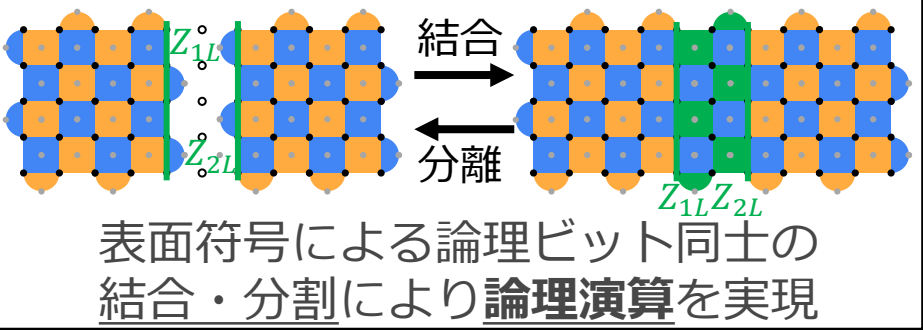
Decoder power consumption per one logical qubits

Suppose $d = 9$,

$$9 \times 8 \times 2 \times 2.78_{[\mu W]} = \boxed{400 \mu W}$$

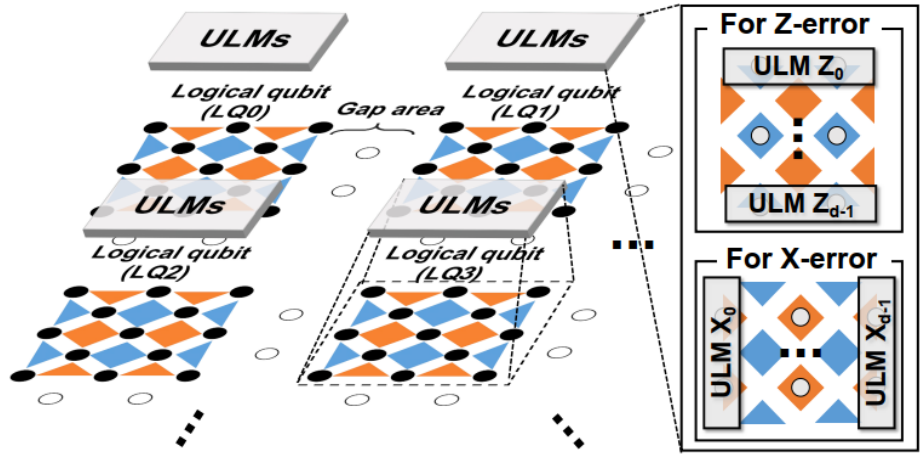
QULATIS: 論理演算を行う場合への拡張

格子手術



提案手法 : QULATIS

- より複雑な形状のグラフマッチング問題を解くSFQ誤り推定アーキテクチャ
- 格子手術の誤り推定を具体的に考えた世界初のアーキテクチャ



Ueno, Kondo, Tanaka, Suzuki, and Tabuchi. QULATIS: A Quantum Error Correction Methodology toward Lattice Surgery. HPCA2022.

QECOOOL/QULATISまとめ

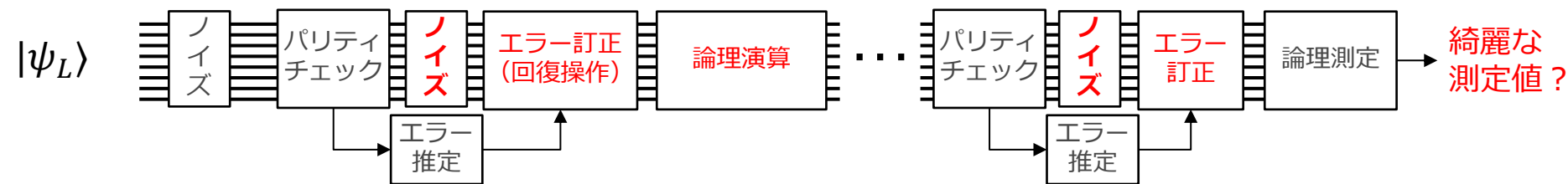
- 極低温環境での誤り推定はスケーラブルな超伝導FTQCの実現のために必須
- SFQ回路で実装したQECOOOL誤り推定器は
レイテンシ制約を満たしつつ極低温環境で動作する
- 1論理ビット ($d = 9$) あたりの誤り推定器の消費電力は $400 \mu\text{W}$
 - 4K環境の許容消費電力を1Wとすると、2500論理ビットの誤り推定を冷凍機内で行える
- QULATIS: 格子手術に基づく論理演算へとQECOOOLを拡張
 - 論理ビットの境界が動的に変わる状況に対処

今日の内容

- 量子誤り訂正の基礎
 - 量子誤り訂正に求められる条件
 - 古典符号と量子符号
 - スタビライザ形式
 - Shorの符号
- 表面符号
 - 表面符号の誤り推定
 - Stim + Crumbleを用いた表面符号シミュレーション
 - 極低温環境での表面符号の誤り推定処理
- 表面符号 + 格子手術に基づく用いた誤り耐性量子計算
- まとめ

表面符号上の論理演算

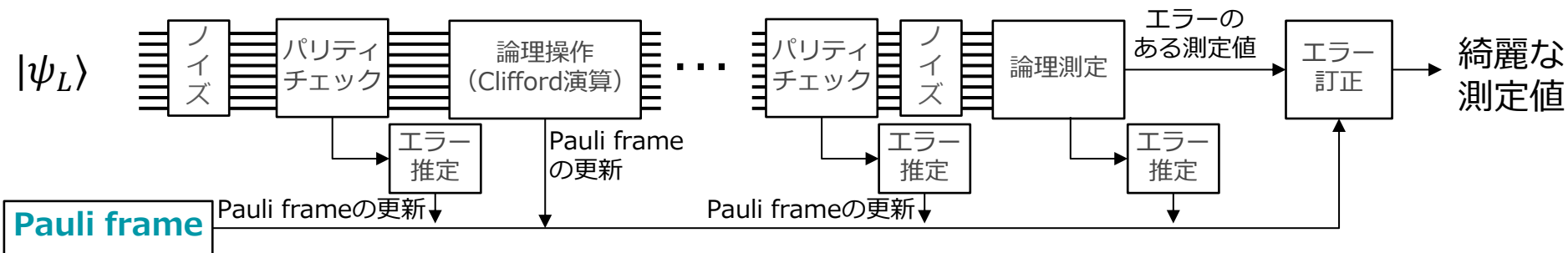
論理演算を行う場合のフローのイメージ（実際はうまくいかない）



- ナイーブな論理演算手順はうまくいかない
 - エラー推定中に新たなノイズが生じる可能性あり
 - エラー訂正操作自体がエラーを発生させる
 - パリティチェックの間隔を一定に保つ必要あり
 - 論理演算にかかる時間が符号の規模に応じて長くなるとまずい
- これを回避するためにエラーがいくつかの論理操作と可換であることを利用

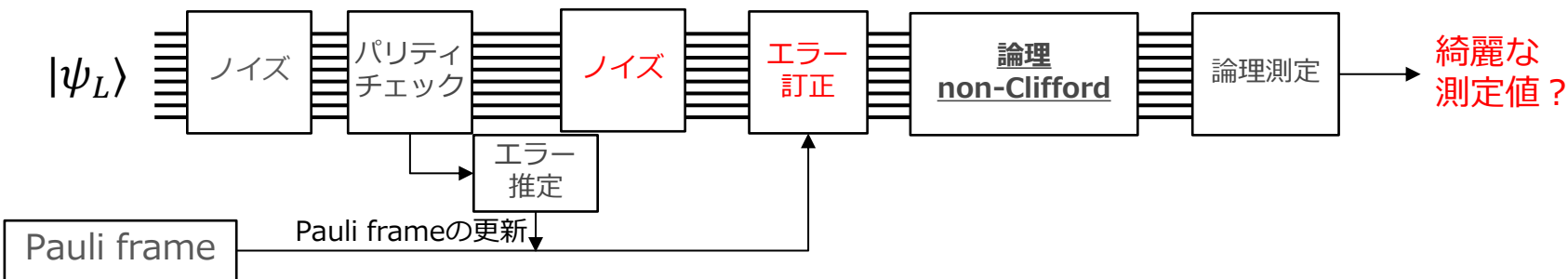
Pauli frameを用いたエラー訂正の後ろ倒し

実際の論理演算のフロー



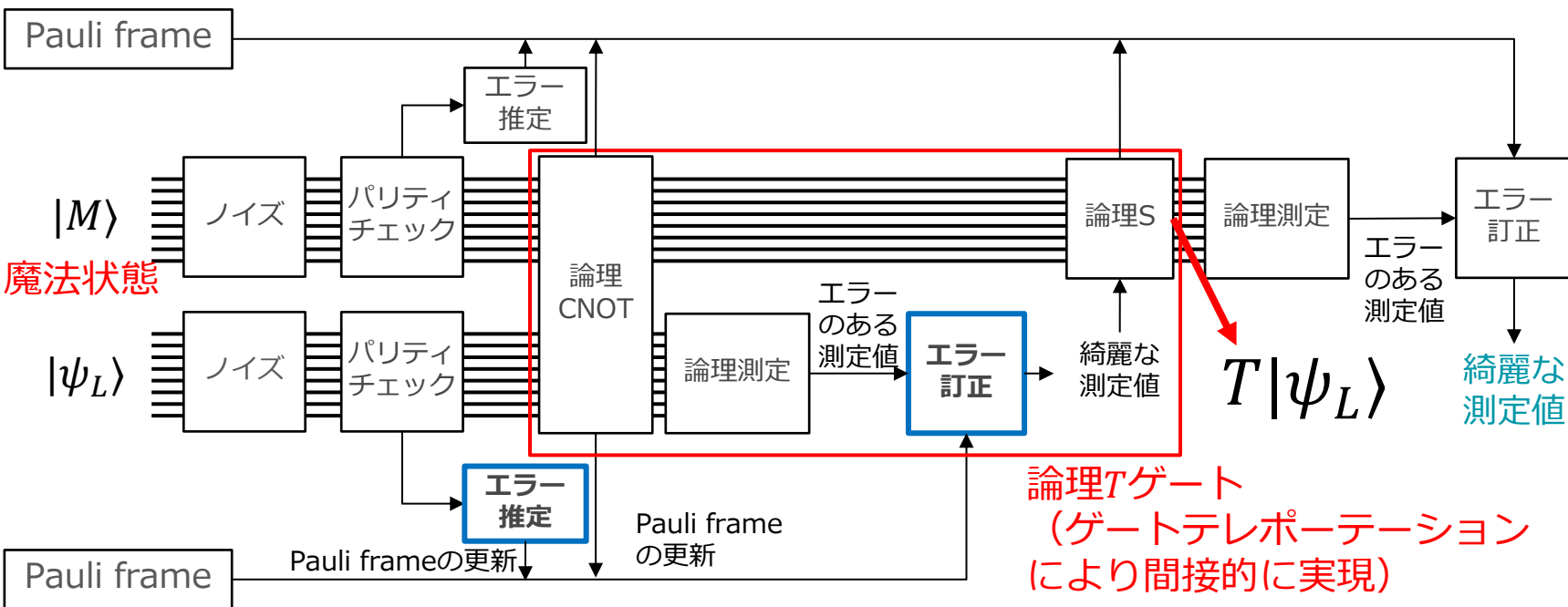
- Clifford演算はエラー訂正と可換であることを利用してエラー訂正操作を後回しにする
- 後で訂正する予定のPauli操作をPauli frameとして保持
- 論理Pauli演算はPauli frameの更新のみで実現

論理non-Clifford演算の難しさ



- 直接論理non-Clifford演算を行おうとするとPauli frameの書き戻しが必須となる
 - Non-Clifford演算は訂正操作と非可換
 - 復号している間に新たなノイズが乗ってしまい完全にエラーを解消できない
- Cliffordな論理操作で間接的にnon-Clifford演算を実現する
ゲートテレポーテーションが必須

ゲートテレポーテーションによる実現

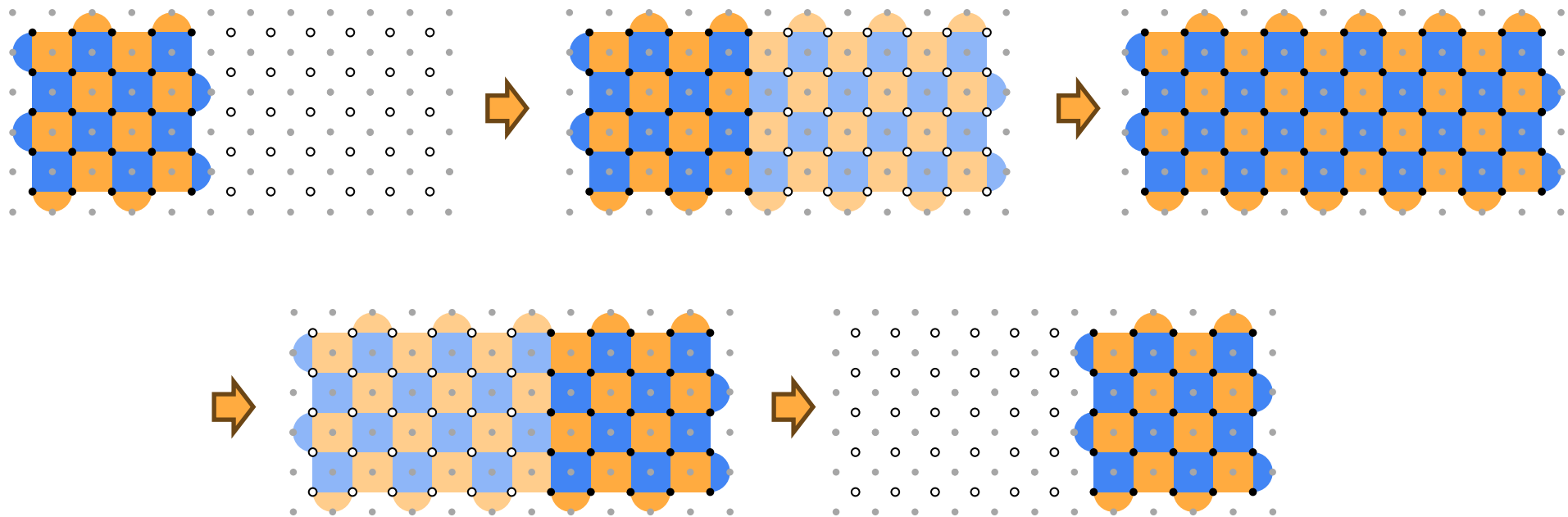


- エラー率 p の魔法状態 $|M\rangle$ は作れる（魔法状態注入）
- エラー率 p の $|M\rangle$ を15個集めてエラー率 $35p^3$ の $|M\rangle$ を1つ作れる（魔法状態蒸留）

表面符号における論理演算

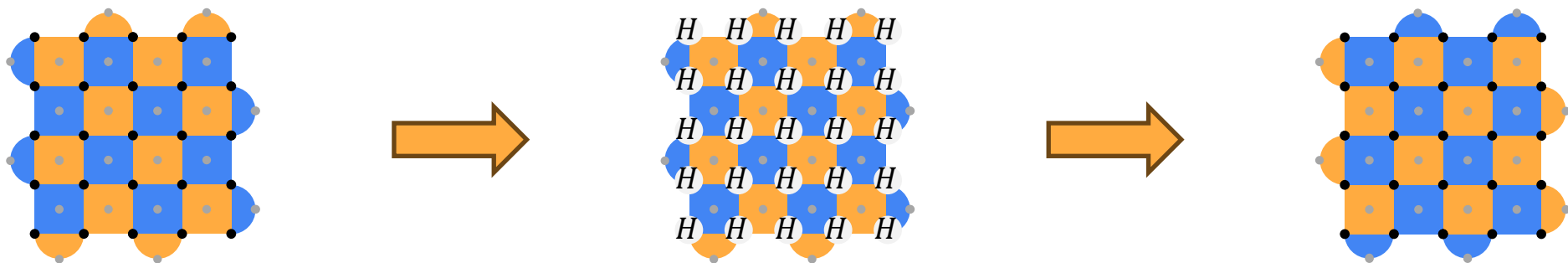
- 表面符号上のユニバーサルゲートセット $\{H, S, CNOT, T\}$
 - H : トランスバーサルな操作で実現
(+必要に応じて符号の変形)
 - S : 符号の変形により実現
 - $CNOT$: **格子手術** (符号変形的一种) による
多体測定 of 組み合わせで実現
 - T : 魔法状態 (特殊な乱数みたいなもの) の準備
+ Clifford操作で実現
 - 特定の状態 ($|0\rangle, |+\rangle, |M\rangle$) への初期化
 - Pauli基底での論理測定
- Clifford演算
- Non-Clifford演算

符号の変形



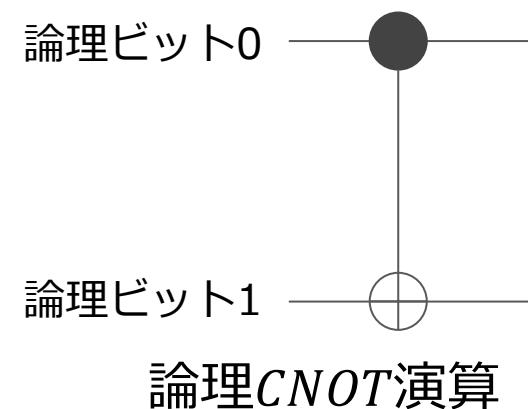
- パリティチェックの組を変えることで符号を変形
 - 論理状態を保ったまま変形 -> 拡大、縮小、**移動**、回転
 - 論理状態を変えながら変形 -> 論理S, CNOTゲート

論理Hゲート

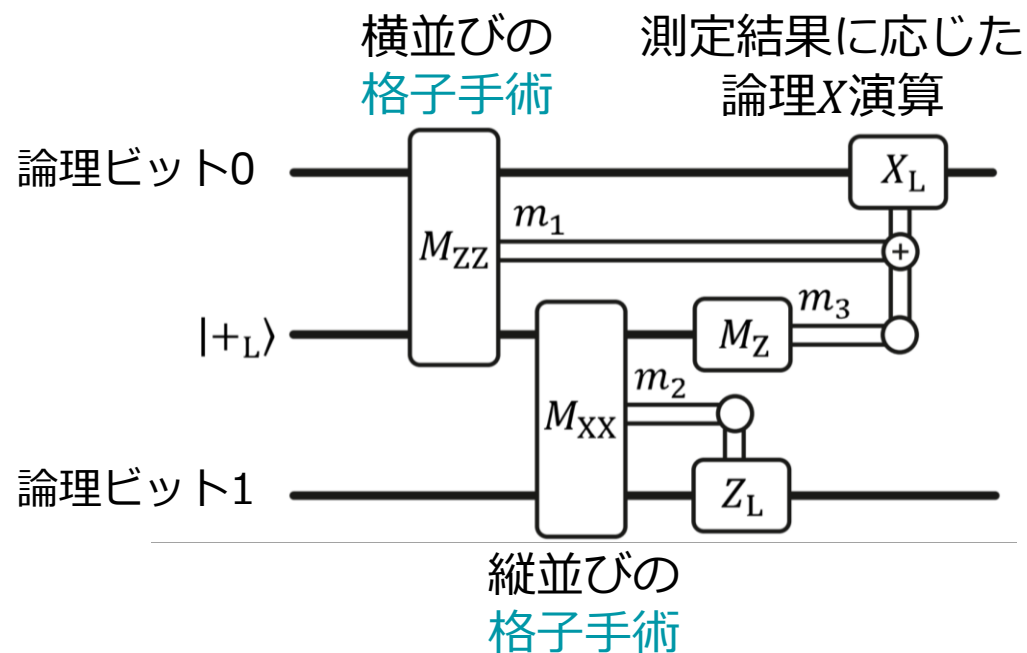


- H はtransversalityがある→全データ量子ビットに H をかける
 - 符号距離によらず論理演算が定数時間で完了
- X と Z のスタビライザが入れ替わるので、左右と上下の境界の意味が交換→必要に応じて回転操作で境界の変化を補正

多体測定を組み合わせたによるCNOTゲート

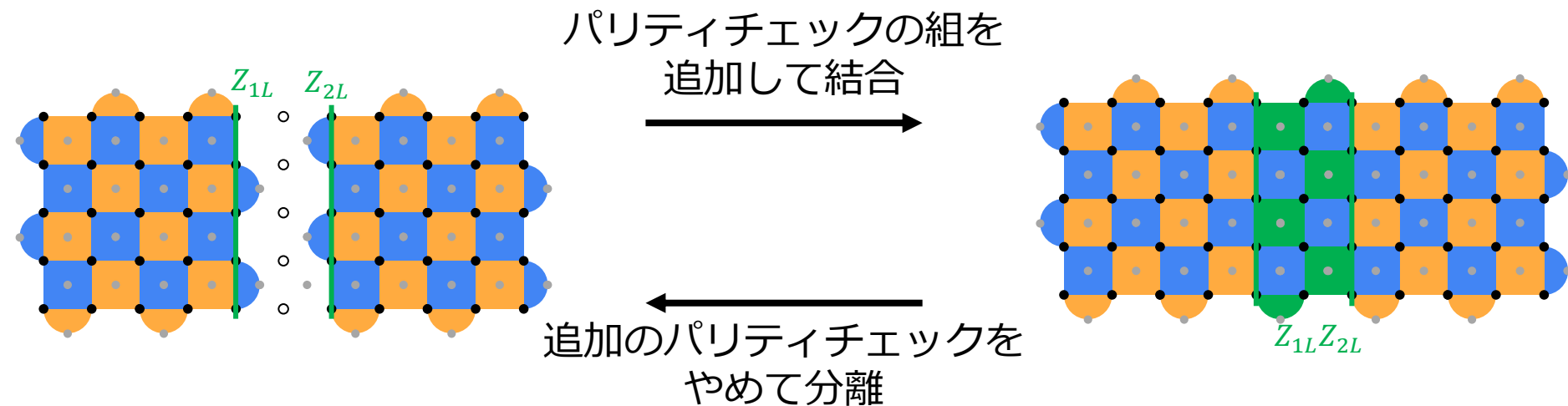


等価な操作



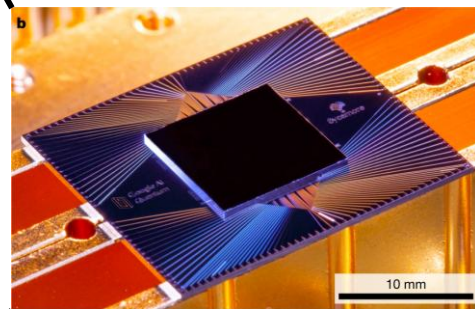
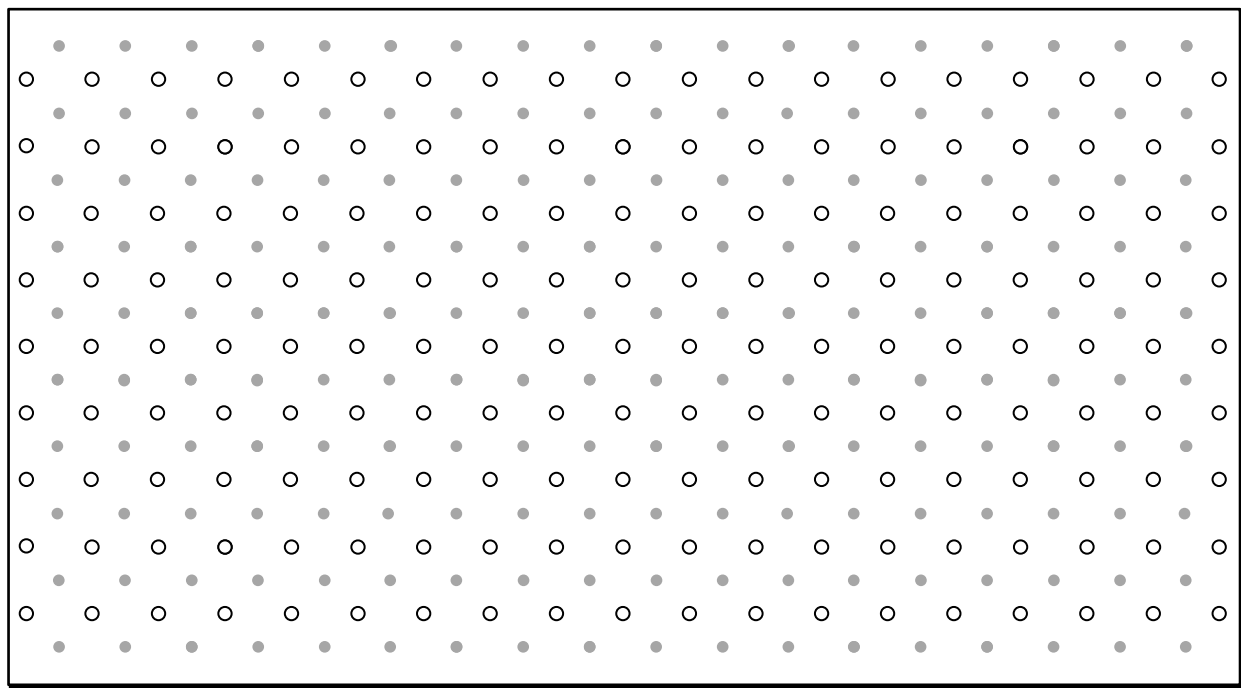
- 格子手術による多体Pauli測定ができれば
論理CNOTを実現できる

格子手術による多体Pauli測定



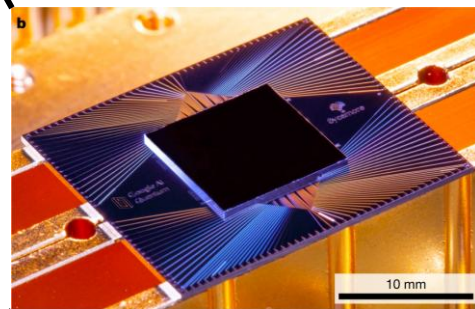
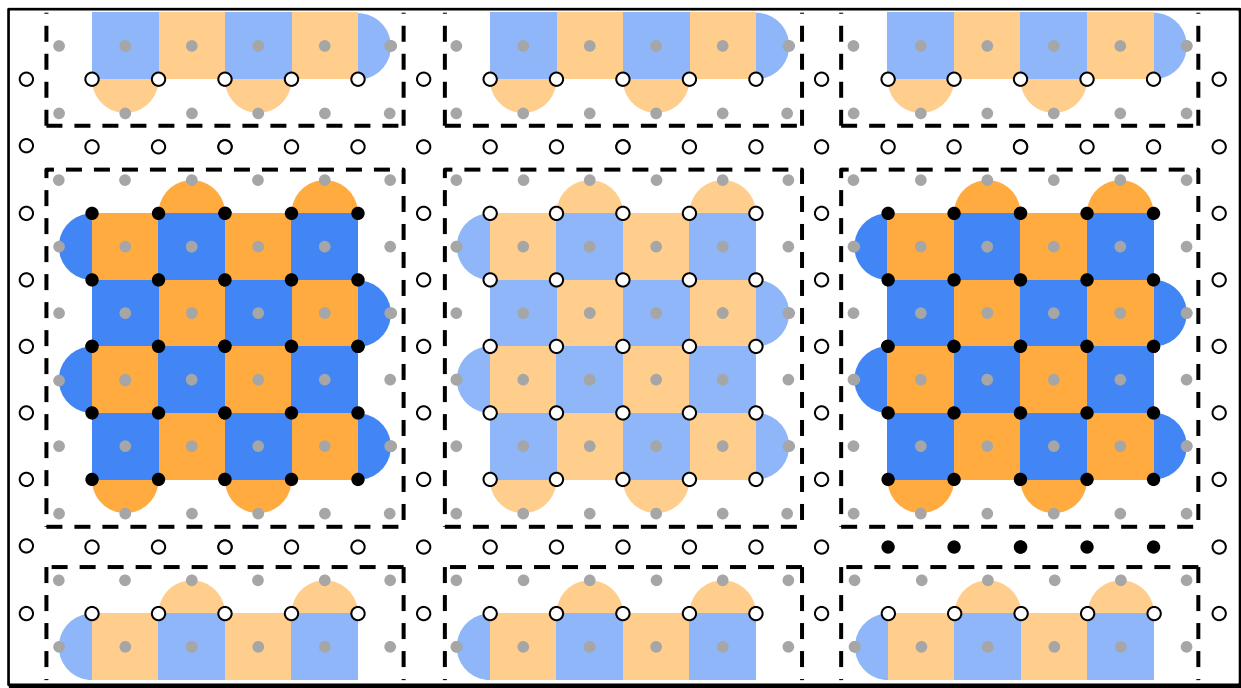
- 論理ビットの間のパリティチェックの結果が2論理ビットのZZ測定になる

実際の量子ビットでの実装



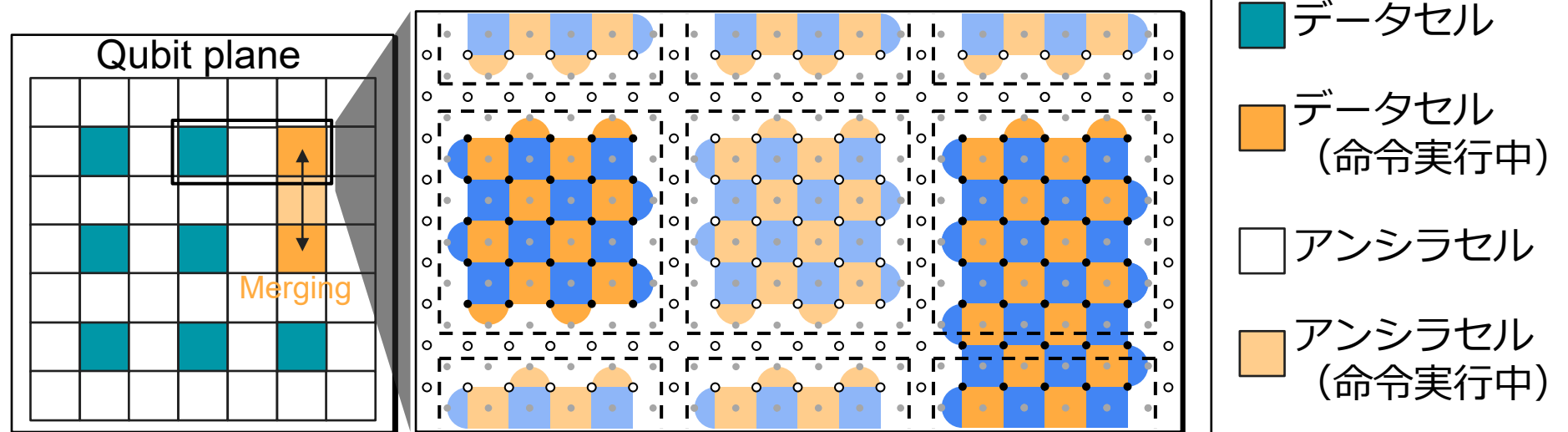
- 表面符号で計算を実行できるように区画を分ける
- 一部に論理状態を確保して一部は計算用に空けておく

実際の量子ビットでの実装



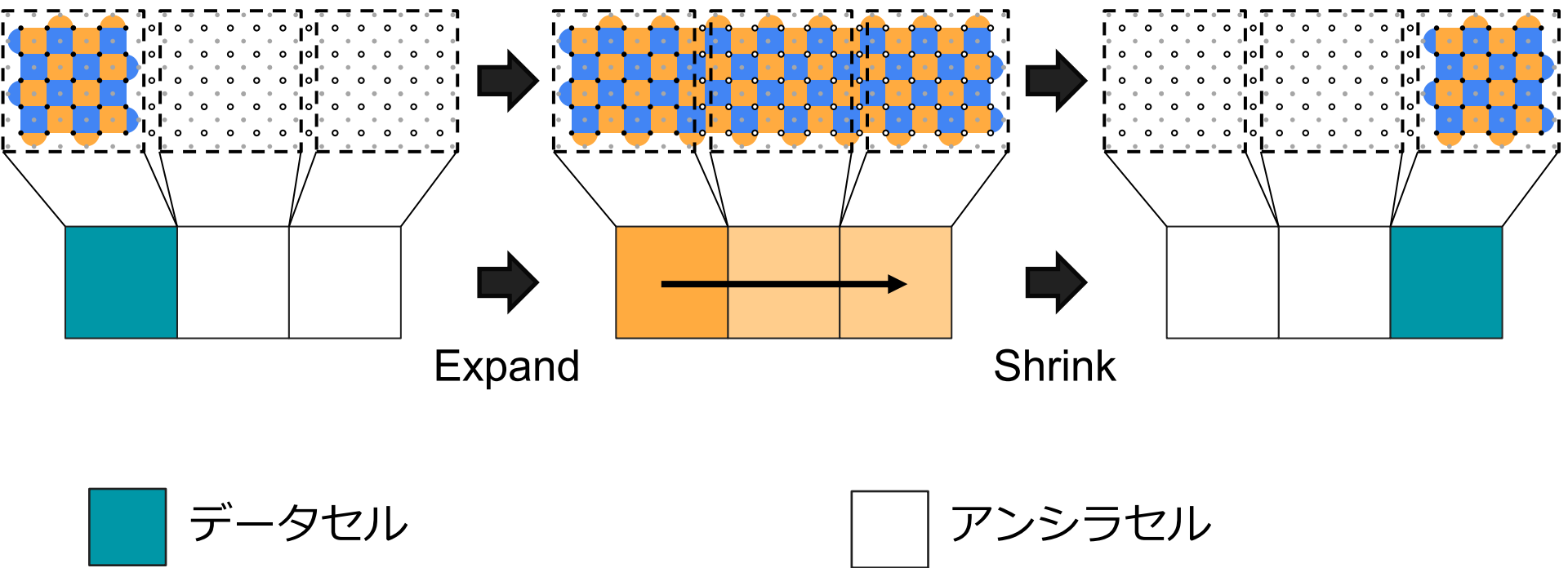
- 表面符号で計算を実行できるように区画を分ける
- 一部に論理状態を確保して一部は計算用に空けておく

表面符号による論理演算を実行するQubit plane

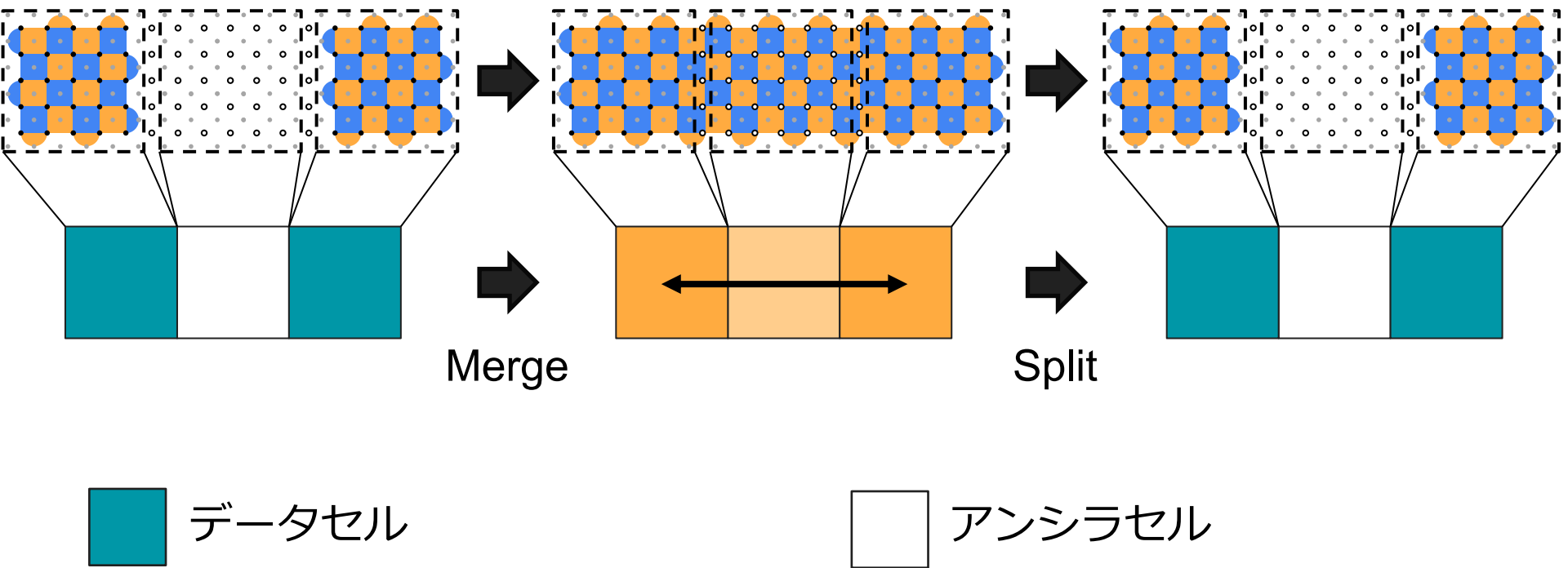


- 1つ1つのセルが符号距離 d の表面符号を構成できる物理量子ビットを持つ
- **データセル**に計算用の論理状態を確保
- 空きスペース (**アンシラセル**) を使って
計算 (= データセルの**拡大・縮小**、**結合・分離**) を行う

例：論理ビットの移動

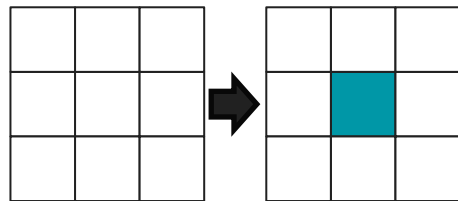


例：格子手術

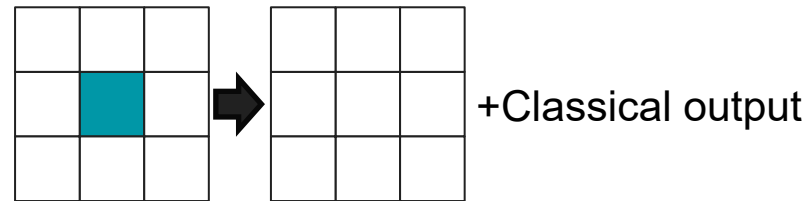


格子手術命令セットの一例

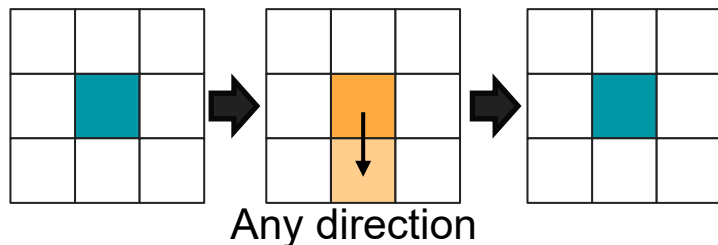
INIT_Z, INIT_X (0 code beat)



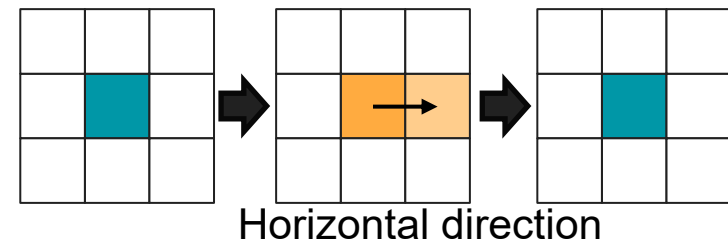
MEAS_X, MEAS_Z (0 code beat)



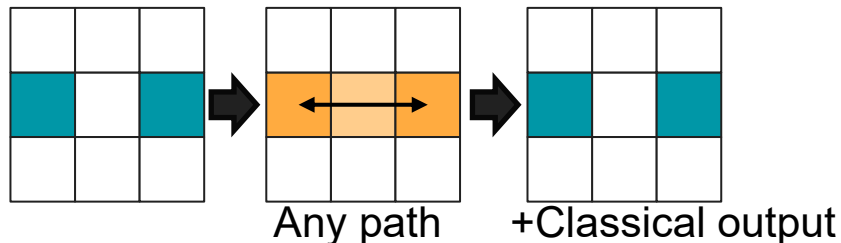
OP_H (3 code beats)



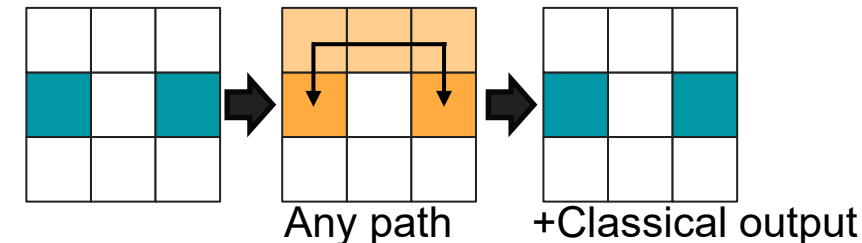
OP_S (2 code beats)



MEAS_ZZ (1 code beat)



MEAS_XX (1 code beat)





LSQCA: Resource-Efficient Load/Store Architecture for Limited-Scale Fault-Tolerant Quantum Computing

Takumi Kobori², Yasunari Suzuki¹, [Yosuke Ueno](#)^{1, 2}
Teruo Tanimoto³, Synge Todo¹, Yuuki Tokunaga⁴

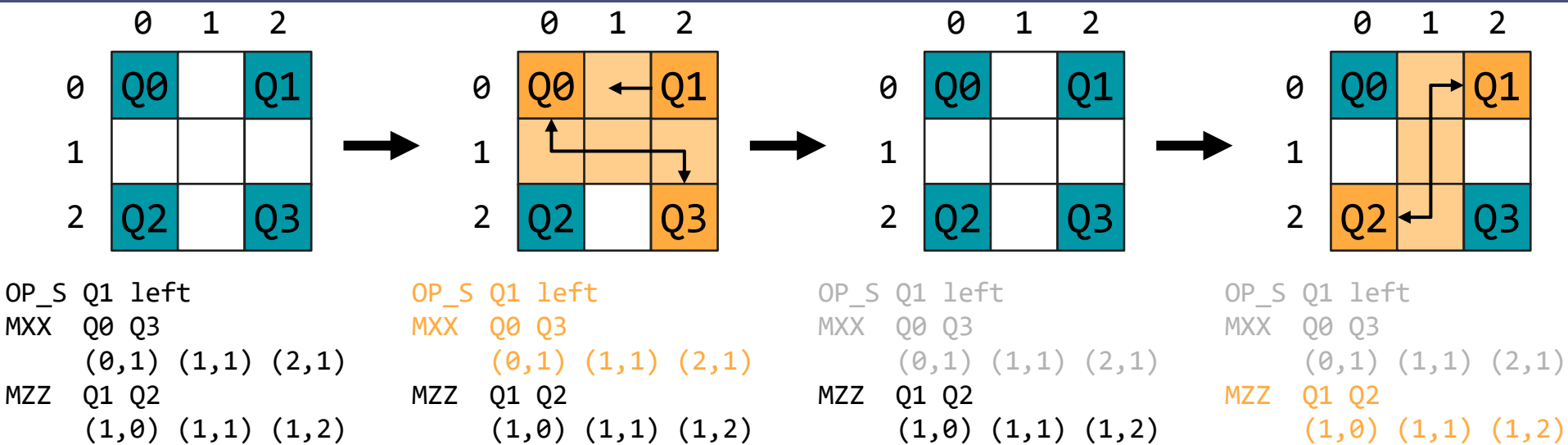
[Physicists](#) and [Computer architects](#)

¹RIKEN, ²The University of Tokyo, ³Kyushu University, ⁴NTT

arXiv 2412.20486

Takumi Kobori, Yasunari Suzuki, [Yosuke Ueno](#), Teruo Tanimoto, Synge Todo, Yuuki Tokunaga, LSQCA: Resource-Efficient Load/Store Architecture for Limited-Scale Fault-Tolerant Quantum Computing, IEEE International Symposium on High-Performance Computer Architecture (HPCA) 2025, pp. 304-320.

格子手術のコンパイルと問題点

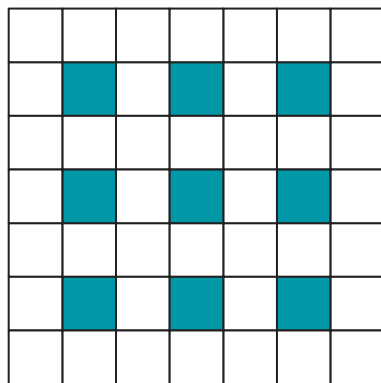


従来の格子手術コンパイルの問題点1

- 各命令がハードウェア情報に依存しすぎておりポータビリティが低い
 - チップの変形やデータ位置の変更によりコンパイルがやり直しになる

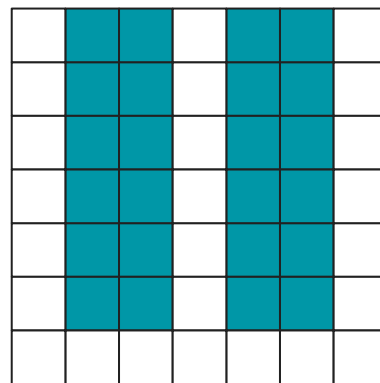
データセル配置と実行時間

- データセル配置密度と実行時間の間のトレードオフがある



Low: $1/4$

データセル密度



High: $2/3$

データセル配置と実行時間

- データセル配置密度と実行時間の間のトレードオフがある

	Z		H		Z	
	X		Z		Z	
	Z		X		Z	

Low: $1/4$

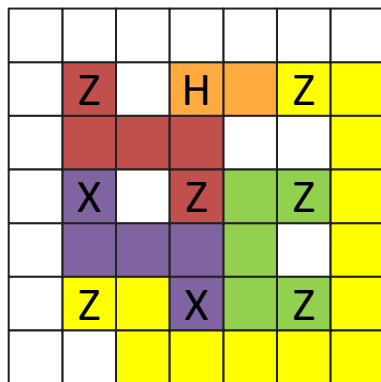
データセル密度

High: $2/3$

	Z	H			Z	
		Z				
		Z		Z		
	X			Z	X	

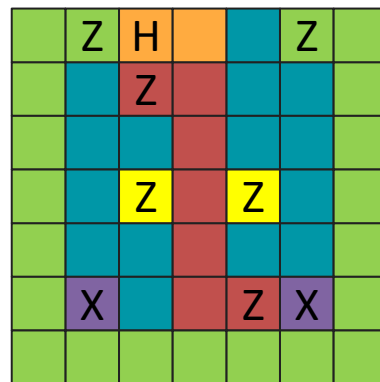
データセル配置と実行時間

- データセル配置密度と実行時間の間のトレードオフがある



Low: $1/4$

Short

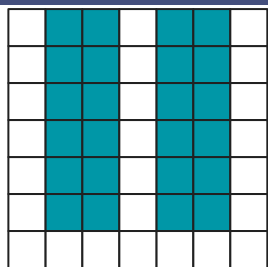


High: $2/3$

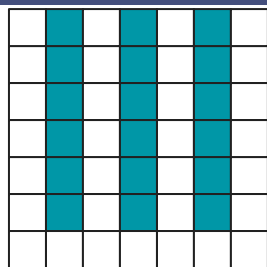
Long

データセル密度
実行時間

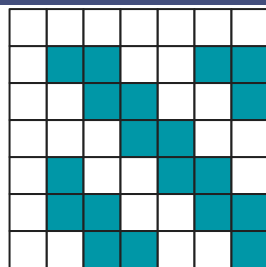
従来のデータセル配置



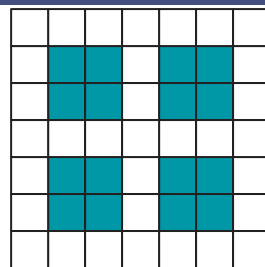
2/3 [1]



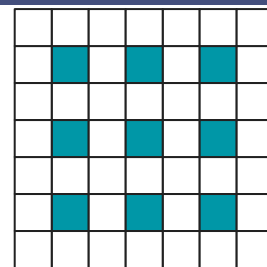
1/2 [2]



1/2 [3]



4/9 [4]



1/4 [5]

データセル
配置密度

任意のデータセルがアンシラセルに隣接しており、
常に任意のデータセルに対する操作が可能



従来の格子手術コンパイルの問題点2

- 空間の利用効率が低い
 - 領域の多くが演算用のアンシラセルで、利用されない時間が多い

[1] J. Lee et al., PRX Quantum 2, 030305 (2021). [2] M. Beverland et al., arXiv:2211.07629 (2022). [3] Y. Ueno et al., arXiv:2411.17519 (2024).

[4] C. Chamberland and E. T. Campbell, PRX Quantum 3, 010331 (2022). [5] M. Beverland et al., PRX Quantum 3, 020342 (2022).

従来のデータセル配置



「すべてのデータセルにいつでもアクセスできる」ことは
必須なのか？

2/3 [1]

1/2 [2]

1/2 [3]

4/9 [4]

1/4 [5]

データセル
配置密度

任意のデータセルがアンシラセルに隣接しており、
常に任意のデータセルに対する操作が可能



従来の格子手術コンパイルの問題点2

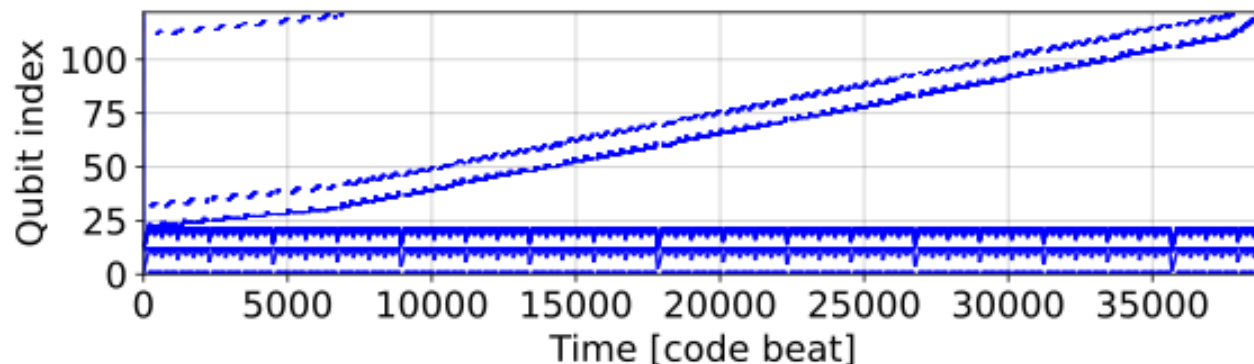
- 空間の利用効率が低い
 - 領域の多くが演算用のアンシラセルで、利用されない時間が多い

[1] J. Lee et al., PRX Quantum 2, 030305 (2021). [2] M. Beverland et al., arXiv:2211.07629 (2022). [3] Y. Ueno et al., arXiv:2411.17519 (2024).

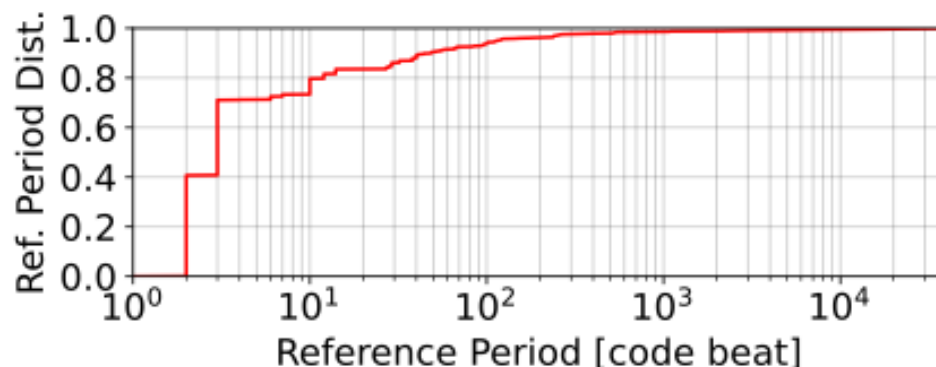
[4] C. Chamberland and E. T. Campbell, PRX Quantum 3, 010331 (2022). [5] M. Beverland et al., PRX Quantum 3, 020342 (2022).

QPEにおけるデータセルへのアクセスパターン

Reference time stamp for SELECT subroutine of quantum phase estimation

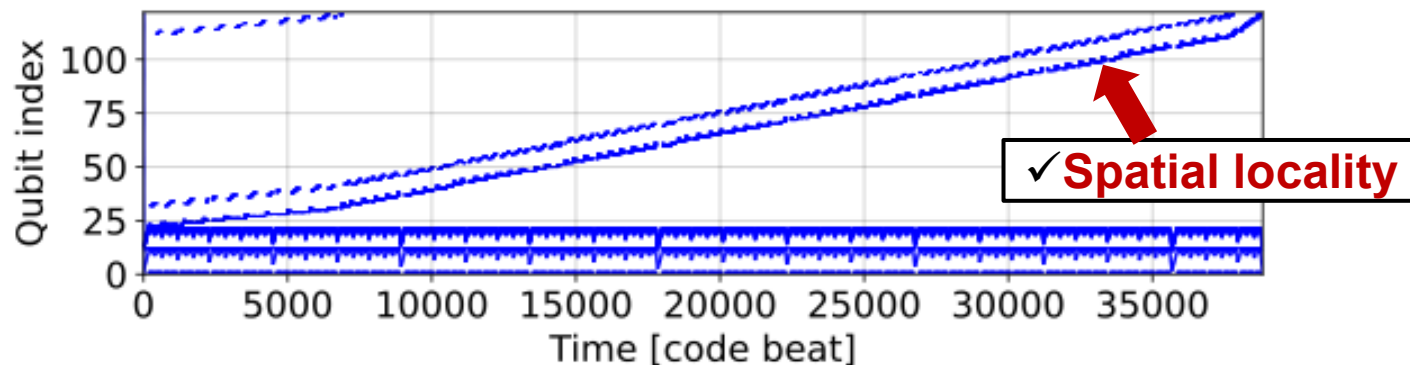


Cumulative distribution of average reference period for SELECT circuit

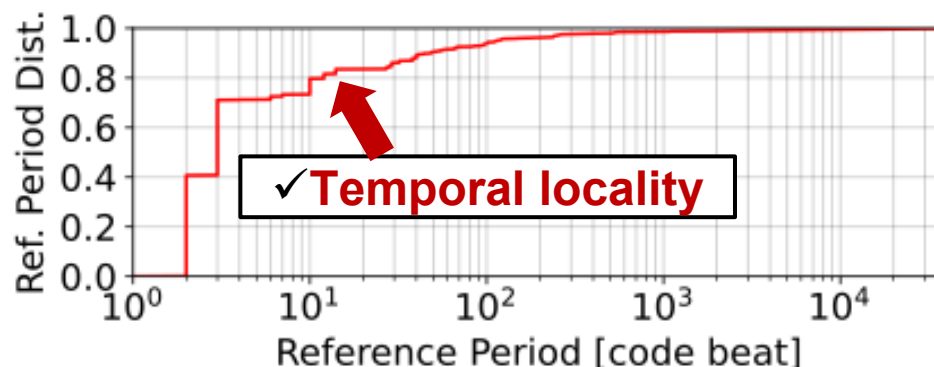


QPEにおけるデータセルへのアクセスパターン

Reference time stamp for SELECT subroutine of quantum phase estimation

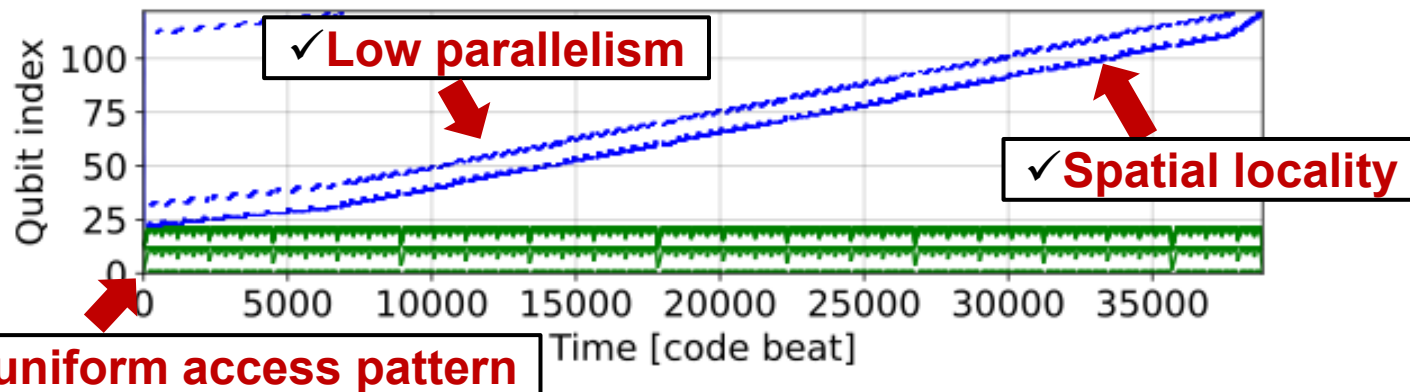


Cumulative distribution of average reference period for SELECT circuit

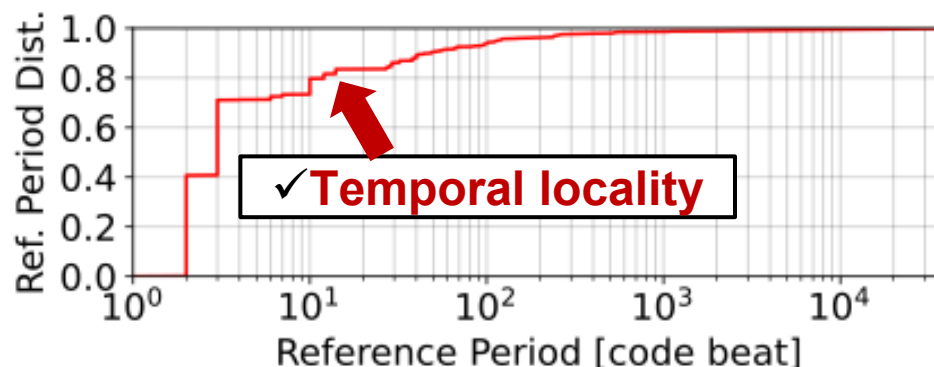


QPEにおけるデータセルへのアクセスパターン

Reference time stamp for SELECT subroutine of quantum phase estimation

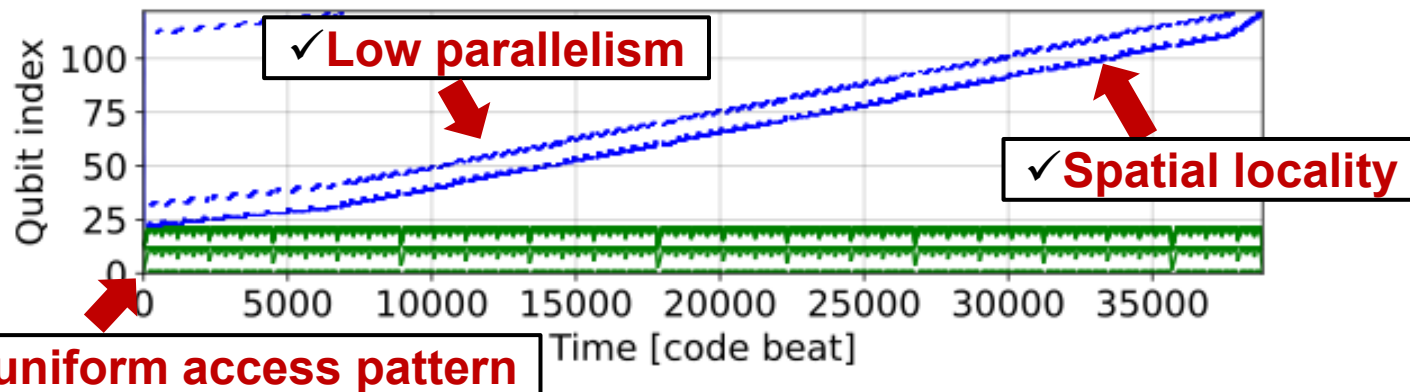


Cumulative distribution of average reference period for SELECT circuit

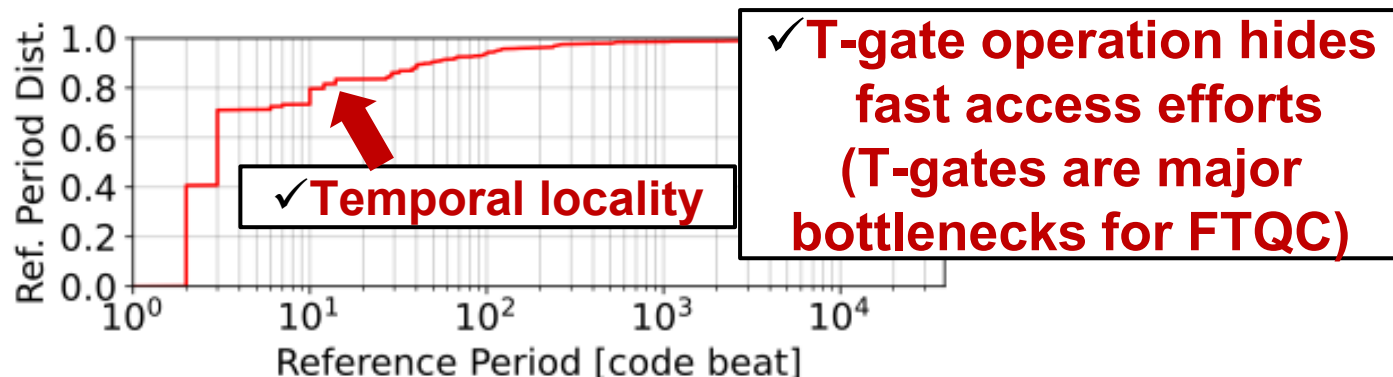


QPEにおけるデータセルへのアクセスパターン

Reference time stamp for SELECT subroutine of quantum phase estimation

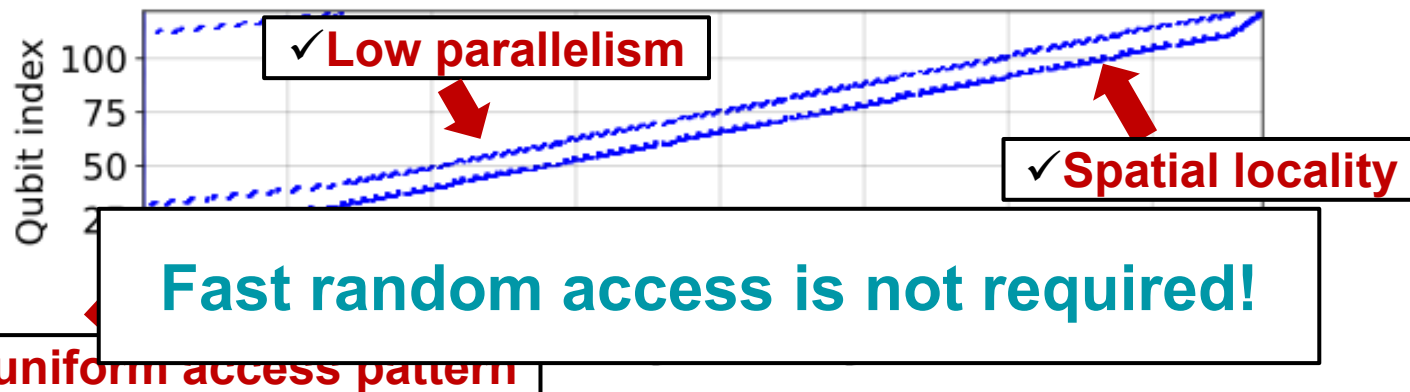


Cumulative distribution of average reference period for SELECT circuit

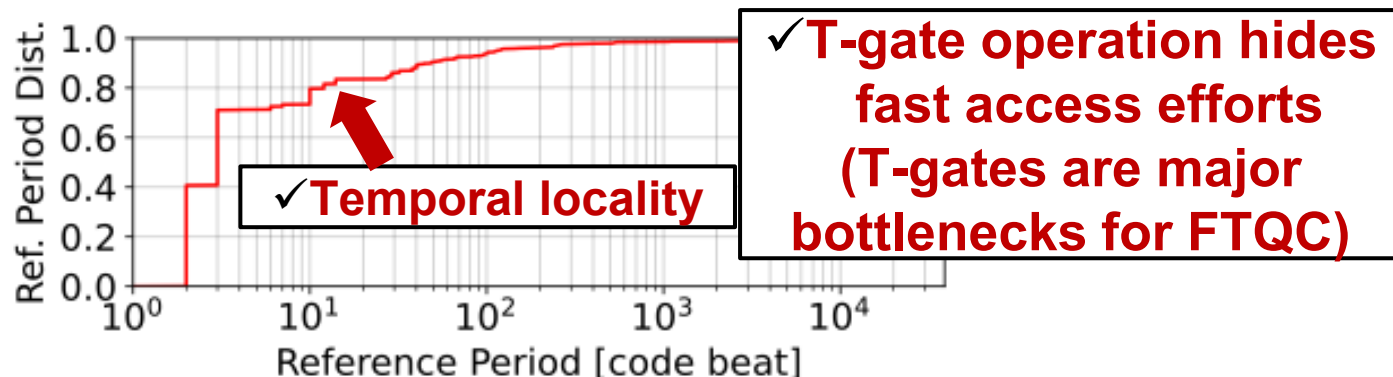


QPEにおけるデータセルへのアクセスパターン

Reference time stamp for SELECT subroutine of quantum phase estimation



Cumulative distribution of average reference period for SELECT circuit



QPEにおけるデータセルへのアクセスパターン

Reference time stamp for SELECT subroutine of quantum phase estimation

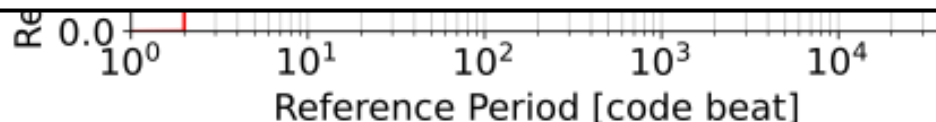


Our goal

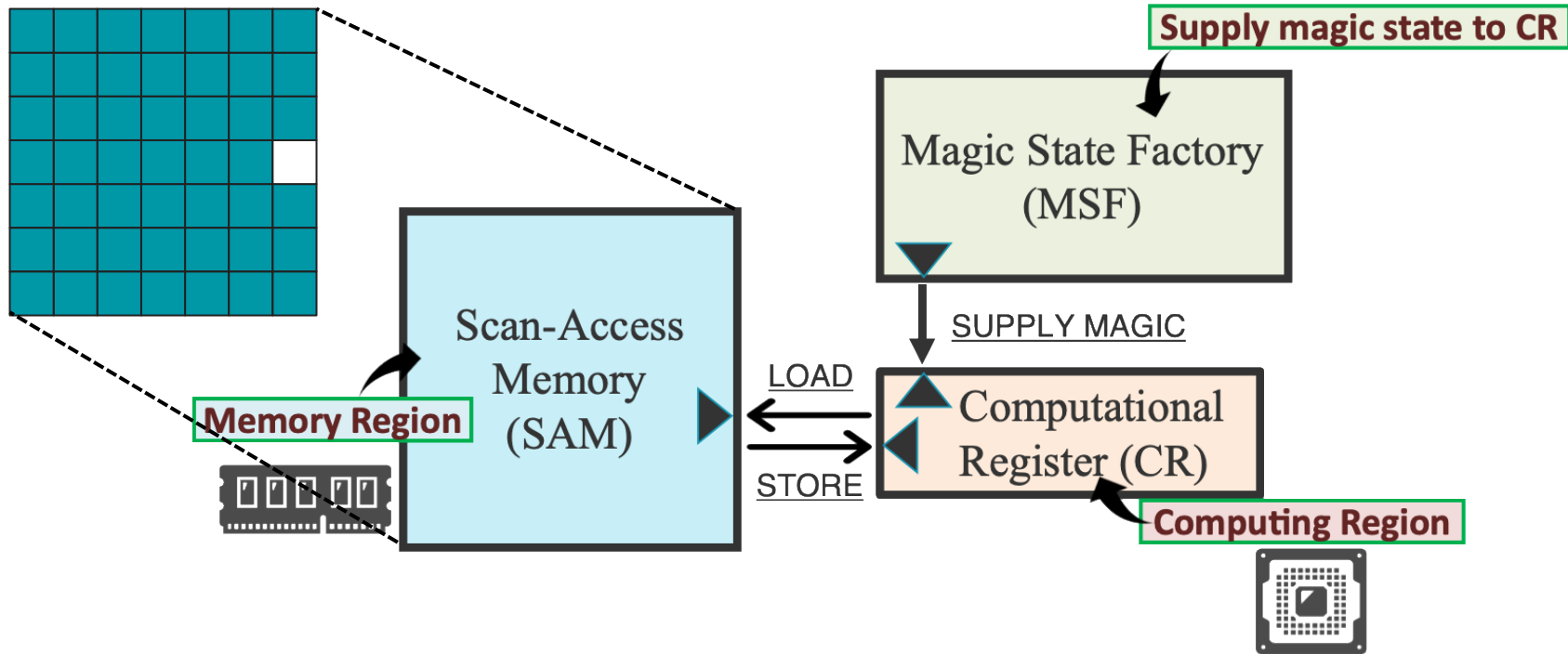
Higher memory density with a small time overhead

Our approach

- ✓ Divide qubit plane into register and memory region
- ✓ Utilize access locality and optimize for biased access pattern
- ✓ Conceal the overhead by T-gate operations and bottleneck operation



ロードストア型誤り耐性量子計算機アーキテクチャ



- 高密度にデータセルを配置するメモリ領域を構成
- データセルの移動を活用して計算領域に読み出し

ISA of our load/store architecture for FTQC

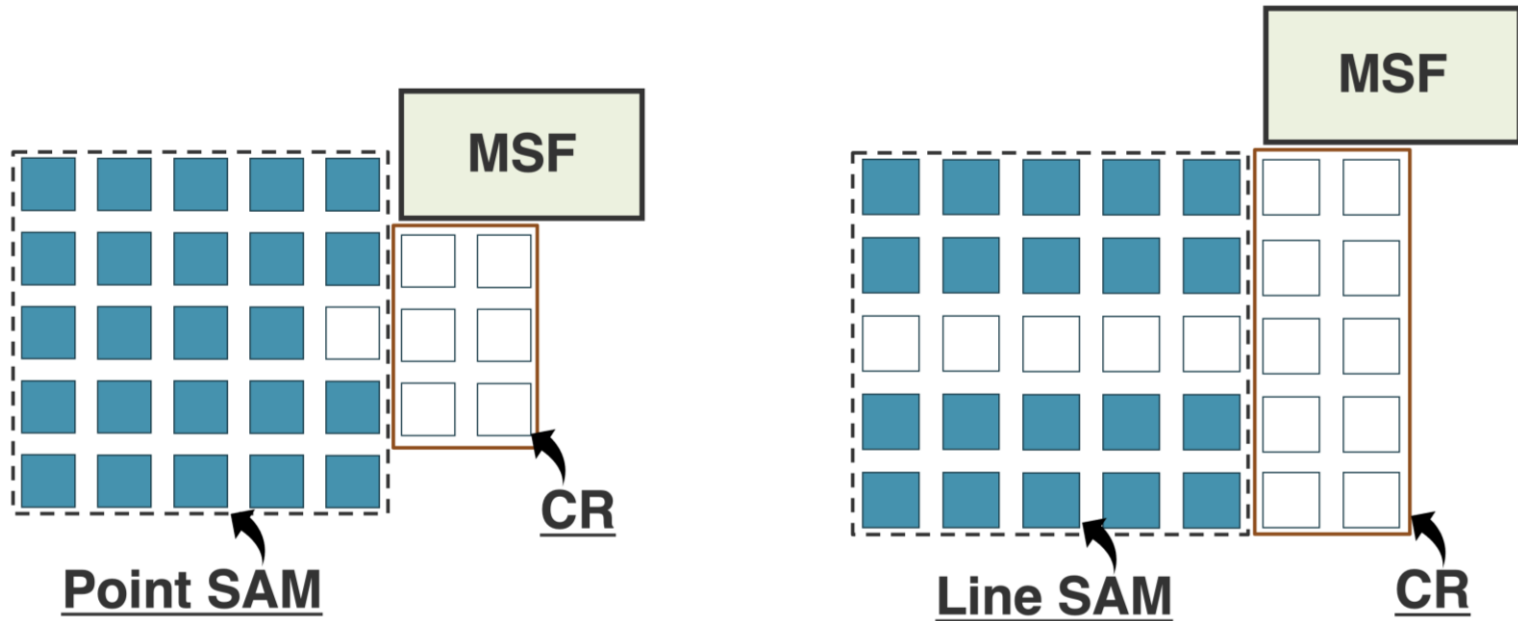
Load/
Store

Type	Syntax	Latency	Description
Memory	LD M C	variable	(Load) Load logical qubit from SAM to CR
	ST C M	variable	(Store) Store logical qubit from CR to SAM
Preparation	PZ.C C	0 beat	(Zero-Init) Initialize a logical qubit to $ 0\rangle$ state
	PP.C C	0 beat	(Plus-Init) Initialize a logical qubit to $ +\rangle$ state
	PM C	variable	(Magic-init) Move magic state from MSF to CR
Unitary	HD.C C	3 beat	(Hadamard) Hadamard gate on a logical qubit
	PH.C C	2 beat	(Phase) Phase gate on a logical qubit
Measurement	MX.C C V	0 beat	(Pauli-X Meas) Pauli-X measurement on a logical qubit and store outcome
	MZ.C C V	0 beat	(Pauli-Z Meas) Pauli-Z measurement on a logical qubit and store outcome
	MXX.C C1 C2 V	1 beat	(Pauli-XX Meas) Pauli-XX measurement on logical qubits and store outcome
	MZZ.C C1 C2 V	1 beat	(Pauli-ZZ Meas) Pauli-ZZ measurement on logical qubits and store outcome
Control	SK V	variable	(Skip) Skip next instruction if a provided value is zero
In-Memory Preparation	PZ.M M	0 beat	(Zero-Init) Initialize a logical qubit to $ 0\rangle$ state
	PP.M M	0 beat	(Plus-Init) Initialize a logical qubit to $ +\rangle$ state
In-Memory Unitary	HD.M M	variable	(Hadamard) Hadamard gate on a logical qubit
	PH.M M	variable	(Phase) Phase gate on a logical qubit
In-Memory Measurement	MX.M M V	0 beat	(Pauli-X Meas) Pauli-X measurement on a logical qubit and store outcome
	MZ.M M V	0 beat	(Pauli-Z Meas) Pauli-Z measurement on a logical qubit and store outcome
	MXX.M C M V	variable	(Pauli-XX Meas) Pauli-XX measurement on logical qubits and store outcome
	MZZ.M C M V	variable	(Pauli-ZZ Meas) Pauli-ZZ measurement on logical qubits and store outcome
Optimized Unitary	CX M1 M2	variable	(CNOT) CNOT gate on logical qubits

- ✓ Load/Store instructions are independent of memory region implementation (e.g., QEC code, allocation)
- ✓ Enable of abstraction, easily programable, and high program portability

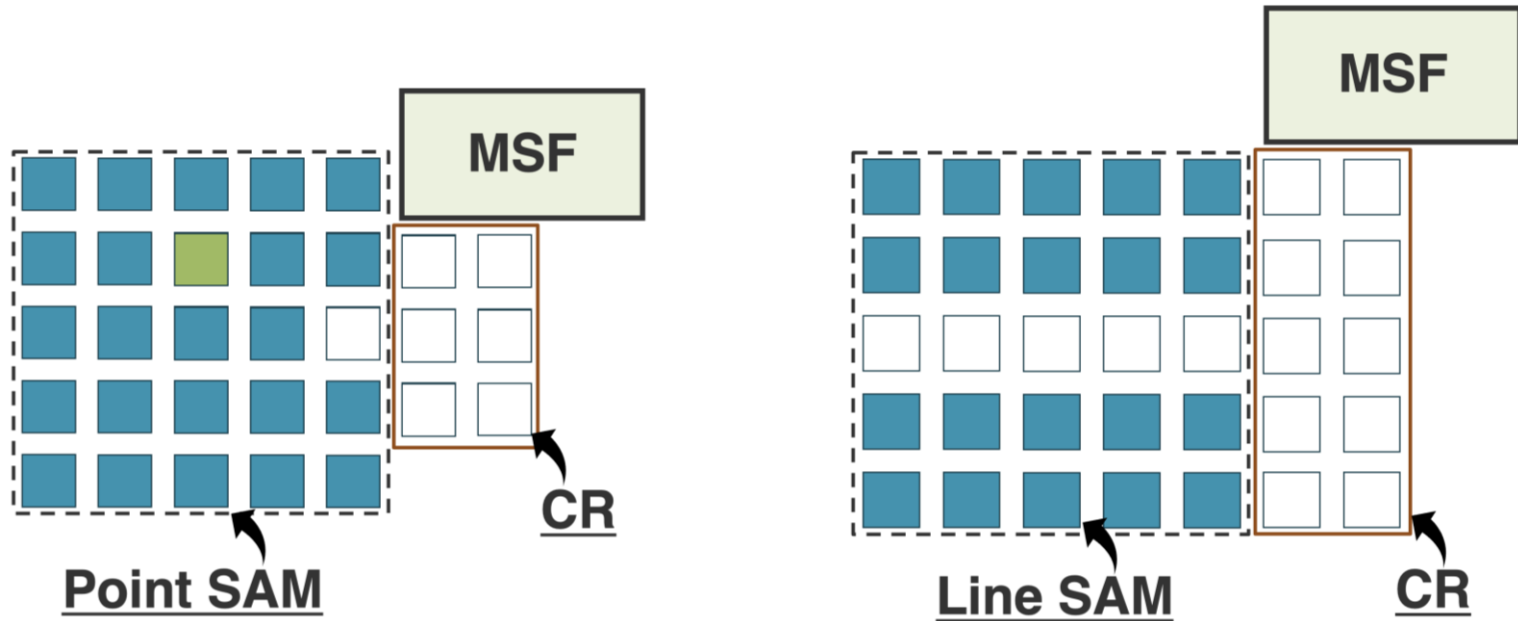
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



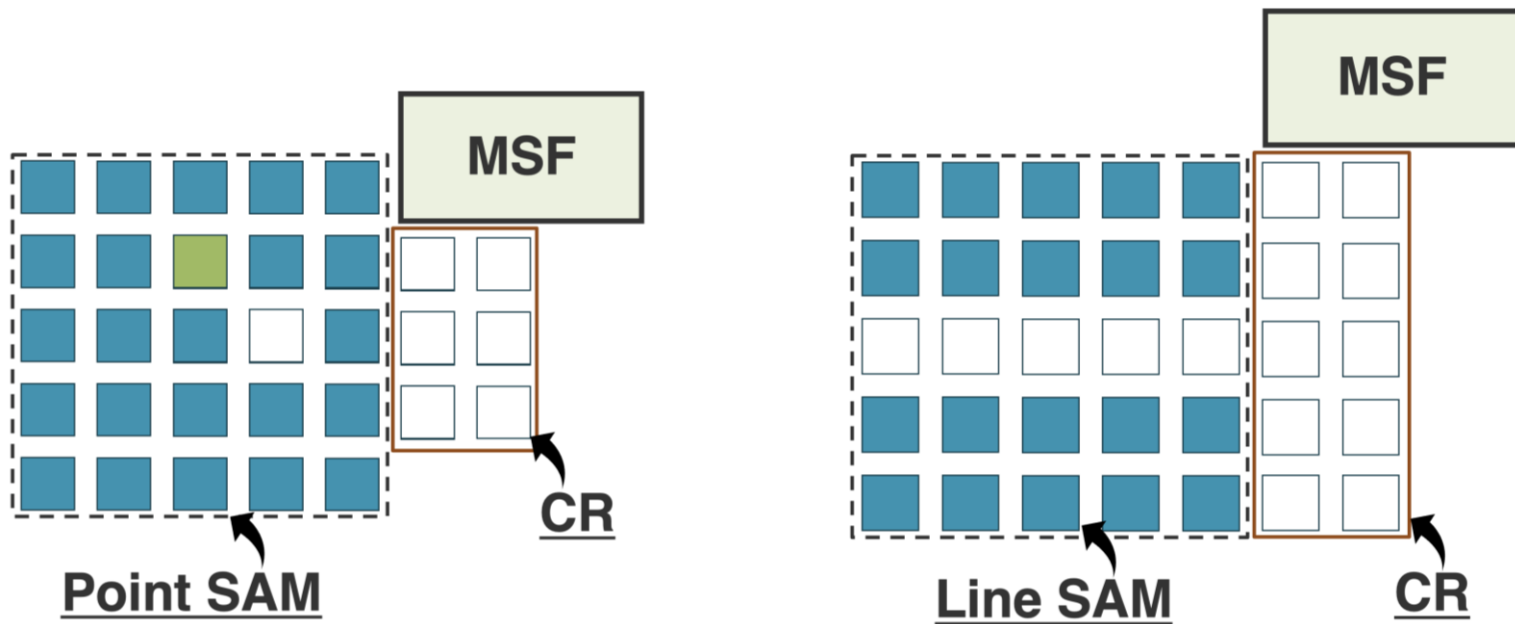
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



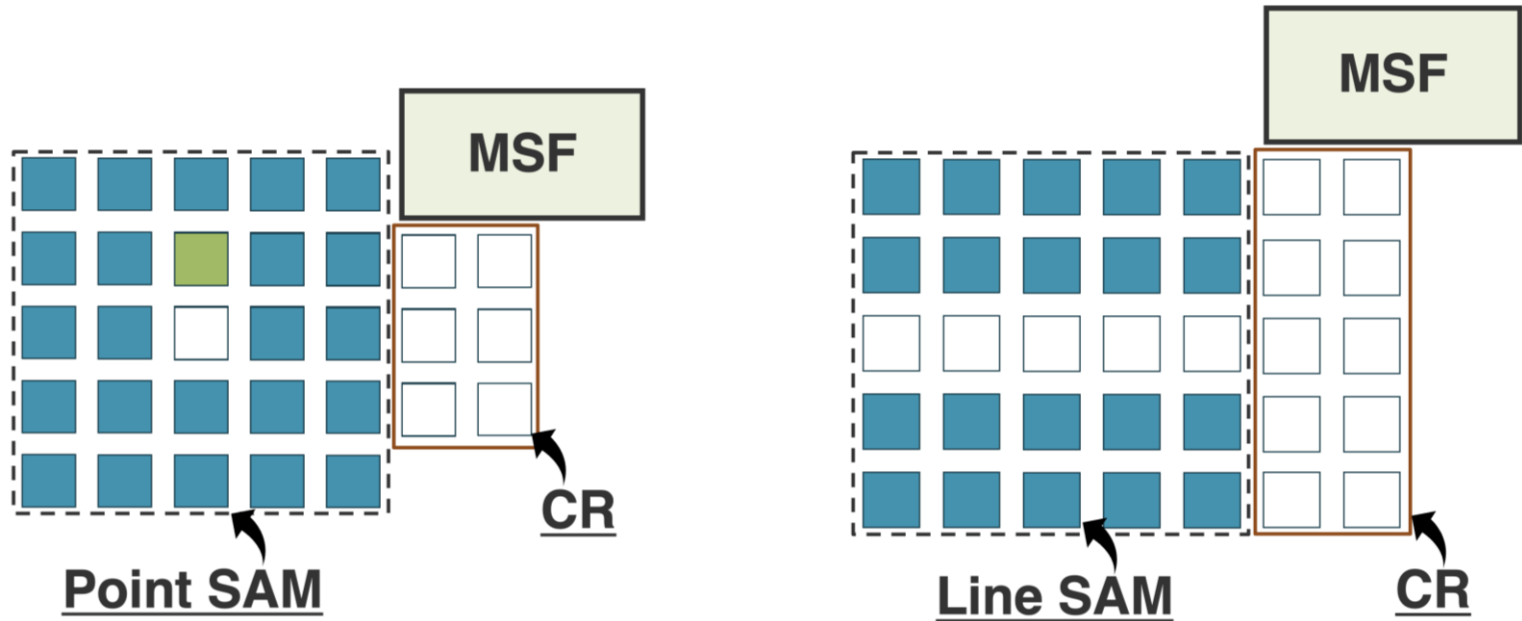
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



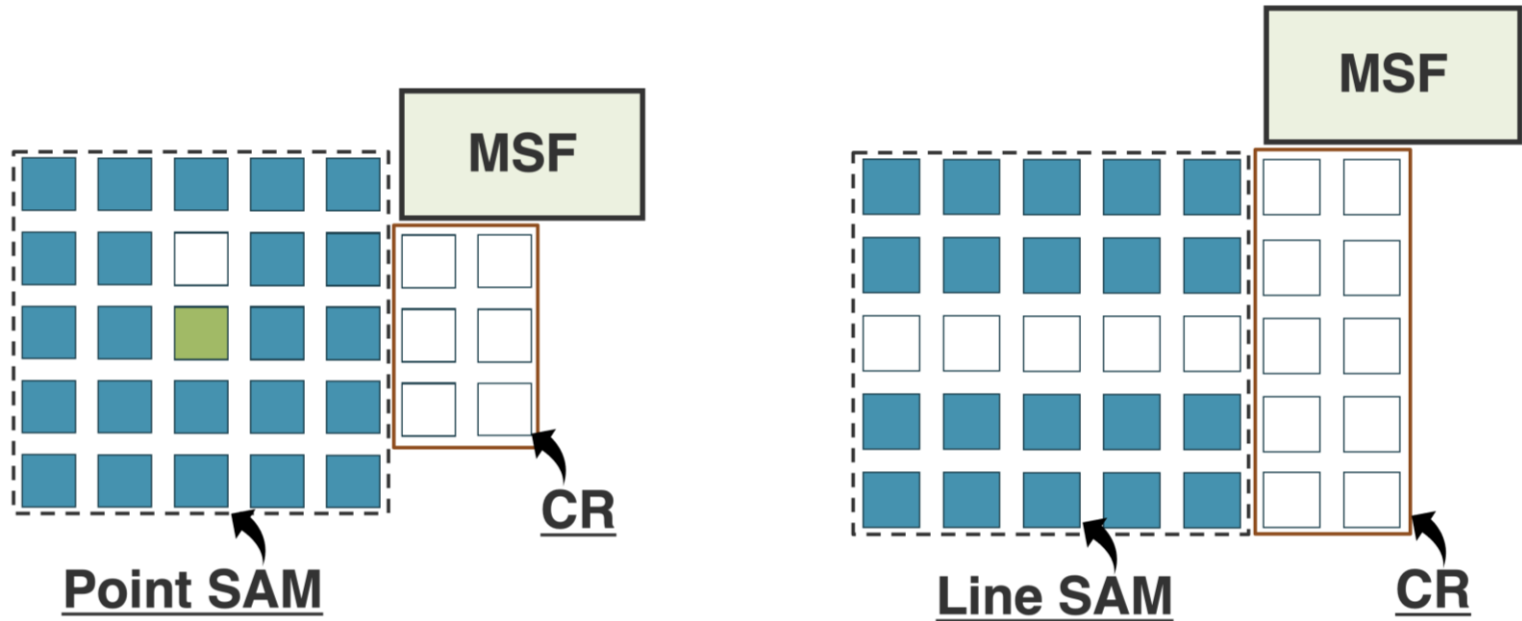
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



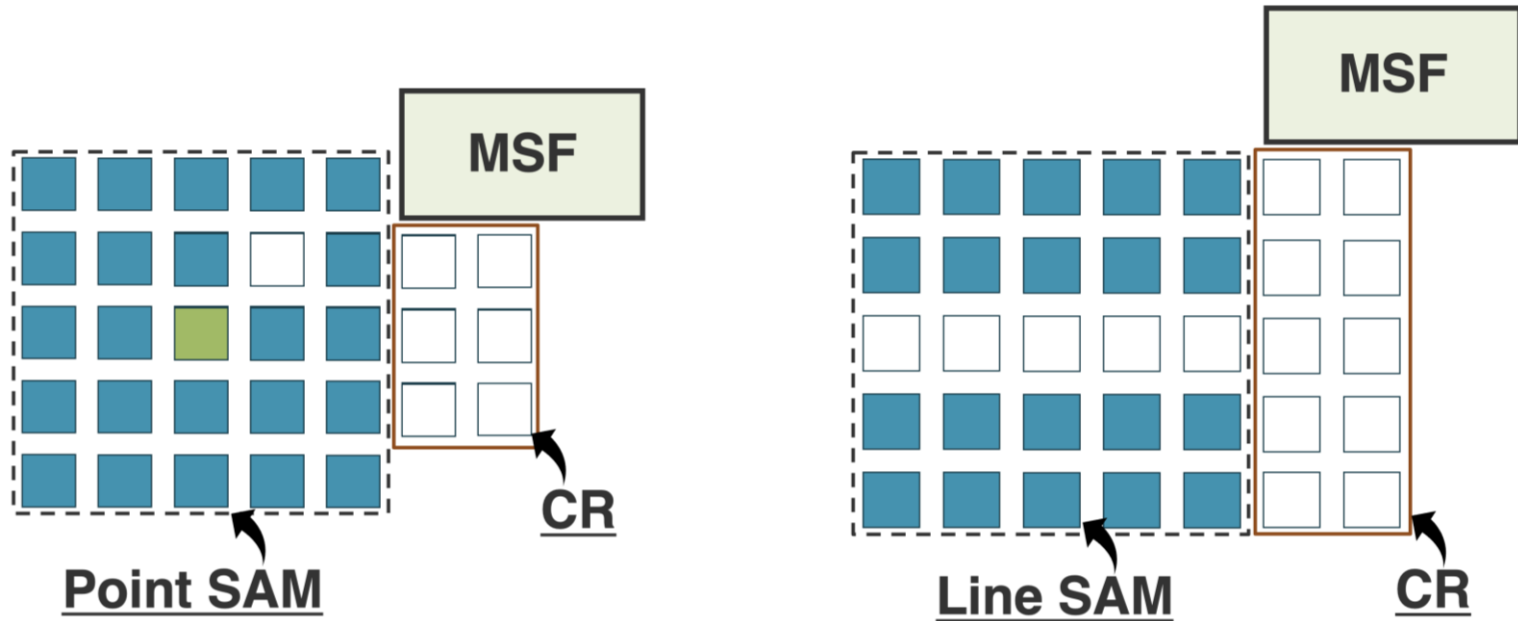
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



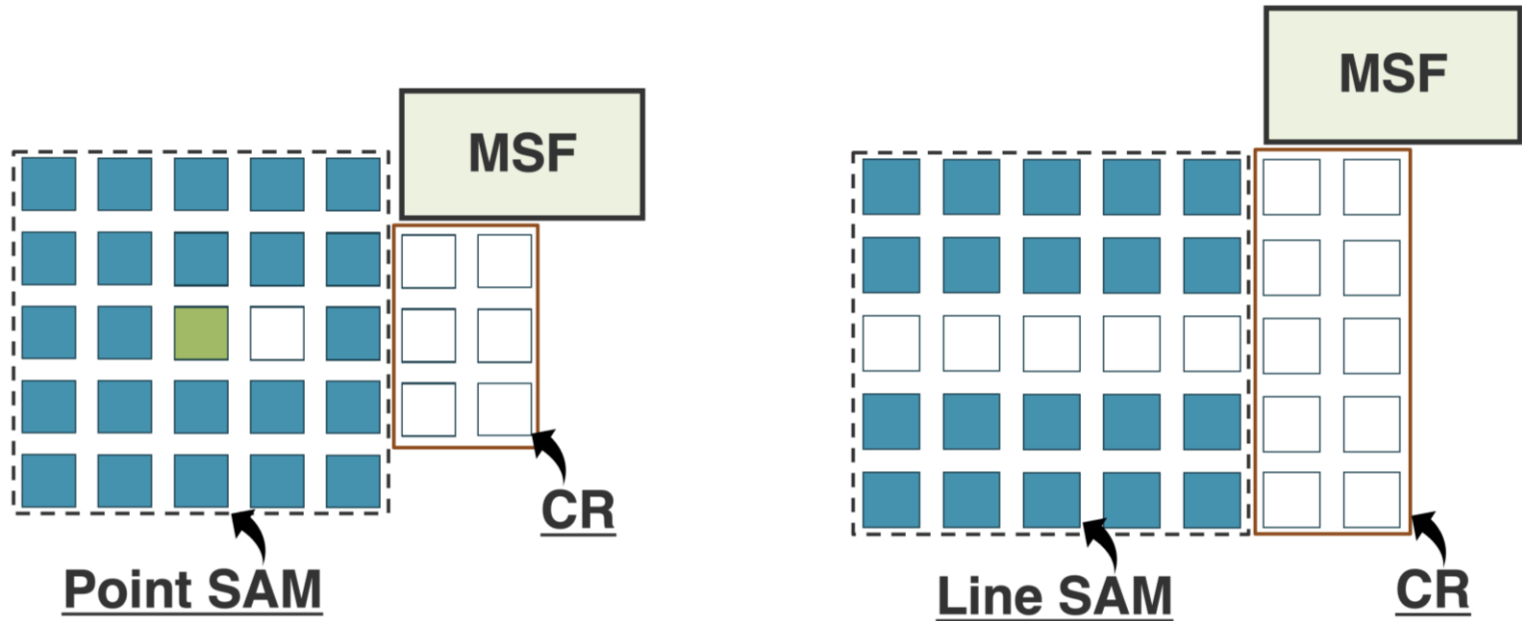
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



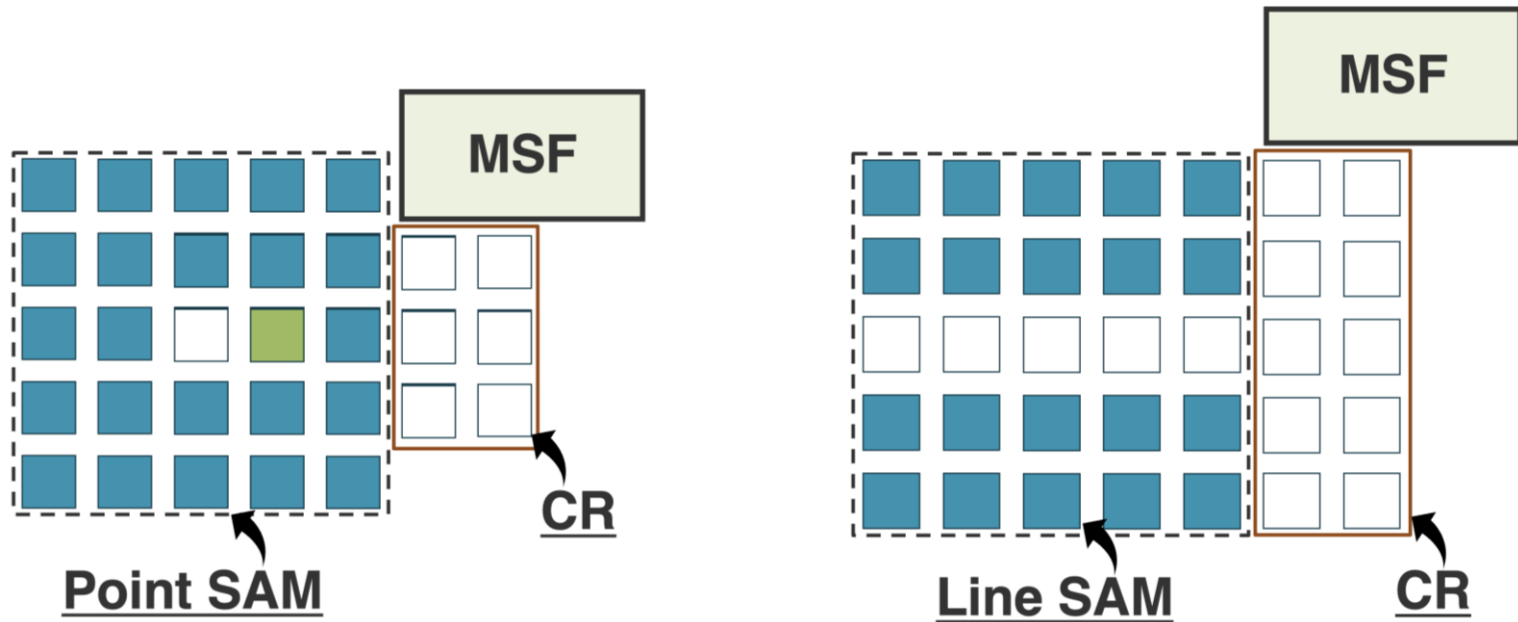
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



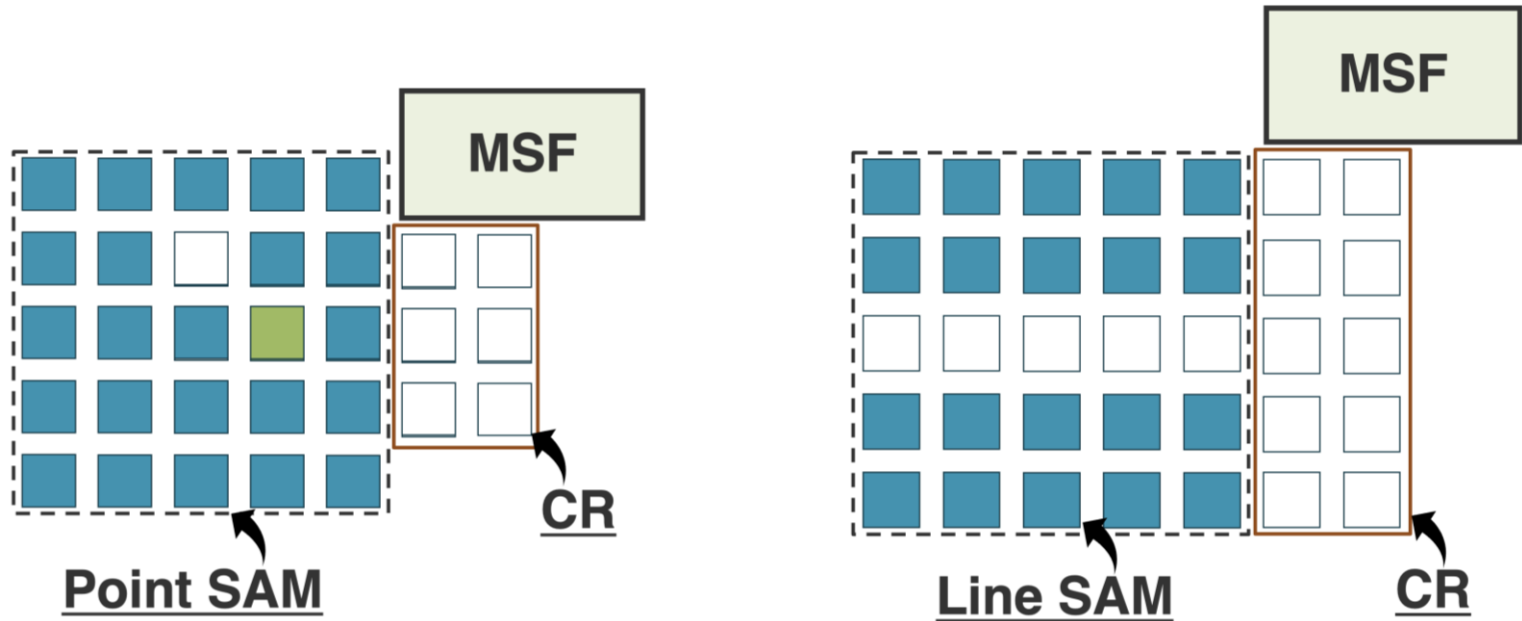
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



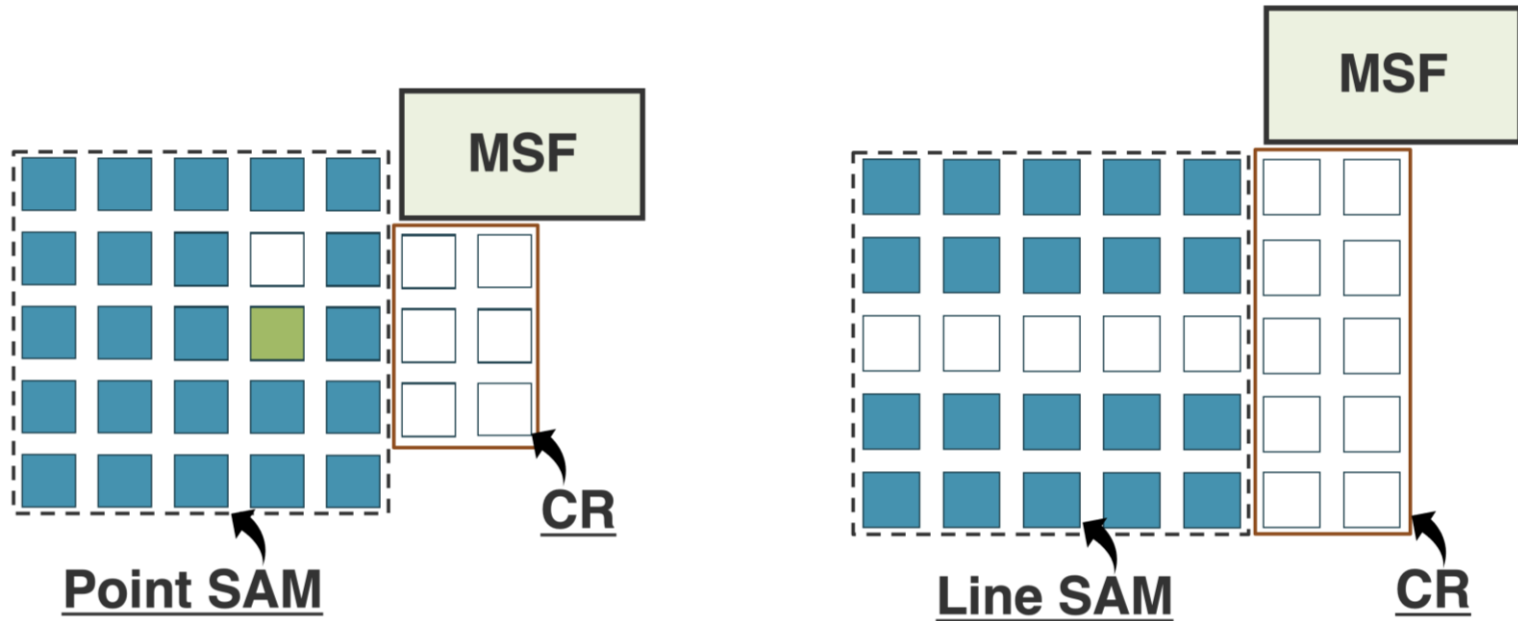
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



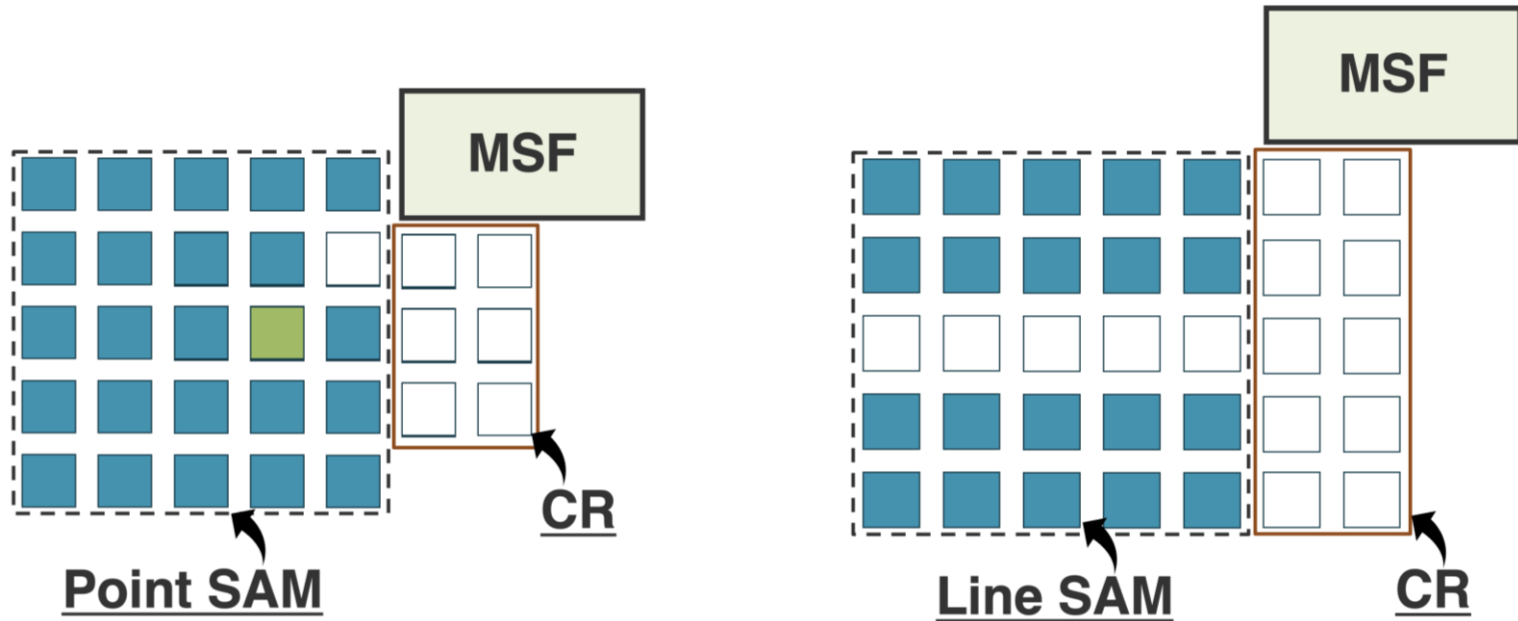
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



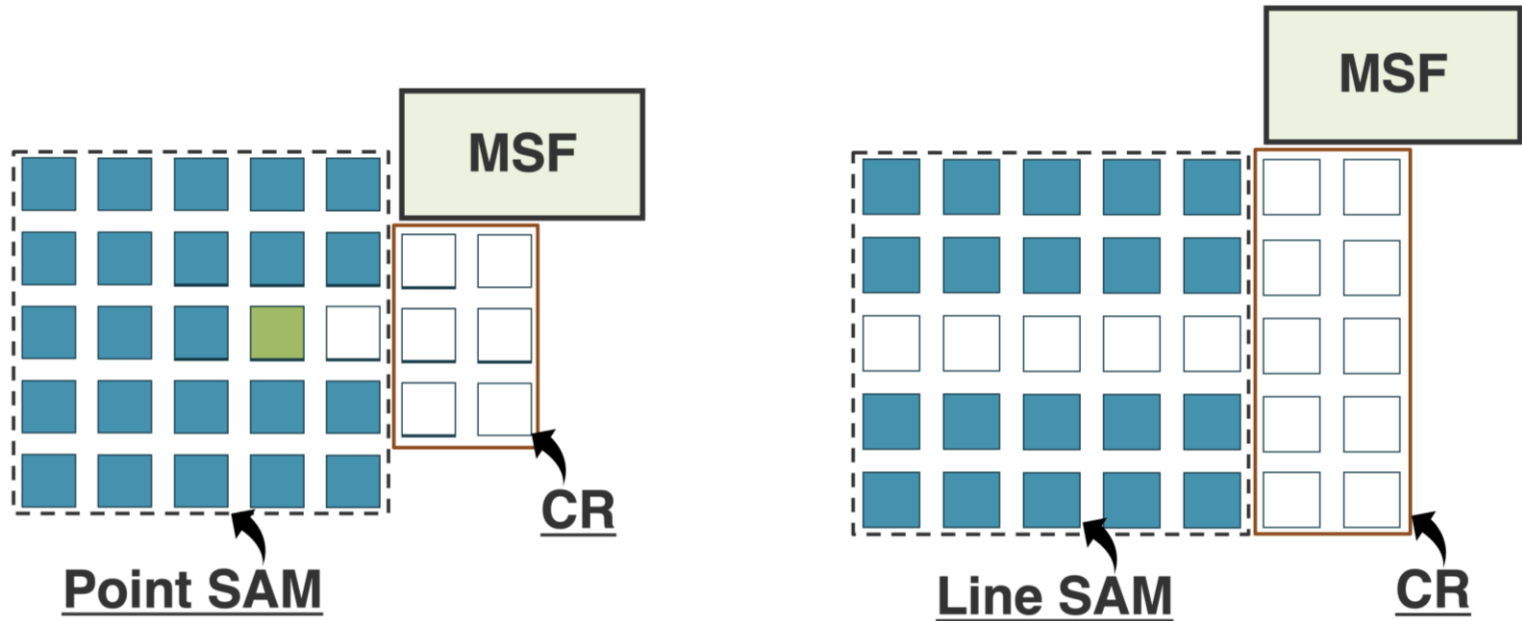
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



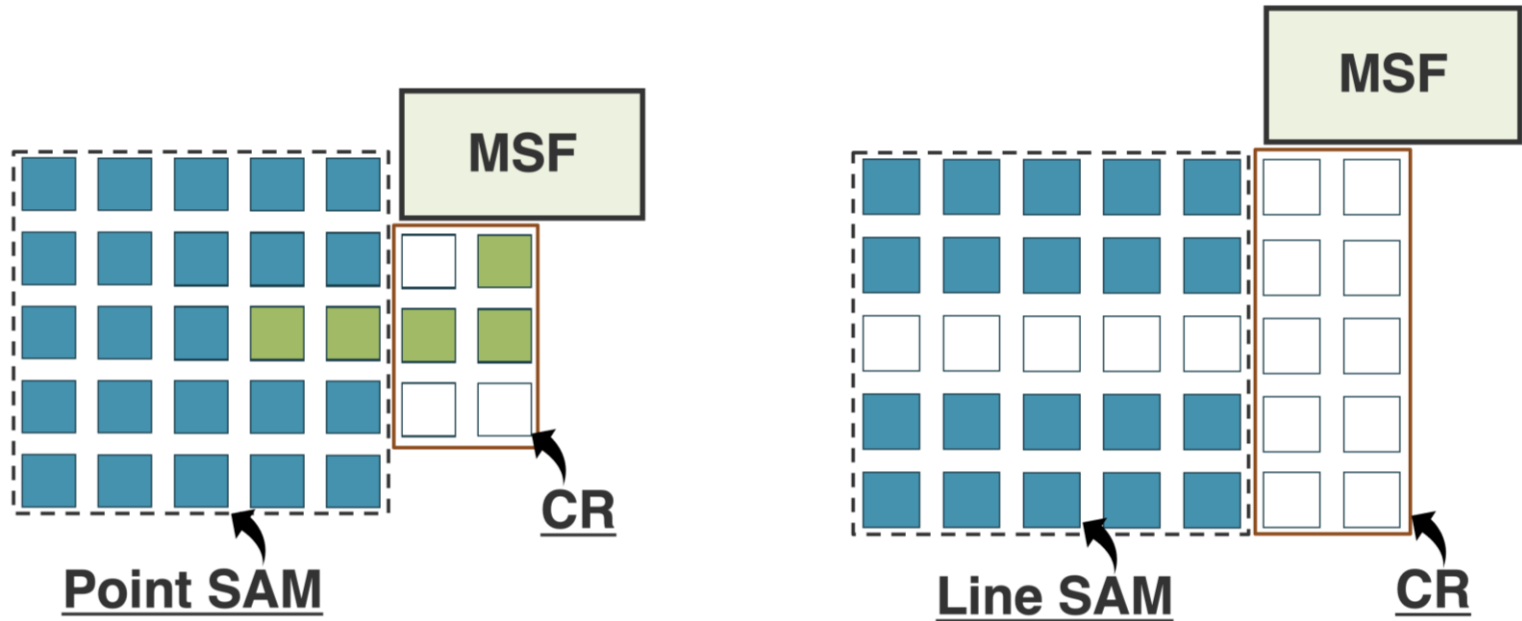
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



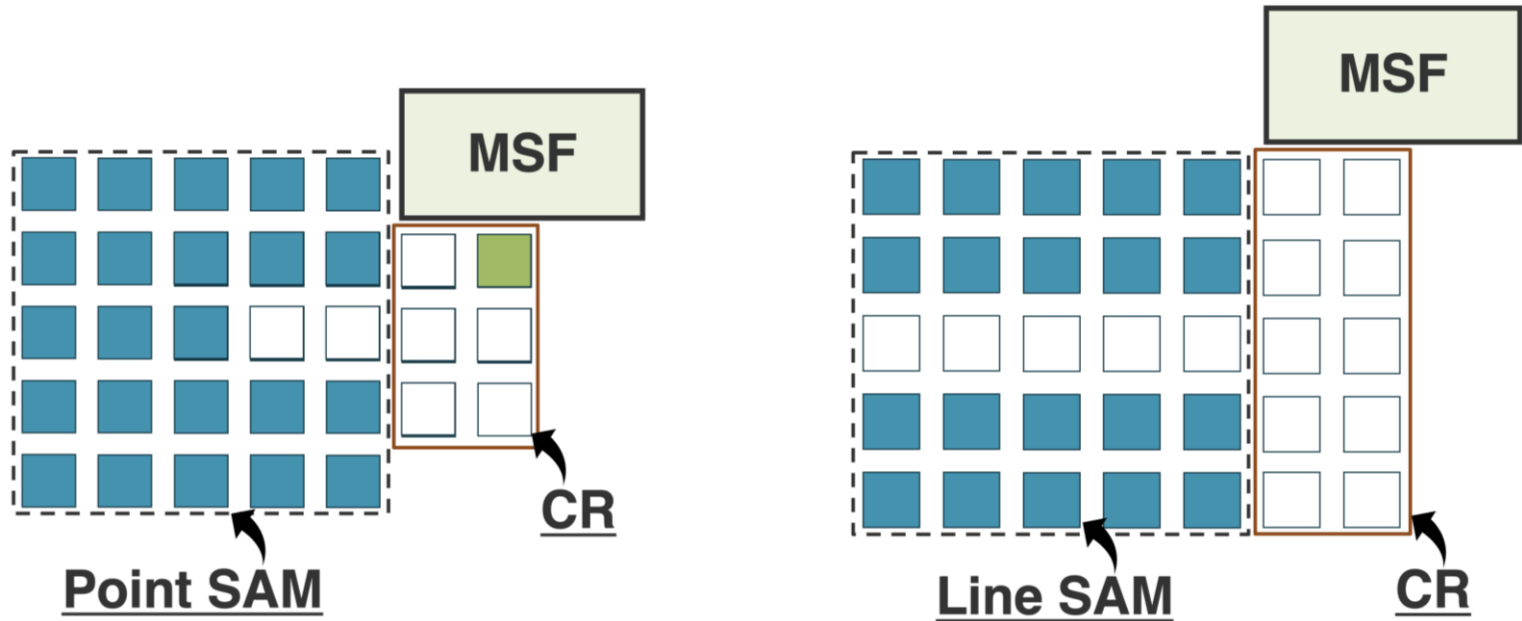
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



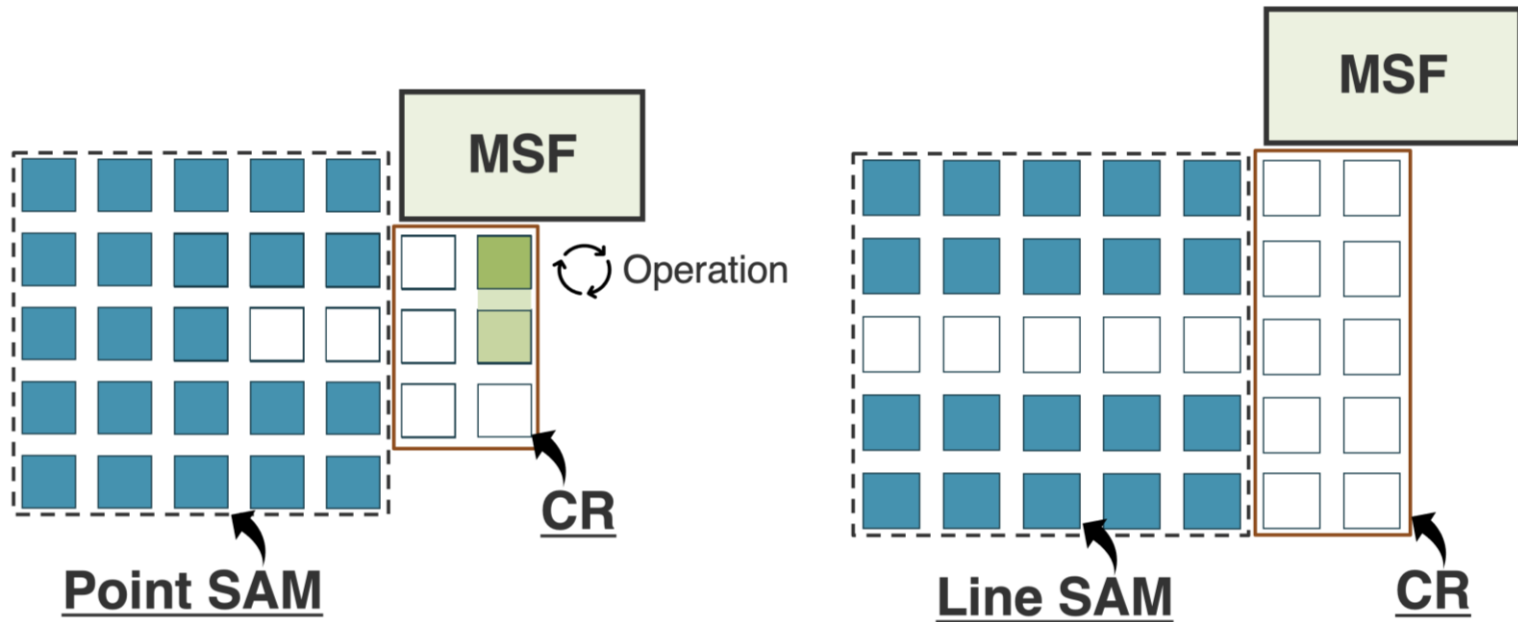
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



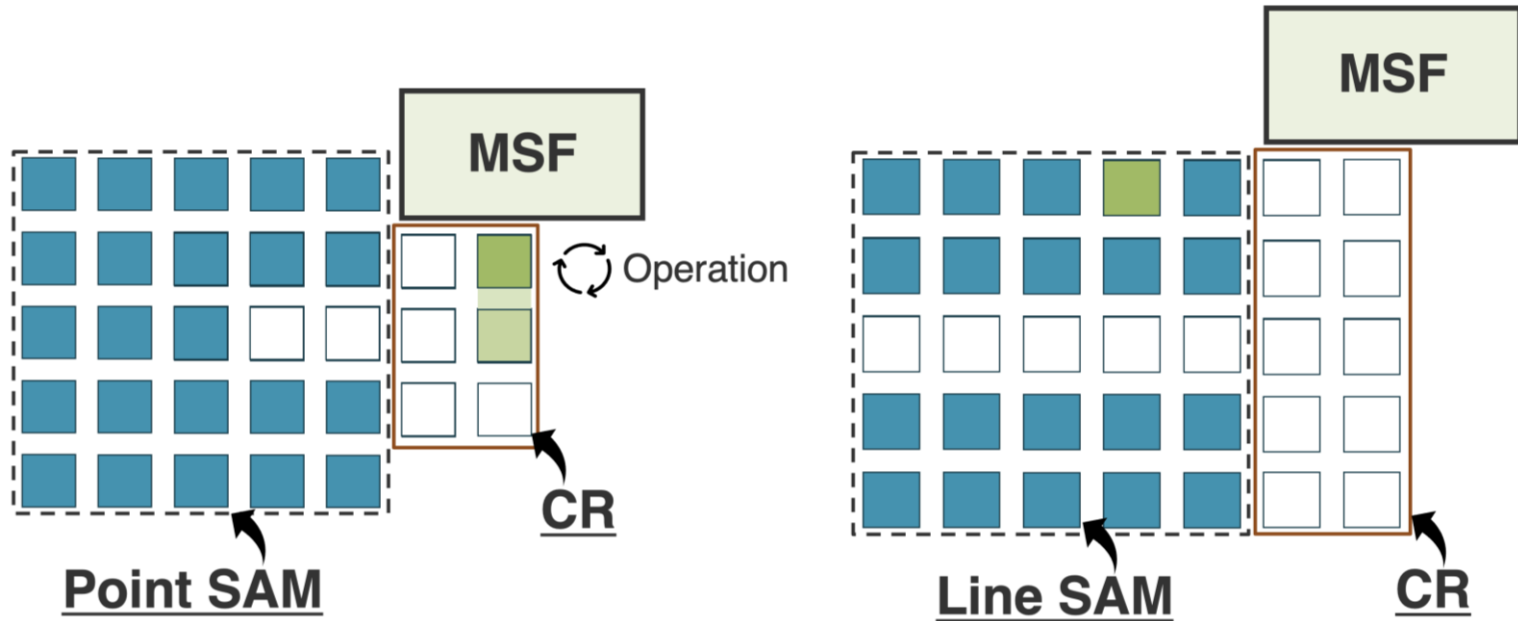
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



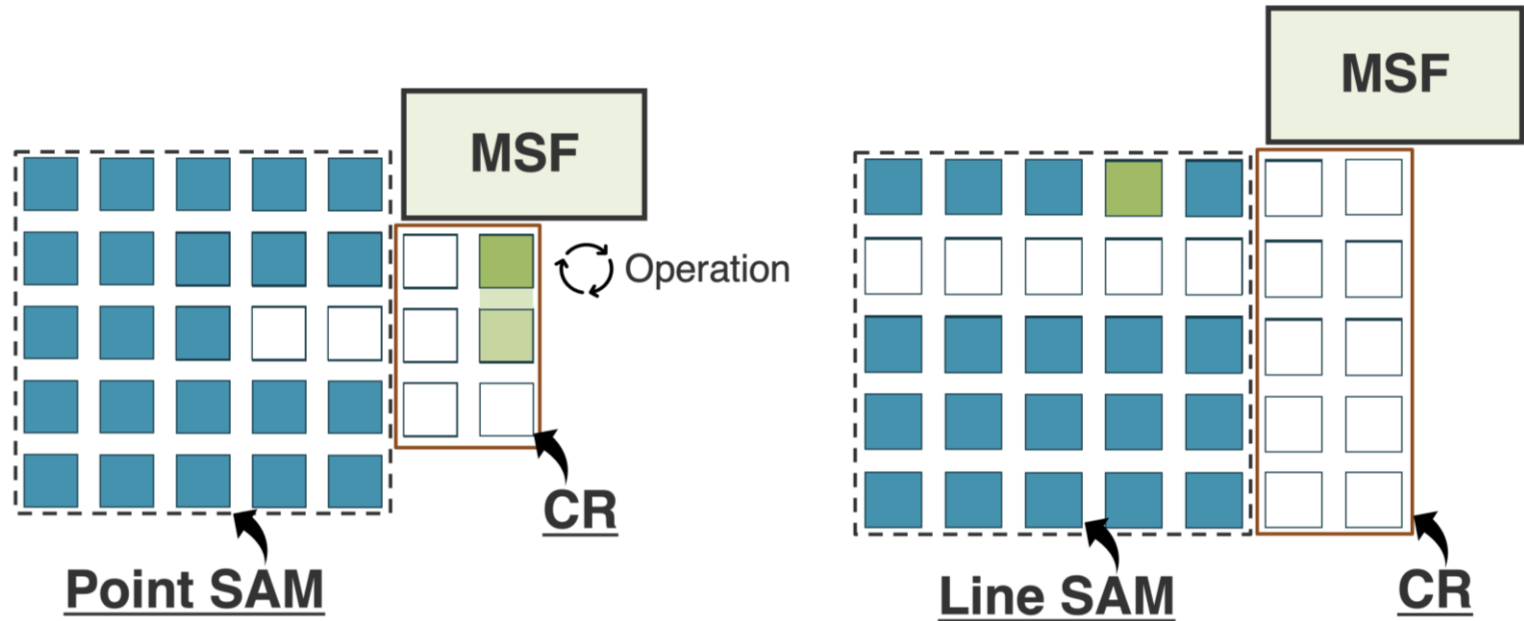
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



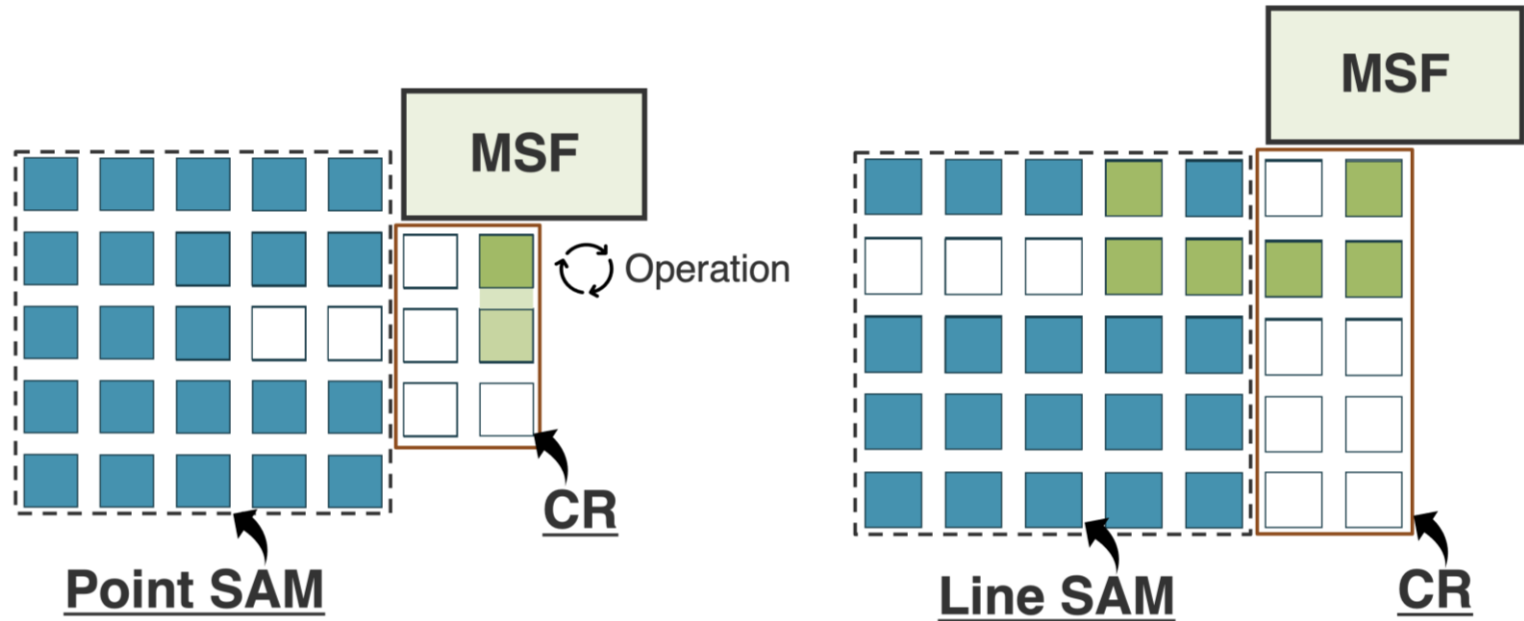
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



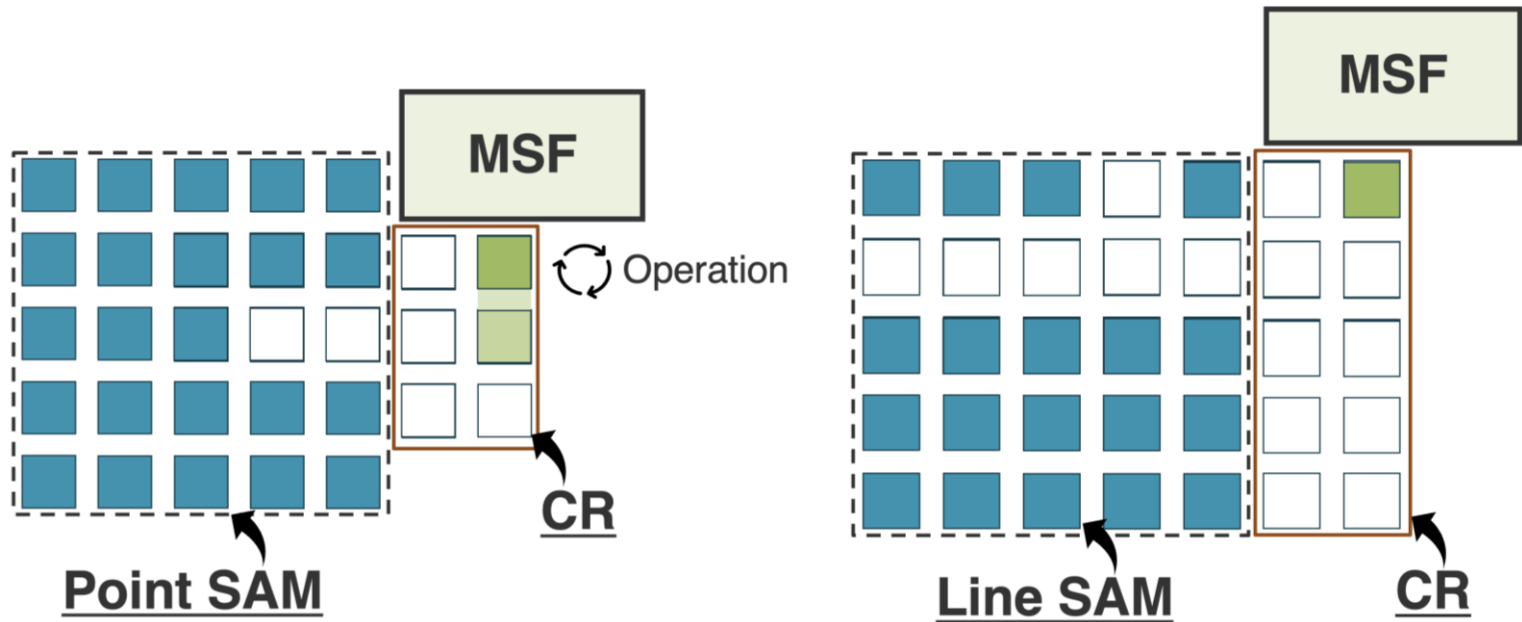
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



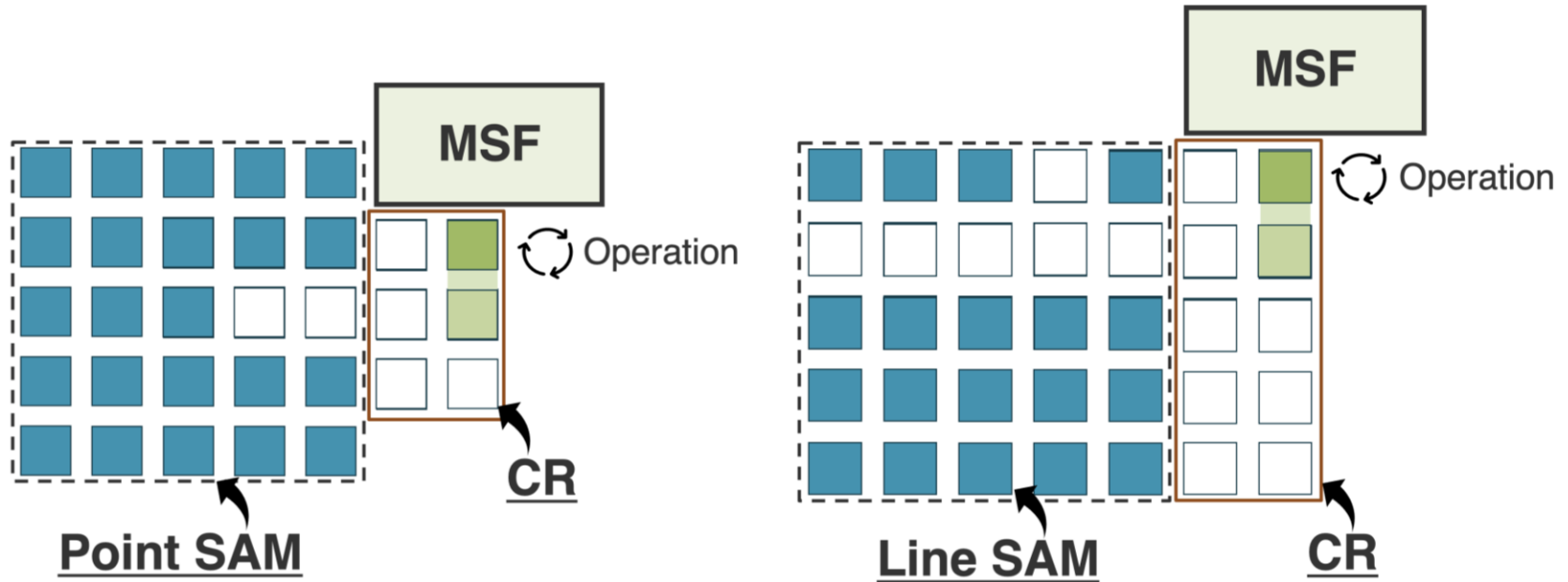
ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory



ロードストア型誤り耐性量子計算機アーキテクチャ

- Scan-Access Memory

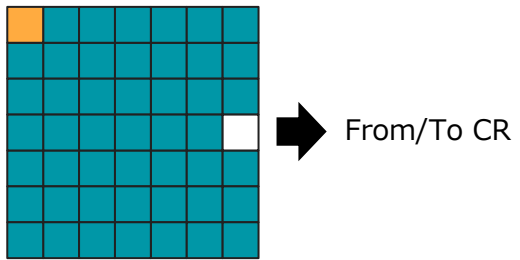
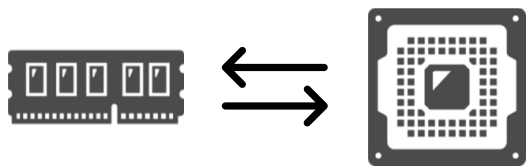


ロードストア型を採用するコスト

ロードストア型の欠点はメモリ-レジスタ間通信による計算の低速化

プログラムによっては通信の遅さがボトルネックに

最悪ケースでは最もアクセスの遅いセルが
繰り返し呼び出される形になる



通常の計算機では典型的プログラムが持つ偏りを活用し様々なテクニックでこれらの問題を解決

キャッシュ / 専用メモリ

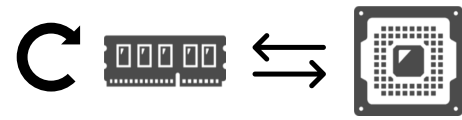
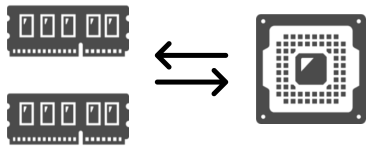
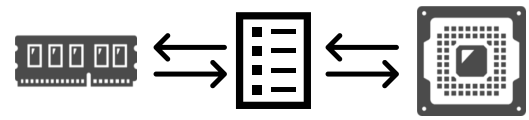
頻繁にアクセスするものや
アクセスが予測されるものを
中規模で高速なメモリに保存する

マルチチャンネルメモリ

複数のメモリバンクに分割し、
並列して読み出しを行うことで
実効的な読み出し速度を上げる

インメモリ計算

メモリ内部に軽量の計算機構を入れ
一部の計算をメモリ内部で行う



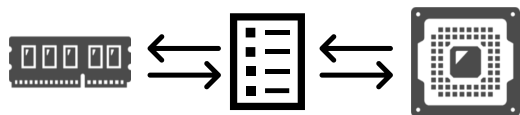
キャッシュなどの技術はメモリアクセスが典型的な特性を持っていることを仮定している 量子も同様の性質を持つか？

ロードストアの影響を軽減する最適化

最適化機構の提案

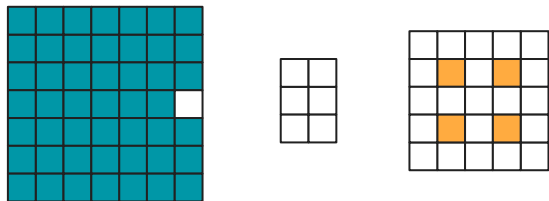
指針：キャッシュ/専用メモリ

頻繁にアクセスするものや
アクセスが予測されるものを
中規模で高速なメモリに保存する



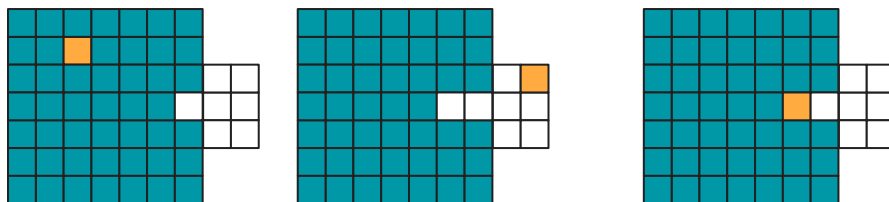
最適化2：Hybrid Floorplan

レジスタのサイズを拡張し、静的解析に基づいて
頻繁にアクセスされる少数のデータを専用領域に配置する

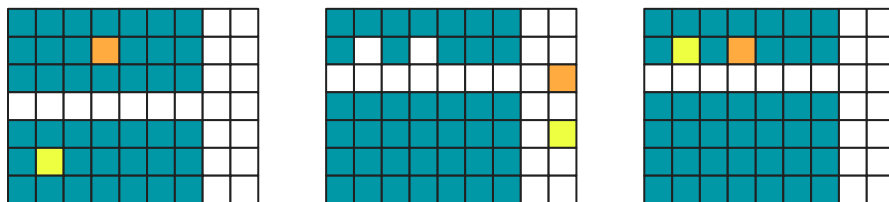


最適化1：Locality-aware store

利用したデータは元の場所ではなくアクセスの早い場所に戻す
これにより、プログラムの知識を用いずに再参照を早くする



同時に利用したデータは同時にアクセスしやすい場所に戻す
これにより、空間的な局所性を活用し典型的ロードを早くする

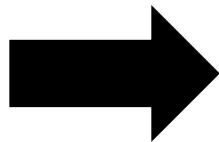
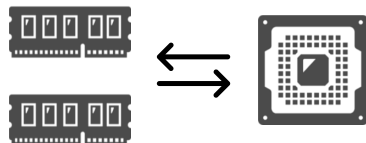


ロードストアの影響を軽減する最適化

最適化機構の提案

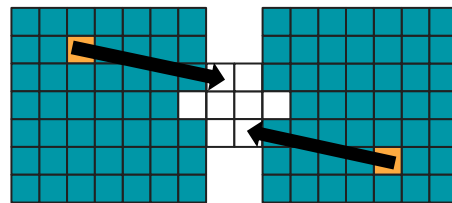
指針 2 : マルチチャンネルメモリ

複数のメモリバンクに分割し、
並列して読み出しを行うことで
実効的な読み出し速度を上げる



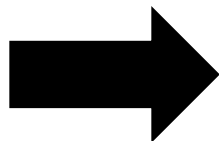
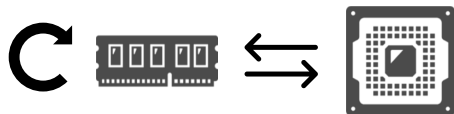
最適化 3 : Multi-bank SAM

データを複数のSAMに分割して配置し、
2並列でのLOAD/STOREを可能にする



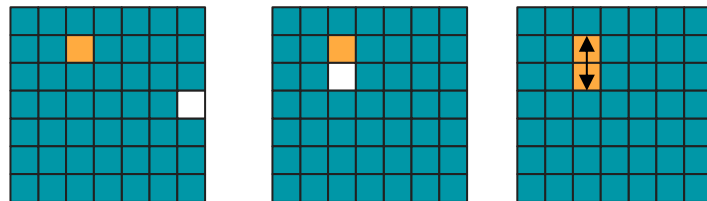
指針 3 : インメモリ計算

メモリ内部に軽量の計算機構を入れ
一部の計算をメモリ内部で行う



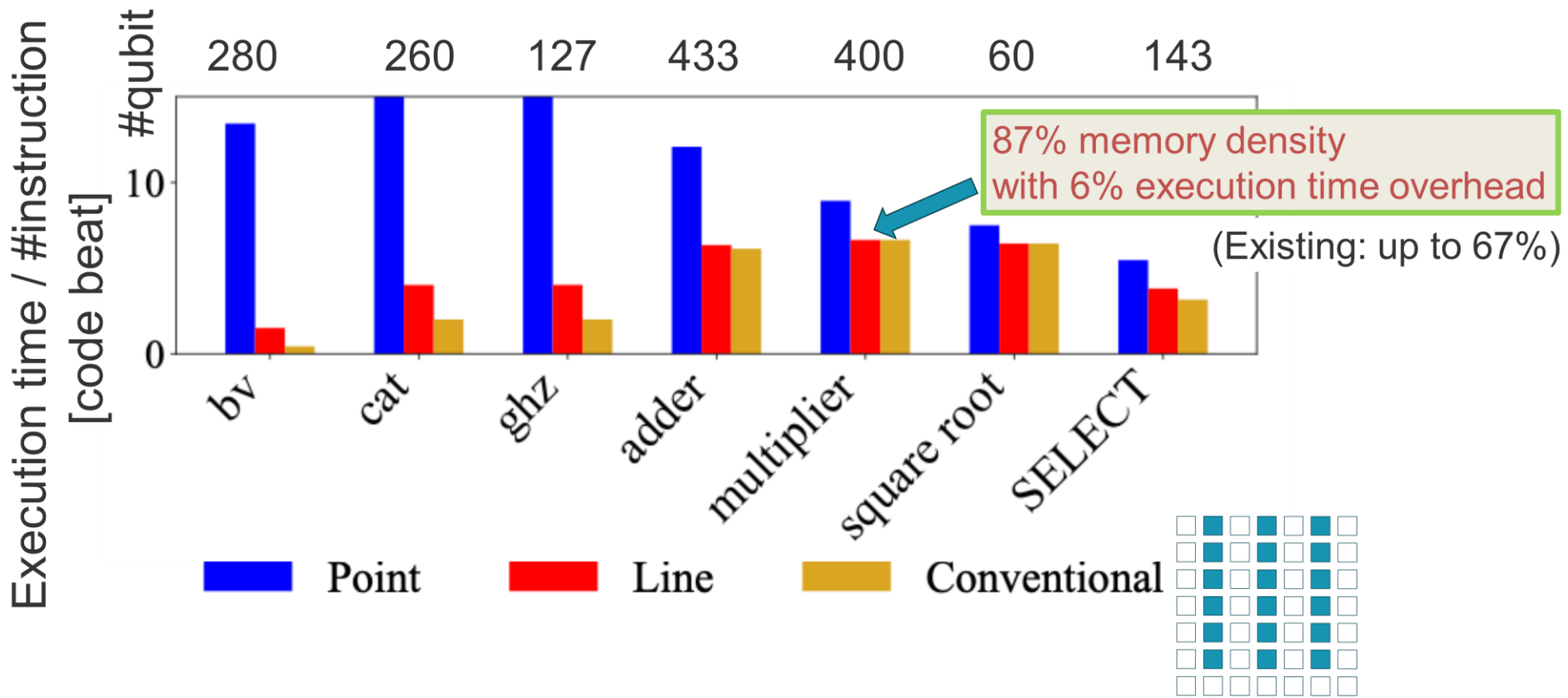
最適化 4 : In-memory operations

移動用の追加セルで完結できる論理操作は、
CRに読みださずにメモリ内部で完結させる

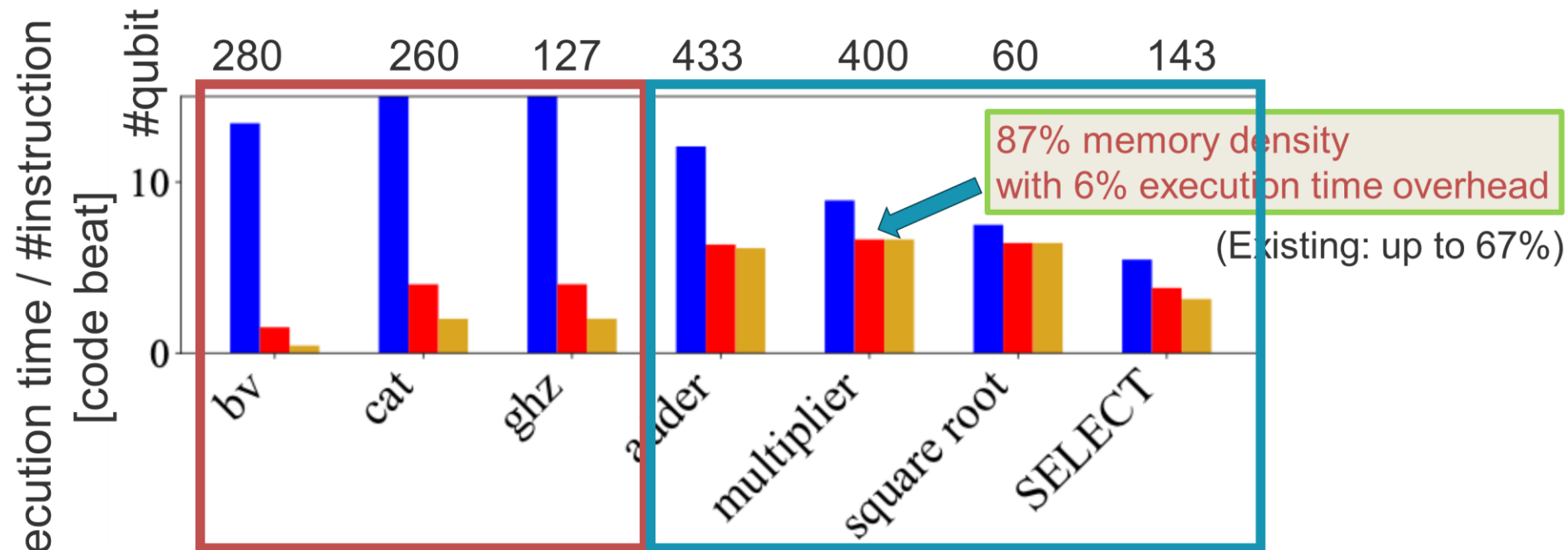


論理HゲートやSゲートをその場で実行

Basic evaluation for benchmark program



Basic evaluation for benchmark program



Large time overhead

- Less T-gates
- Not bottleneck in practice

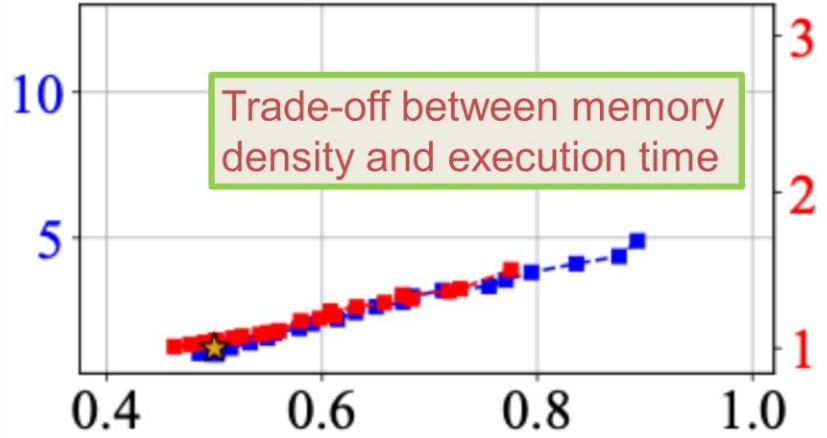
Small time overhead

- Low parallelism and many T-gates
- Bottleneck for many practical applications

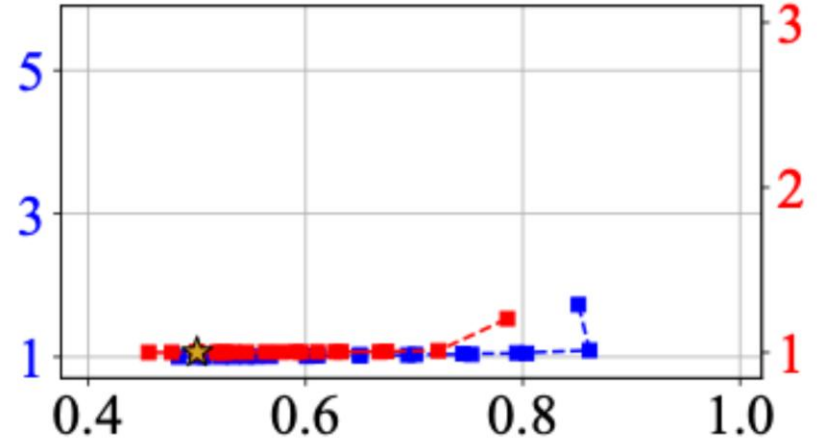
Hybrid floorplan for benchmark program

Execution time overhead

GEOMEAN



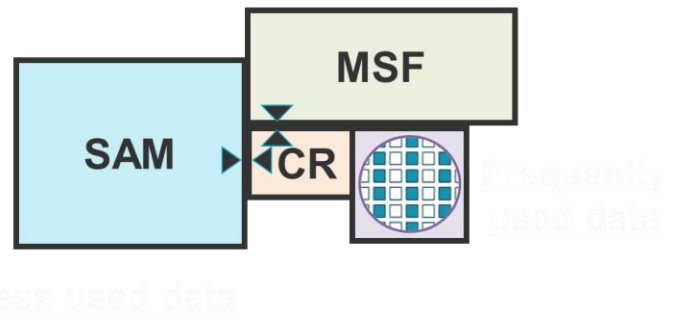
SELECT



Memory density

---●--- Point -.-.-.- Line ☆ Conventional

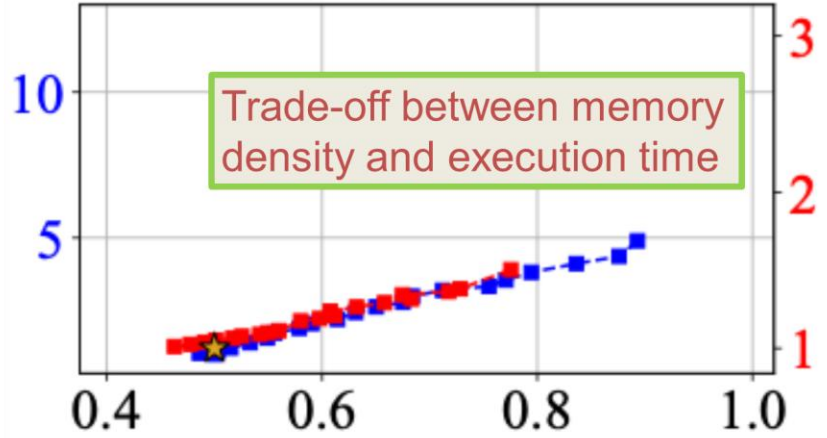
Each Points: how large the conventional floorplan in Hybrid floorplan (Left is larger)



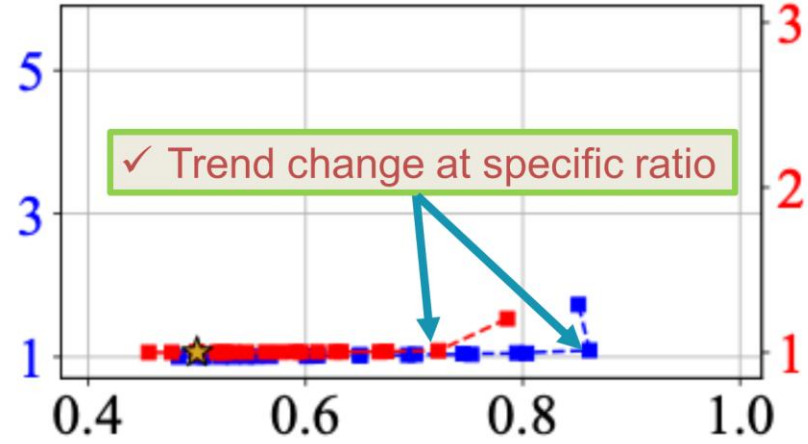
Hybrid floorplan for benchmark program

Execution time overhead

GEOMEAN



SELECT

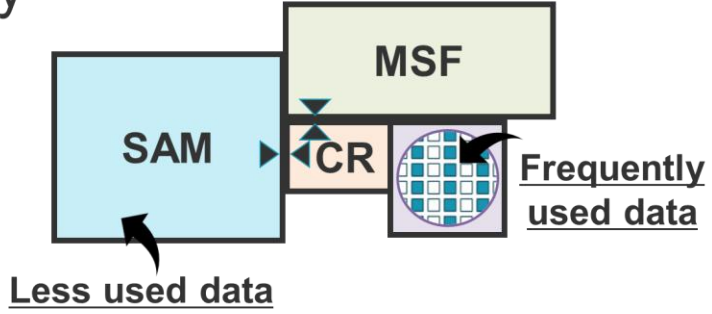


Memory density

---●--- Point -.-.-.- Line ☆ Conventional

Each Points: how large the conventional floorplan in Hybrid floorplan (Left is larger)

Utilize non-uniform access

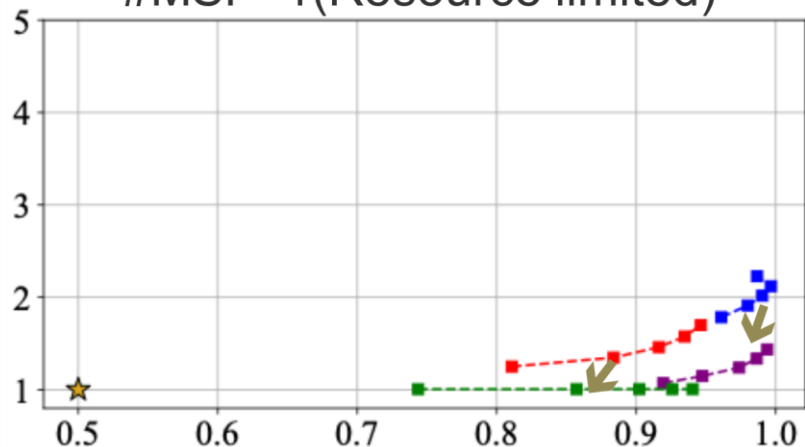


Optimized hybrid floorplan evaluation for SELECT circuit

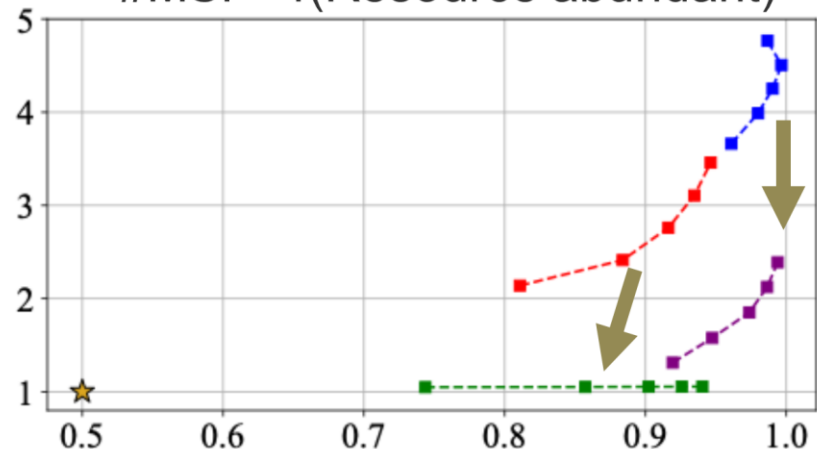
Execution time overhead

Each point: different #Logical qubits=467, 1711, 3753, 6595, 10235 (Right is larger)

#MSF=1(Resource limited)



#MSF=4(Resource abundant)



Memory density

--■-- Point --■-- Line --■-- Hybrid Point --■-- Hybrid Line ★ Conventional

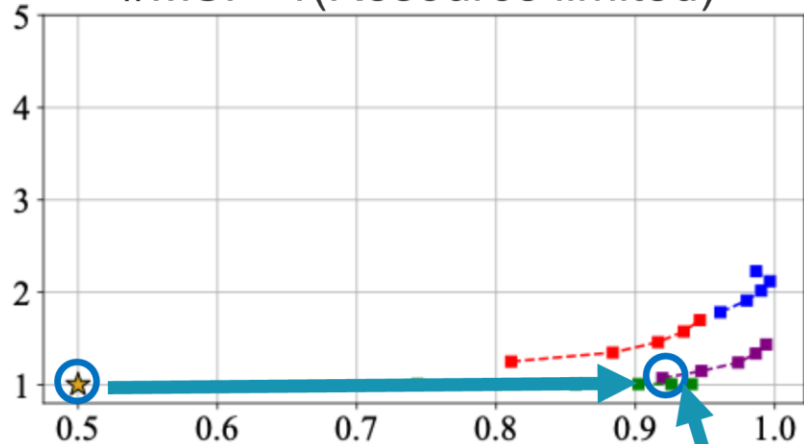
✓ Hybrid floorplan can reduce execution time by slightly decreasing memory density

Optimized hybrid floorplan evaluation for SELECT circuit

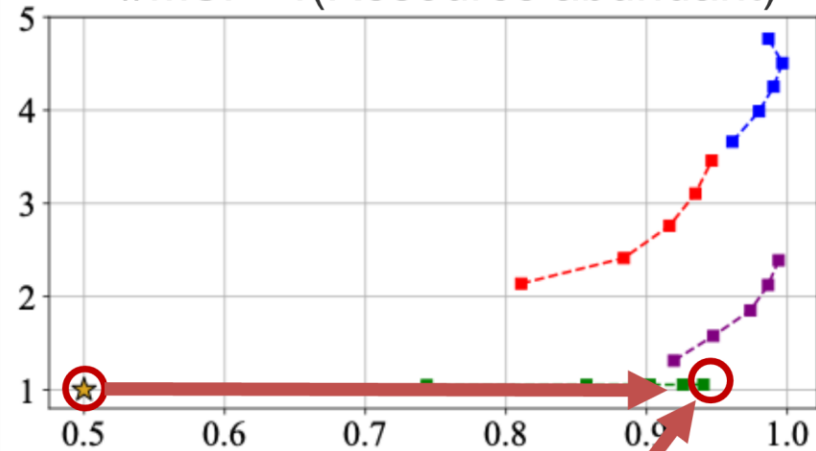
Execution time overhead

Each point: different #Logical qubits=467, 1711, 3753, 6595, 10235 (Right is larger)

#MSF=1(Resource limited)



#MSF=4(Resource abundant)



Memory density

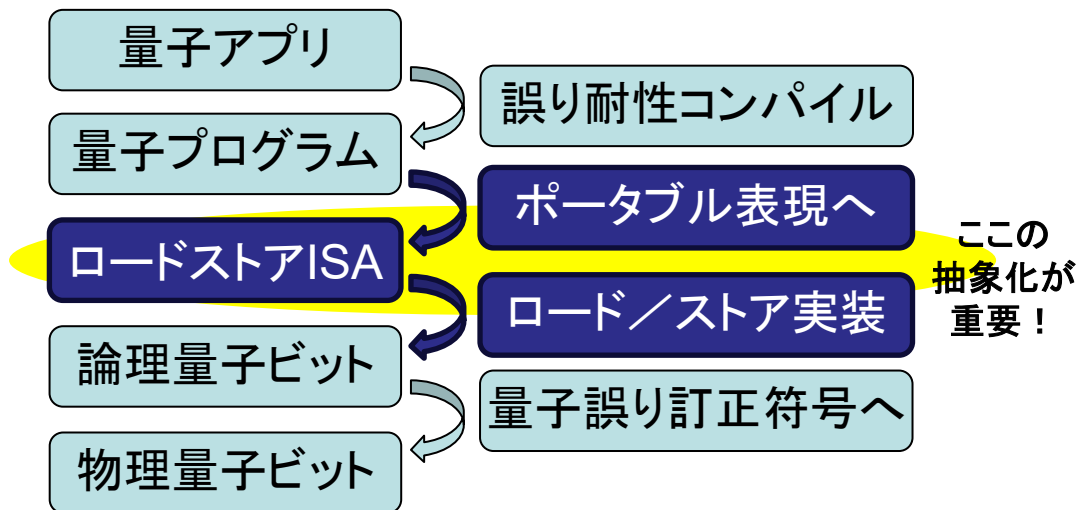
---■ Point ---■ Line ---■ Hybrid Point ---■ Hybrid Line ☆ Conventional

factory=1, instance size=21 → 92% memory density with 7% execution time overhead by Hybrid point-SAM architecture

factory=4, instance size=101 → 94% memory density with 6% execution time overhead by Hybrid line-SAM architecture

ロードストア型FTQCまとめ

- 高密度メモリ領域＋小さな演算領域の導入で空間効率を向上
 - メモリアクセスレイテンシ軽減手法による時間オーバヘッドの低減
- ロードストア型ISAによる抽象化でプログラムのポータビリティを向上



今日の内容

- 量子誤り訂正の基礎
 - 量子誤り訂正に求められる条件
 - 古典符号と量子符号
 - スタビライザ形式
 - Shorの符号
- 表面符号
 - 表面符号の誤り推定
 - Stim + Crumbleを用いた表面符号シミュレーション
 - 極低温環境での表面符号の誤り推定処理
- 表面符号 + 格子手術に基づく用いた誤り耐性量子計算
- まとめ

本日の内容まとめ

- スタビライザ形式：量子状態を「その状態を変えない演算子＝スタビライザ」の組で表す
- スタビライザ符号：複数の可換なスタビライザの固有値が全て+1になる状態を符号語として扱う量子誤り訂正符号
- 表面符号：実装上の素性の良さから有望視されている代表的な量子誤り訂正符号
- 格子手術：表面符号の変形および結合・分離により誤り耐性量子計算を行う手法